



Gold Line Drive Administration Manual

December 2012 (Ver. 1.100)



www.elmomc.com

Notice

This guide is delivered subject to the following conditions and restrictions:

- This guide contains proprietary information belonging to Elmo Motion Control Ltd. Such information is supplied solely for the purpose of assisting users of the Gold Line servo drive in its installation.
- The text and graphics included in this manual are for the purpose of illustration and reference only. The specifications on which they are based are subject to change without notice.
- Information in this document is subject to change without notice.



Elmo Motion Control and the Elmo Motion Control logo are registered trademarks of Elmo Motion Control Ltd.



EtherCAT Conformance Tested. EtherCAT® is a registered trademark and patented technology, licensed by Beckhoff Automation GmbH, Germany.

Document no. MAN-G-ADMING (Ver. 1.100)

Copyright © 2012

Elmo Motion Control Ltd.

All rights reserved.

Revision History

Version	Date	Changes
Ver. 1.001	May 2012	Initial Release
Ver. 1.002	Oct 2012	Update to chapter 2: Feedback Emulation
Ver. 1.100	Dec 2012	Update to include chapter 7 on The External Reference Generator

Elmo Worldwide

Head Office

Elmo Motion Control Ltd.

60 Amal St., P.O. Box 3078, Petach Tikva 49516
Israel

Tel: +972 (3) 929-2300 • Fax: +972 (3) 929-2322 • info-il@elmomc.com

North America

Elmo Motion Control Inc.

42 Technology Way, Nashua, NH 03060
USA

Tel: +1 (603) 821-9979 • Fax: +1 (603) 821-9943 • info-us@elmomc.com

Europe

Elmo Motion Control GmbH

Hermann-Schwer-Strasse 3, 78048 VS-Villingen
Germany

Tel: +49 (0) 7721-944 7120 • Fax: +49 (0) 7721-944 7130 • info-de@elmomc.com

China

Elmo Motion Control Technology (Shanghai) Co. Ltd.

Room 1414, Huawen Plaza, No. 999 Zhongshan West Road, Shanghai (200051)
China

Tel: +86-21-32516651 • Fax: +86-21-32516652 • info-asia@elmomc.com

Asia Pacific

Elmo Motion Control

#807, Kofomo Tower, 16-3, Sunae-dong, Bundang-gu, Seongnam-si, Gyeonggi-do,
South Korea

Tel: +82-31-698-2010 • Fax: +82-31-698-2013 • info-asia@elmomc.com

Table of Contents

Chapter 1: Introduction	8
1.1. Scope	8
1.2. Abbreviations.....	8
Chapter 2: Communication	10
Chapter 3: Sensors and Sockets.....	11
Chapter 4: Commutation.....	12
Chapter 5: Control and Filters	13
Chapter 6: Profilers and Stop Manager	14
Chapter 7: Enabling a Motor	15
Chapter 8: Fault Detection and Handling	16
Chapter 9: The Auxiliary Reference	17
Chapter 10: The Recorder and Development Aid	18
Chapter 11: Digital Inputs and Outputs	19
Chapter 12: Homing and Position Capture	20
12.1. Introduction.....	20
12.2. Position Capture-Related Commands	24
12.2.1. IL[N] Command	24
12.2.2. GI[N] Command	25
12.2.3. GX[N] Command	25
12.2.4. GY[N] Command	25
12.2.5. HM[N] Command.....	25
12.2.6. HF[N] Command	25
12.2.7. IF[N] Command.....	25
12.3. Elmo Homing and Capture Commands	25
12.3.1. HM[N] Examples	28
12.3.2. HF[N] Examples.....	29
12.4. DS 402 Touch Probe	31
12.4.1. Definition of Object 0x60B8	32
12.4.2. Definition of Object 0x60B9	33
12.4.3. Definition of Objects 0x60BA to 0x60BD	35
12.4.4. Definition of Object 0x2E10:.....	36
12.5. DS 402 Homing	40
12.6. PLCopen Homing on Block.....	43
12.7. Position Capture Notes and Limitations	46
12.8. Position Capture-Related Error Codes (EC)	46

Chapter 13: Output Compare	48
13.1. Introduction.....	48
13.2. Commands Related to Output Compare	49
13.2.1. OL[N] Command	49
13.2.2. GO[N] Command	49
13.2.3. GV[N]/GW[N] Commands.....	50
13.2.4. OC[N] Command.....	50
13.3. Supported Types of Output Compare	51
13.3.1. Absolute Position Table Compare Mode.....	51
13.3.2. Duration Time Table Compare Mode	54
13.3.3. Absolute Start Position + Delta Position Compare Mode.....	56
13.4. Output Compare Configuration.....	57
13.5. Output Compare Comments and Limitations	59
13.6. Minimum Time between Output Compare Pulses.....	61
13.7. Error Codes (EC) Related to Output Compare.....	62
13.8. Output Compare Examples.....	63
13.8.1. Absolute + Delta Output Compare Examples	63
13.8.2. Position Table-Based Output Compare Examples	65
13.8.3. Time Table-Based Output Compare Examples	67
Chapter 14: Feedback Emulation.....	69
14.1. Introduction.....	69
14.2. Commands Related to Feedback Emulation	70
14.2.1. OL[N] Command	70
14.2.2. GO[N] Command	70
14.2.3. CA[N] Command	71
14.2.4. EA[N] Command	71
14.3. Supported Types of Feedback Emulation.....	72
14.4. Feedback Emulation Configuration	76
14.5. Feedback Emulation Comments and Limitations.....	78
14.6. Feedback Emulation Runtime Errors.....	78
14.7. Feedback Emulation Error Codes (EC).....	79
14.8. Feedback Emulation Examples	80
14.8.1. Port A Quadrature Feedback Emulation to Pulse/Direction Example.....	80
14.8.2. Port B Quadrature Feedback Emulation to Up/Down Example	80
14.8.3. Port A BiSS Feedback Emulation to Quadrature Example	81
Chapter 15: Buffering	82
Chapter 16: Operating Gold Drives with the Gold Maestro (G-MAS).....	83
16.1. Scope and Purpose	83
16.2. General	83
16.3. Concept and Features.....	83
16.3.1. Compatibility	83
16.3.2. Motion Concept.....	83
16.3.3. User Program.....	84
16.3.4. Communication Concept	84
16.3.5. Interpreters.....	85

16.3.6.	CPU's and Memory	85
16.3.7.	Position Reference and Modulo Concept	85
16.3.8.	Indications	86
16.3.8.1.	LED Indications	86
16.3.8.2.	Motor Fault Indications	86
16.3.8.3.	Status Register Indications	88
16.3.9.	Feedbacks	89
16.3.10.	I/Os	90
16.3.10.1.	Digital Inputs	90
16.3.10.2.	Digital Outputs	91
16.3.10.3.	Analog input	91
16.3.10.4.	Mode of Operation, Unit Mode and Control Loops	91
16.3.10.5.	Units	92
16.3.10.6.	Current Versus Torque	93
16.3.10.7.	Standardization	93
16.4.	Terminology	94
16.4.1.	Command Table Entries	94
16.4.2.	Type/Size	94
16.4.3.	Range/Unit	95
16.4.4.	General	96
Chapter 17: Error Correction		97
17.1.	Introduction	97
17.2.	Correction Table Structure	98
17.3.	Commands Related to Error Mapping	99
17.3.1.	The PC[N] Command	100
17.3.2.	The GP[N] Command	100
17.4.	Error Mapping Configuration	100
17.5.	Error Mapping with Modulo	101
17.6.	Error Mapping Comments and Limitations	102
17.7.	Error Codes (EC) Related to Error Mapping	103
17.8.	Error Mapping Examples	104
Chapter 18: The External Reference Generator		106
18.1.	Introduction	106
18.2.	Commands Related to External Reference Generator	107
18.2.1.	EM[N] command	Error! Bookmark not defined.
18.2.2.	ET[N] command	Error! Bookmark not defined.
18.2.3.	EI command	Error! Bookmark not defined.
18.2.4.	RM command	Error! Bookmark not defined.
18.2.5.	CA[N] command	Error! Bookmark not defined.
18.2.6.	YM[N] command	Error! Bookmark not defined.
18.2.7.	KV[N] command	Error! Bookmark not defined.
18.3.	Follower	109
18.4.	ECAM	111
18.4.1.	ECAM table types	111
18.4.1.1.	Constant master gap table (EM[11]:bit2=0)	112

18.4.1.2.	Non-Constant Master Gap Table (EM[11]:bit2=1)	113
18.4.1.3.	Table index search (EM[11]:bit5)	114
18.4.2.	ECAM table interpolation	115
18.4.2.1.	Linear interpolation (EM[11]:bit4=1).....	115
18.4.2.2.	Quadratic interpolation (EM[11]:bit4=0)	116
18.4.3.	Non-periodical ECAM (EM[1]=1)	117
18.4.4.	Periodical ECAM (EM[1]=2)	118
18.5.	Speed and Acceleration Output Filtering	120
18.6.	External Reference Generator Initialization.....	120
18.7.	Engage/Disengage	121
18.7.1.	Engage/Disengage by RM Command	121
18.7.2.	Engage/Disengage by Digital Inputs	123
18.7.3.	Engage/Disengage by Ratio	124
18.8.	Jump-Free Motor Starting Rules.....	125
Chapter 19:	Troubleshooting.....	126

Chapter 1: Introduction

This Administration Guide is created to explain and illustrate the Commands detailed in the Gold Line Command Reference Guide. It actually consists of a considerable number of chapters which are continuously updated, and the manual is gradually developing in stages, building chapter by chapter, reviewed carefully for relevance of its material.

Therefore, at this stage the manual only consists of the following chapters:

- Chapter 1: Introduction
- Chapter 2: Feedback Emulation
- Chapter 3: Output Compare
- Chapter 4: Gold Maestro (G-MAS)
- Chapter 5: Homing and Position Capture
- Chapter 6: Error Correction
- Chapter 7: The External Reference Generator

1.1. Scope

The following describes the document scope and other sections of this chapter to be written:

Introduction to Gold drive: include general description of the drive as part of the solution (EAS & GMAS). Will include description of the main functionalities (profilers, control, feedbacks, user program, special functions e.g. output compare emulation) , some words about compatibility and general FW structure. The intro chapter will include terms that appear later on in detail

1.2. Abbreviations

The following abbreviations are used in this document:

Term	Explanation
Download	Transfer of data from the host to the drive
DSP	Digital signal processor
EDS	Electronic data sheet. The list of CAN objects supported by a device, in a form suitable for standard configuration software
IDE	Integrated development environment
PDO	Process data object. A CAN message type, which eliminates the need for



	allocation the data payload for object addressing by pre-agreement concerning the message contents (PDO mapping)
Upload	Transfer of data from the drive to the host

Chapter 2: Communication

The following describes the sections of this chapter to be written:

Some words about communication, languages and supported stacks (CAN, ECAT ,Ethernet) with addressing to the relevant documents.

Here we will add some words about CANopen objects vs Elmo's legacy command.

And some more terms relevant to interpreters, EtherCAT, CANopen - JUST A GLANCE.

Chapter 3: Sensors and Sockets

The following describes the sections of this chapter to be written:

Supported sensors, concept of sockets, selection of sockets, function of sockets relative to the function of sensors.

What can work with what and the 3 exist ports (Port A, Port B and the special Port B) and their functionality.

Chapter 4: Commutation

The following describes the sections of this chapter to be written:

All the commutation modes and basic concept. Pro & cons when to use what and some basic troubleshooting, lose of commutation and how to detect that we are in commutation.

Chapter 5: Control and Filters

The following describes the sections of this chapter to be written:

Torque, velocity and position structure and optional filters.

Some sub chapter about tuning, band width and control abilities (Step vs sine sweep, various FF, gain scheduling, scheduled filters, Feed forward, phase advance, gain limiter...).

How to program the filters, how to check the filter settings. Units.

Differences between motors: brush, brushless, linear..

Chapter 6: Profilers and Stop Manager

The following describes the sections of this chapter to be written:

Structure of profilers, smoother and stop manager, Elmo Vs DS402 differences, legacy Unit.

Modes with respect to the profiler activation, relevant commands setting...

Chapter 7: Enabling a Motor

The following describes the sections of this chapter to be written:

Enabling via Elmo, CANopen state machine, the commutation, brake activation during motor on and during motor off, indications of the motor enable (the MO vs the SO) ...

Chapter 8: Fault Detection and Handling

The following describes the sections of this chapter to be written:

Possible faults (communication, motion, profilers, sensors) and relevant commands that assist to detect the fault.

Different of fault detection in different communication (Elmo, CANopen, ECAT...)

Chapter 9: The Auxiliary Reference

The following describes the sections of this chapter to be written:

Structure, differences of the auxiliary inputs (analog, socket reference vs ECAM, CAN encoder).

Setup, use of filters engage disengage of auxiliary reference, use of ECAM different options, emulating ECAM reference (virtual profiler)



Chapter 10: The Recorder and Development Aid

The following describes the sections of this chapter to be written:

Relevant command and usage mainly via the EAS.

Recorder via user program.

Recording user program parameters.

How the recorder and other option assist the user to detect problems and debug his system

Chapter 11: Digital Inputs and Outputs

The following describes the sections of this chapter to be written:

Settings, the functions, logic levels and usage (user program, CANopen events etc)

Chapter 12: Homing and Position Capture

12.1. Introduction

This chapter describes the method by which a Gold drive captures position and sets the position according to the user requirements.

Capture is the ability of the drive to obtain the absolute position with respect to an event. The possible events are typically an index signal and/or a strobe signal (home input). The strobe signal is derived from any of the general-purpose digital inputs.

The index signal is either a pulse that is received from a feedback (quad or sine/cosine analog) or an emulated signal from an analog feedback (resolver, analog Halls). Emulation is achieved either by emulating the signal from analog to AQB & index (analog Halls, digital sensors) or by firmware emulation such as the index in a resolver.

Homing is the ability of the drive to modify its original position to a predefined position with or without motion. During the homing process, the drive adjusts all internal variables, making sure that no jump in motion occurs due to the position change. Homing is performed according to a hardware signal (such as a captured signal), other inputs, such as a limit switch and a general-purpose digital input, or software signals. Another homing method is immediate homing, which allows setting the position without a hardware trigger.

The following describes the sections of this chapter to be written:

The Homing procedure: Elmo legacy and improvements vs CANopen?

Capture signals

There are several options in the form of signals for triggering capture (and homing) events. The following subsections detail these signals.

Software signals

Software signals are typically used for immediate homing. A software signal arrives via a communication channel, and it is used in the Elmo homing mode and in the DS 402 homing mode. By signaling the drive, a host requests the drive to perform homing without any physical signal. All the optional communication channels are valid: CANopen, EtherCAT, USB, UDP and User Program. Refer to **HM[]/HF[]** and DS 402 homing mode for more details.

Fast input signals

Signals which are defined as fast inputs are signals that are routed to the hardware with a response time of <5 µsec. These types of inputs include the following:

- Any general-purpose input that is routed by the **GI[]** command to the index or strobe signal of the Port A or Port B feedback inputs.
- Any motor index pulse that is routed to the Port A or Port B feedback inputs.

Fast inputs can be used in all capture and homing modes. Refer to Figure 17.

In the case of the Elmo homing mode and the DS 402 homing mode, the selected “Home” input must be defined using the **IL[]** command. For more information, see the **HM[]** examples in Section 5.3.1.

Note that unlike the SimplIQ servo drives, where only Input 5 and Input 6 can be used as home inputs, in a Gold drive any general-purpose input can be used as the home signal.

The minimum time between captures for a fast input is $2 \cdot TS$ (the default is 100 μ sec, which is equivalent to a frequency of 10 kHz).

The “active high” or “active low” logic of the fast input is specified by bit 0 of the **IL[]** command.

Fast inputs are not subject to **IF[]** filtering.

Software-sensed input signals

- Any general-purpose input which is used for homing and is not routed to the hardware channel by **GI[]** can be used as a software-sensed INPUT.
- Software-sensed signals are not available for the touch probe function (refer to Figure 17), but are subject to **IF[]** filtering.
- The logic level (“active high” or “active low”) is determined by the **IL[]** command.
- The response time of the software-sensed signal is $>250 \mu$ sec plus the input filter time, which is defined in **IF[]**.
- The time between captures (the capture frequency) is 250μ sec + **IF[]** (the filter time).

The following three drive modules use position capture options:

1. **Elmo’s homing modes** - These legacy homing modes, which are activated with **HM[]** or **HF[]** (the SimplIQ **HY[]** has been replaced by **HF[]**, allowing homing on any socket), are a group of commands that enable the user to build any desired homing sequence. The Elmo homing methods can be activated in any motion mode, and they do not confine the user to a specific motion mode. Elmo homing enables the user to:
 - a. Select the homing trigger from any digital input, strobe, index or limit switch.
 - b. Capture the present actual position without modifying the position origin.
 - c. Capture the present position while modifying the actual position for a new origin from the present position.
 - d. Capture the present position while modifying the actual position from a new origin as an absolute position.
 - e. Select a capture buffer that allows up to 2048 captured positions to be stored or, alternatively, an endless non-stored position.
 - f. Select an optional after-event functionality, namely, continue the motion or stop the motion, or set a digital output that will be generated when homing is completed.
 - g. Trigger a user program’s auto routine when homing is completed.
 - h. Trigger the procedure of either the main position socket or any other selectable socket.

2. **DS 402 homing methods** – These homing methods are defined by the CANopen standard. The standard allows selecting among 36 different homing methods, which already define the motion. These standard homing methods are activated when the DS 402 operating mode is set using object 0x6060. The homing methods are built from predefined sequences, where the user uses CANopen objects to select the following parameters:
 - i. The homing method, which is one of 36 homing sequences (object 0x6098).
 - j. The homing acceleration, which is used for deceleration as well (object 0x609A).
 - k. The homing fast speed, which is typically used to travel to the limit switch (object 0x6099[1]).
 - l. The homing slow speed, which is typically used to travel to the homing trigger, which is the strobe (home input), motor index or a combination of the two (object 0x6099[2]).
 - m. The homing offset, which specifies the offset from the home signal to the zero (datum) of the machine (object 0x607C).

DS 402 homing defines the motion to be performed. When one of the homing methods is selected (for example, go to the reverse limit switch (RLS) and then reverse direction and home to the first Index after the RLS resets to 0), the motion mode is also described. The drive gets into **jogging mode** (either a speed control loop or a position control loop) and travels according to the required sequence. When homing is completed, the homing offset is set to the homing position, and the software position limits are modified accordingly.
3. **PLCopen homing in block mode.** In this mode, homing is performed against a physical wall that mechanically blocks movement. In this homing mode, there is no limit switch or reference pulse. Adequate torque limits are required in order not to damage the mechanical devices during the homing process. The block conditions are that the pre-defined torque limit has been reached and that the real velocity has fallen below 5% of requested velocity. Block mode does not actually set the position, but it stops the motion when the conditions are fulfilled. The homing setting is typically achieved by either commanding homing to index or commanding immediate home.
4. **DS 402 Touch probe mode.** DS 402 defines the functionality of a touch probe. A touch probe is a function that can capture up to two inputs on the main position feedback sensor. The capturing can be performed on both the rising and falling edges. The touch probe functionality can be switched on and off under any motion mode (except for the DS 402 homing motion mode). Touch probe captures the rising edge before the falling edge. This means that if the input is 1 and the touch is required on a falling edge (from 1 to 0) the first falling edge will be ignored, and the capture will be performed on the next falling edge.

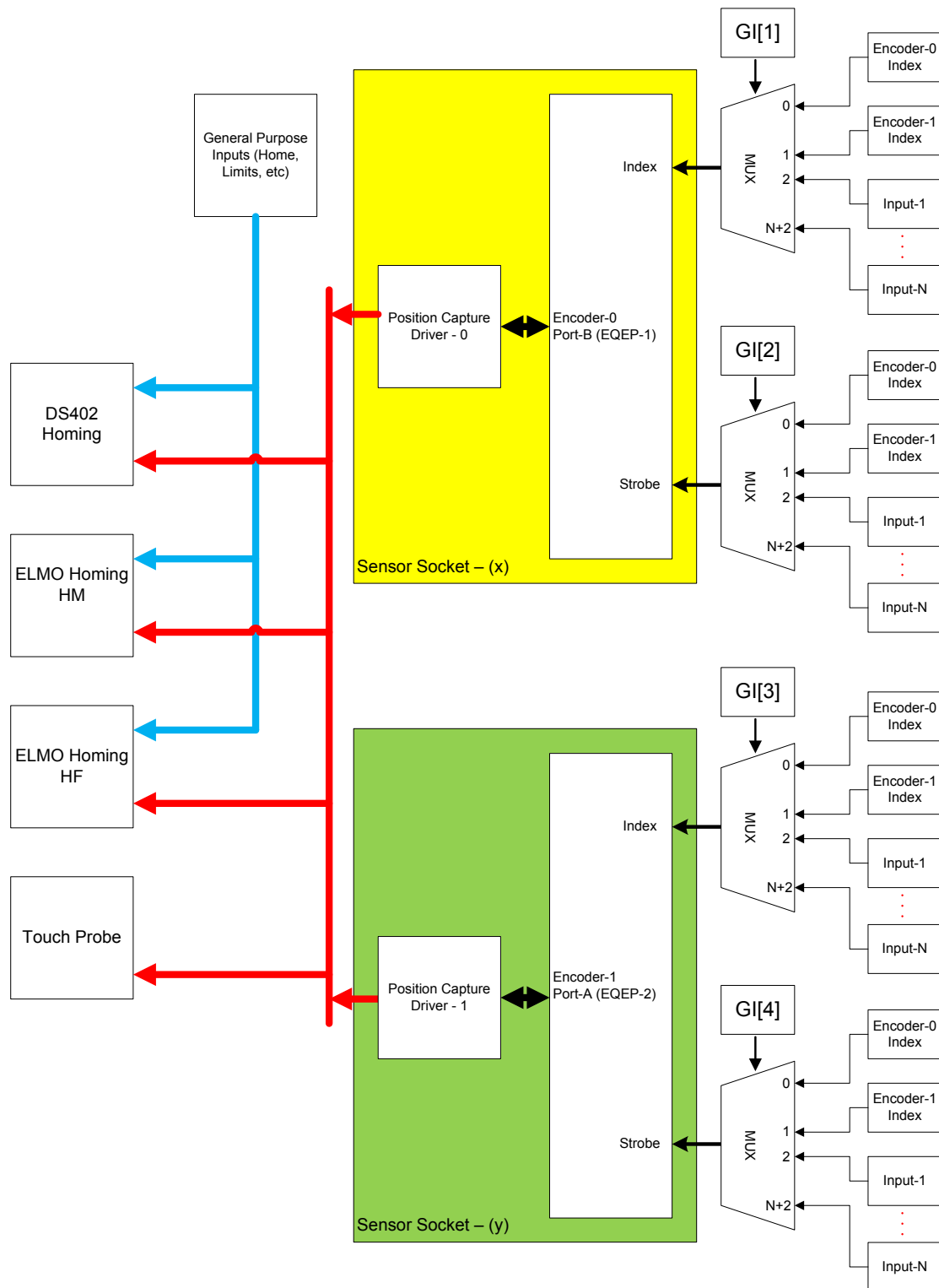


Figure 1: General Input Capture Structure

Figure 17 depicts the principal position capture functionality. As is shown, the touch probe uses only fast inputs, while other modules can incorporate fast and slow inputs. DS 402 homing and touch probe cannot work while **HM[]** or **HF[]** is running, and vice versa.

By default, the main feedback sensor is selected as the captured sensor. The user can change this as follows:

- n. For touch probe socket selection: Object 0x20B0 sub-index 7 or the **CA[87]** command
- o. For DS 402 homing socket selection: Object 0x20B0 sub-index 8 or the **OV[54]** command.

For more details about the 0x20B0 object, refer to the *Elmo Gold CANopen Manual*.

12.2. Position Capture-Related Commands

This section describes the commands and configuration steps for Position capture.

Notes:

- For detailed specifications of the commands, refer to the *Command Reference for Gold line Drives*.
- For CANopen-related objects descriptions, refer to the *Elmo Gold CANopen Manual*.

The following table lists the commands that participate in the Output Compare configuration:

Command	Position Capture Related Description
IL[N]	Input logic, to define the input logic level and dedicated function.
GI[N]	Route the input into one of the Port A or Port B index/strobe signals.
GX[N]	Retrieve captured positions from HM[N] defined table.
GY[N]	Retrieve captured positions from HF[N] defined table.
HM[N]	Configure, start and read status of main capture module.
HF[N]	Configure, start and read status of secondary capture module.
IF[N]	Digital Input software filter. Effects the response time for Software Sensed capture signals.

Table 1: Position Capture-Related Commands

The following subsections describe the commands contribution to position capture mode.

12.2.1. IL[N] Command

The **IL[N]** command is used by position capture for:

- Configuring the logic (“active high” or “active low”) of the input to be used with one of the modules using position capture
- Configuring the behavior, for example, home input, of the input to be used as a fast capture input, limit switch or general purpose

12.2.2. GI[N] Command

This command enables the routing of one of the digital inputs into one of Port A, Port B, index or strobe signals.

The user should use this command before using one of the modules using position capture.

12.2.3. GX[N] Command

Retrieves captured values from capture array, when using **HM[]** with capture into table option.

12.2.4. GY[N] Command

Retrieves captured values from capture array, when using **HF[]** with capture into table option.

12.2.5. HM[N] Command

The **HM[]** command enables the following:

- Configure the position capture options and actions to do when event occurs.
- Report the status of the position capture module.
- This command always selects the main feedback-position socket, as its position capture source.

12.2.6. HF[N] Command

The **HF[]** command enables the following:

- Select a socket to act as the position capture source. The socket must be configured with AqB hardware, i.e. Port A or Port B.
- Configure the position capture options and actions to do when event occurs.
- Report the status of the position capture module.
- HF cannot be enabled on main position feedback, if configured as home.

When both **HM[N]** and **HF[N]** are run, and using the main feedback sensor is selected .

(HF[10] = 1), the results are unpredictable.

12.2.7. IF[N] Command

The **IF[N]** command defines the input filter used when position capture is used with slow inputs. It is applicable only with Elmo's homing mode.

12.3. Elmo Homing and Capture Commands

The **HM[]/HF[]** commands are Elmo homing interface.

The **HM[]** commands are used for main position feedback capture.

The **HF[]** commands are used for user selected socket position capture. The socket must be configured to Port A or Port B.



Elmo's homing enables the following, once an event occurs:

- Modify the sensor position counter
- Log the event position counter into array
- Flag a digital output
- Stop the axis motion

For more detailed specification of the **HM[N]** and **HF[N]** commands, see the *Command Reference Manual*.

The following diagram shows the configuration flow for 'Fast' and 'Slow' input capture.

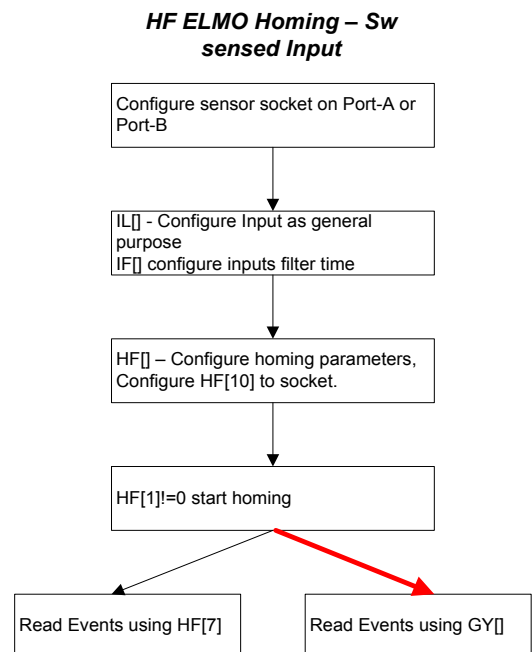
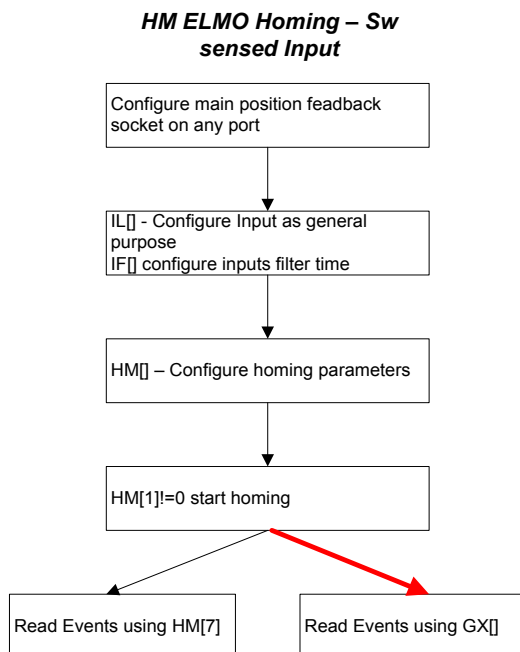
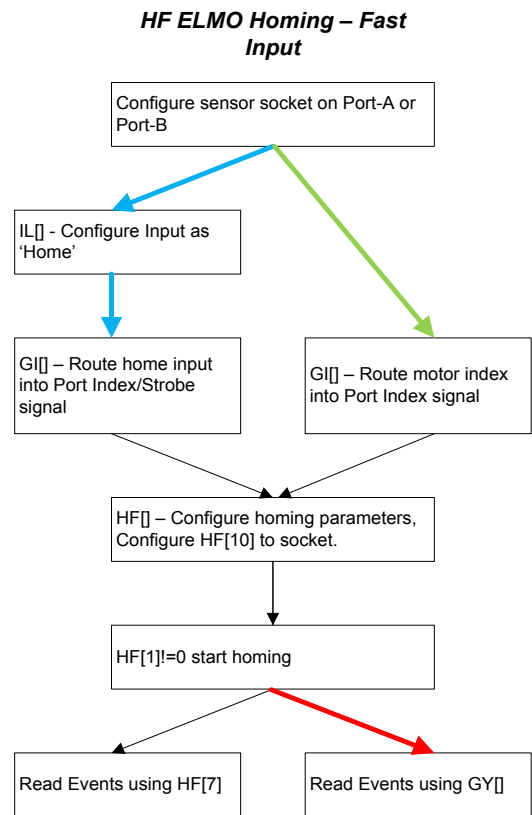
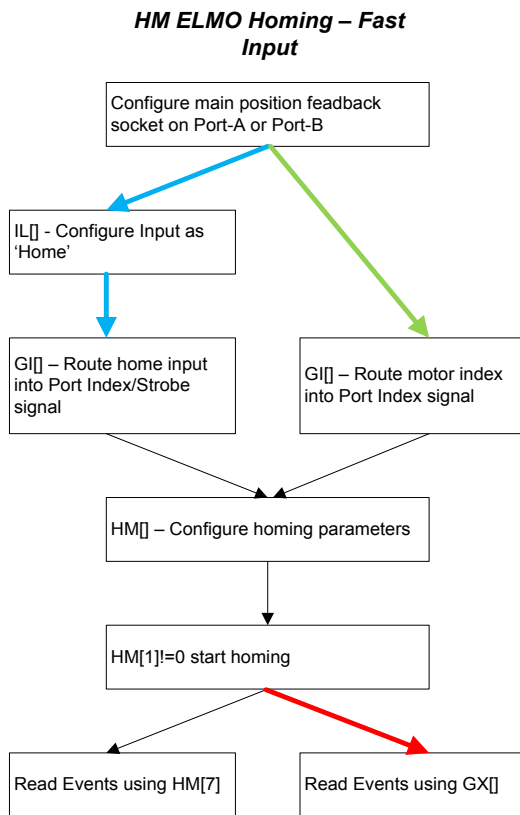
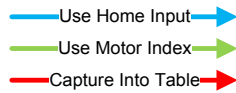


Figure 2: Elmo's Homing Configuration

12.3.1. HM[N] Examples

The following example uses capture on the main homing switch when input 5 is routed into the encoder 1 (Port A) strobe input by the **GI** command.

```
##start
main()
function main()
int iPos
HM[1]=0          /* Disable the ongoing homing sequence */
IL[5]=17         /* Configure input 5 as homing switch with "active high"
                  logic */
GI[4]=0x06       /* Route input 5 into encoder 1 strobe input */
HM[3]=2          /* Configure event on home signal (first rising edge) */
HM[1]=1          /* Start searching for a single event */

/* wait for event */
while(HM[1] > 0)
end

/* Captured position can be read at: */
iPos=HM[7]
return
```

The following example uses capture on the main homing switch when input 6 is routed into encoder 1 (Port A) strobe input by the **GI** command and three position events are saved into the **BH** array.

```
##start
main()
function main()
int iPos1        /* First event position */
int iPos2        /* Second event position */
int iPos3        /* Third event position */
HM[1]=0          /* Disable the ongoing homing sequence */
IL[6]=17         /* Configure input 6 as homing switch with "active high"
                  logic */
GI[4]=0x07       /* Route input 6 into encoder 1 strobe input */
HM[11]=5         /* Select BH array for event position storage */
HM[12]=1         /* BH array low index is 1 */
HM[13]=3         /* BH array high index is 3 */
HM[3]=2          /* Configure event on home signal (first rising edge) */
HM[1]=3          /* Start searching for a single event */

/* wait for event */
while(HM[1] > 0)
end

/* captured positions can be read by: */
iPos1=GX[1]
iPos2=GX[2]
iPos3=GX[3]

return
```

The following example uses capture on motor index when the motor index is routed into the encoder 1 (Port A) index input by the **GI** command. Upon an event, output 1 is set to 1.

```
##start
main()
function main()
int iPos
HM[1]=0          /* Disable the ongoing homing sequence */
GI[3]=0x01       /* Route motor index into encoder 1 index input */
HM[3]=3          /* Configure event on high transition of index pulse */
HM[6]=1          /* OP value will be set to this value at event */
HM[4]=1          /* Set digital output, to value at HM[6] */
HM[1]=1          /* Start searching for a single event */

/* wait for event */
while(HM[1] > 0)
end

/* Captured position can be read at: */
iPos=HM[7]
return
```

12.3.2. HF[N] Examples

All the **HF[N]** examples use encoder 0 configured on socket #2 (CA[42]=1).

The following example uses capture on the homing switch when input 5 is routed into the encoder 0 (Port B) strobe input by the **GI** command.

```
##start
main()
function main()
int iPos
HF[1]=0          /* Disable the ongoing homing sequence */
IL[5]=17         /* Configure input 5 as homing switch with "active high"
                  logic */
GI[2]=0x06       /* Route input 5 into encoder 0 strobe input */
HF[5]=2          /* PX is not set during event */
HF[10]=2         /* Set HF socket to 2 */
HF[3]=2          /* Configure event on home signal (first rising edge) */
HF[1]=1          /* Start searching for a single event */

/* wait for event */
while(HF[1] > 0)
end

/* Captured position can be read at: */
iPos=HF[7]
return
```

The following example uses capture on the main homing switch, when input 6 is routed into the encoder 0 (Port B) strobe input by the **GI** command, and 3 position events are saved into the **BH** array.

```
##start
main()
function main()
int iPos1      /* First event position */
int iPos2      /* Second event position */
int iPos3      /* Third event position */
HF[1]=0        /* Disable the ongoing homing sequence */
IL[6]=17       /* Configure input 6 as homing switch, with "active high"
               logic */
GI[2]=0x07     /* Route input 6 into encoder 0 strobe input */
HF[10]=2       /* Set HF socket to 1 */
HF[11]=5       /* Select BH array for event position storage */
HF[12]=1       /* BH array low index is 1 */
HF[13]=3       /* BH array high index is 3 */
HF[3]=2        /* Configure event on home signal (first rising edge) */
HF[5]=2        /* PX is not set during event */
HF[1]=3        /* Start searching for a single event */

/* wait for event */
while(HF[1] > 0)
end

/* captured positions can be read by: */
iPos1=GY[1]
iPos2=GY[2]
iPos3=GY[3]
return
```

The following example uses capture on motor index when the motor index is routed into encoder 0 (Port B) index input by the **GI** command.

```
##start
int iEvent
int iPos
main()
function main()
global int iPos
global int iEvent
iEvent=0       /* Clear event flag, will be flagged at auto routine */
HF[1]=0        /* Disable the ongoing homing sequence */
GI[3]=2        /* Route motor index into encoder 0 index input */
HF[3]=3        /* Configure event on high transition of index pulse */
HF[1]=1        /* Start searching for a single event*/

/* wait for event */
while(iEvent == 0)
end
iPos=HF[7]
return

#@AUTO_HY
iEvent=1 /* HF is done */
return
```

12.4. DS 402 Touch Probe

Touch probe is defined by the CANopen DS 402 protocol for capturing the main position sensor. The touch probe can use up to two position captures with the rising edge, the falling edge or both edges. For more information, refer to the *Elmo Gold CANopen Manual*.

The two possible capture signals are available to capture four position events:

- Position 1 refers to a socket index input (touch probe 1). Event on rising or/and falling edge
- Position 2 refers to a socket strobe input (touch probe 2). Event on rising or/and falling edge

By default, the socket selection is done on the main position feedback socket. Any other socket can be selected using object 0x20B0 with sub-index 7 (**CA[87]**).

In the case of EtherCAT, touch probe 2 is not mappable to tPDO. The touch position can be retrieved via SDO using objects 0x60BC and 0x60BD.

The user can route different inputs to index/strobe with the **GI[]** command.

Capture always starts on the rising edge (if the falling edge arrives first, it will be ignored).

Touch probe detection is done by hardware without jitters in rate <5 µsec. The sensing resolution between consecutive touch events is 4 kHz (250 µsec).

DS 402 Object	Description
0x60B8	Touch probe function
0x60B9	Touch probe status
0x60BA	Position value of the touch probe 1 at positive edge
0x60BB	Position value of the touch probe 1 at negative edge
0x60BC	Position value of the touch probe 2 at positive edge
0x60BD	Position value of the touch probe 2 at negative edge
0x2E10	Calculate the home position according to the captured value by the touch probe

Table 2: DS 402 Touch Probe Interface

Notes:

Touch probe does not affect the position of the drive, except home on the touch probe (0x2E10).

Touch probe cannot run while **HM[]/HF[]** or DS 402 homing mode is activated.

The captured position reflects the last capture event. No history is maintained.

Homing according to touch probe:

By using object 0x2E10, the last touch probe capture signal can be set as the homing offset (as defined in object 0x607C). Refer to the definition of object 0x2E10 in this manual for more information.

12.4.1. Definition of Object 0x60B8

Object 0x60B8 indicates the configured function of the touch probe.

Object description:

Attribute	Value
Index	0x60B8
Name	Touch probe function
Object code	Variable
Data type	Unsigned16
Category	Optional

Entry description:

Attribute	Value
Sub-index	0x00
Access	Read/write
PDO mapping	See /CiA402-3/
Value range	Unsigned16
Default value	Manufacturer-specific

Bit	Binary Value	Definition
0	0	Switch off touch probe 1
	1	Enable touch probe 1
1	0	Trigger first event
	1	Continuous
3, 2	00	Trigger with touch probe 1 input
	01	Trigger with zero impulse signal or position encoder
	10	Touch probe source as defined in object 0x60D0, sub-index 0x01
	11	Reserved

Bit	Binary Value	Definition
4	0	Switch off sampling at the positive edge of touch probe 1
	S	Enable sampling at the positive edge of touch probe 1
5	0	Switch off sampling at the negative edge of touch probe 1
	1	Enable sampling at the negative edge of touch probe 1
6, 7	-	User defined (for example, for testing)
8	0	Switch off touch probe 2
	1	Enable touch probe 2
9	0	Trigger first event
	1	Continuous
11, 10	00	Trigger with touch probe 2 input
	01	Trigger with zero impulse signal or position encoder
	10	Touch probe source as defined in object 0x60D0, sub-index 0x02
	11	Reserved
12	0	Switch off sampling at the positive edge of touch probe 2
	S	Enable sampling at the positive edge of touch probe 2
13	0	Switch off sampling at the negative edge of touch probe 2
	1	Enable sampling at the negative edge of touch probe 2
14, 15	-	User defined (for example, for testing)

12.4.2. Definition of Object 0x60B9

Object 0x60B9 indicates the status of the touch probe.

Object description:

Attribute	Value
Index	0x60B9
Name	Touch probe status
Object code	Variable
Data type	Unsigned16
Category	Optional

Entry description:

Attribute	Value
Sub-index	0x00
Access	Read-only
PDO mapping	See /CiA402-3/
Value range	Unsigned16
Default value	Not defined

Bit	Binary Value	Definition
0	0	Touch probe 1 is switched off.
	1	Touch probe 1 is enabled.
1	0	Touch probe 1 does not have a positive edge value stored.
	1	Touch probe 1 has a positive edge value stored.
2	0	Touch probe 1 does not have a negative edge value stored.
	1	Touch probe 1 has a negative edge value stored.
3 to 5	0	Reserved
6, 7	-	User defined (for example, for testing)
8	0	Touch probe 2 is switched off.
	1	Touch probe 2 is enabled.
9	0	Touch probe 2 does not have a positive edge value stored.
	1	Touch probe 2 has a positive edge value stored.
10	0	Touch probe 2 does not have a negative edge value stored.
	1	Touch probe 2 has a negative edge value stored.
11 to 13	0	Reserved
14, 15	-	User defined (for example, for testing)

12.4.3. Definition of Objects 0x60BA to 0x60BD

Object 0x60BA provides the value of the position of touch probe 1 at the positive edge.

Attribute	Value
Index	0x60BA
Name	Touch probe 1 positive edge
Object code	Variable
Data type	Integer32
Category	Optional

Object 0x60BB provides the position value of the touch probe 1 at the negative edge.

Attribute	Value
Index	0x60BB
Name	Touch probe 1 negative edge
Object code	Variable
Data type	Integer32
Category	Optional

Object 0x60BC provides the position value of the touch probe 2 at the positive edge.

Attribute	Value
Index	0x60BC
Name	Touch probe 2 positive edge
Object code	Variable
Data type	Integer32
Category	Optional

Object 0x60BD provides the position value of the touch probe 2 at the negative edge.

Attribute	Value
Index	0x60BD
Name	Touch probe 2 negative edge
Object code	Variable
Data type	Integer32



Category	Optional
----------	----------

The entry descriptions for all of the above objects are identical:

Attribute	Value
Sub-index	0x00
Access	Read-only
PDO mapping	See /CiA402-3/
Value range	Integer32
Default value	Not defined

12.4.4. Definition of Object 0x2E10:

Adjust the position of the touch probe socket according to the touch probe captured value and the homing offset (object 0x607C). In other words, treat the touch probe captured value as home.

Position adjustment completion is indicated by returning of the SDO. This object is not mappable.

The object includes the following:

- Value 0 – adjust the position of the touch probe socket according to the touch probe rising edge value and the homing offset (object 0x607C)
- Value 1 – adjust the position of the touch probe socket according to the touch probe falling edge value and the homing offset (object 0x607C)

Object description:

Attribute	Value
Index	0x2E10
Name	Home on touch probe
Object code	Variable
Data type	Integer32
Category	Optional

Entry description:

Attribute	Value
Sub-index	0
Access	Write-only

Attribute	Value
PDO mapping	NA
Value range	Integer16
Default value	Not defined

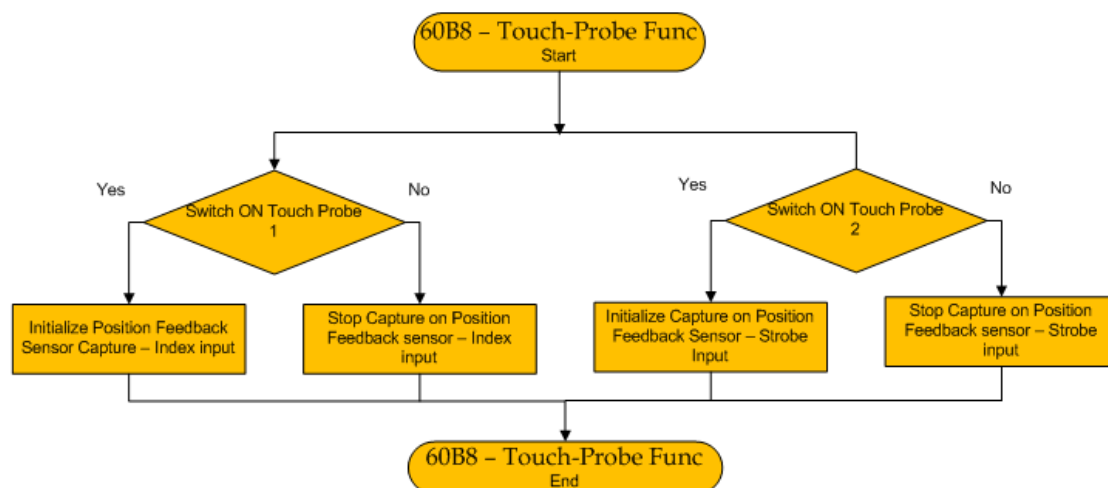


Figure 3: Touch Probe Function Configuration

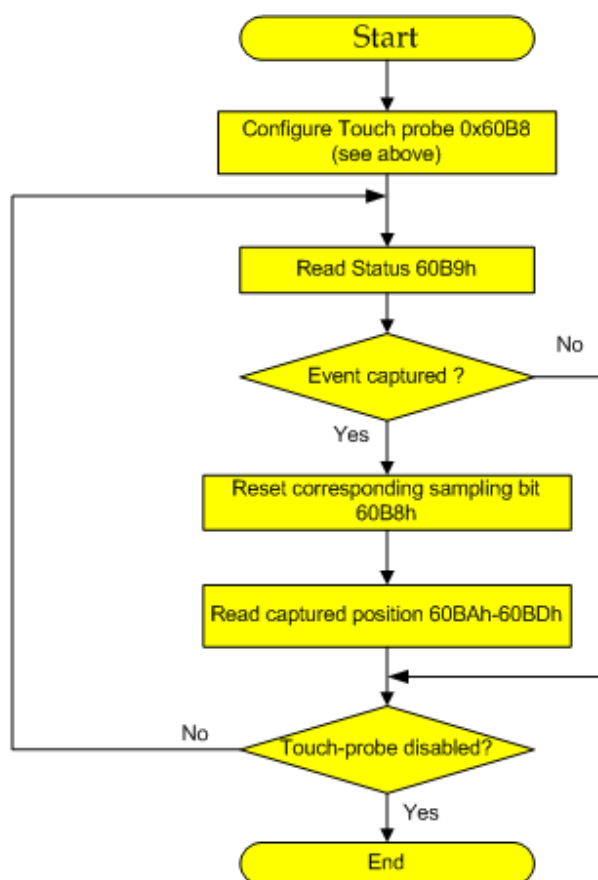


Figure 4: Touch Probe Behavior

Example:

1. Route digital input 1 into encoder 1 index input by setting GI[3]=2 .
2. Route digital input 2 into encoder 1 strobe input by setting GI[4]=3.
3. Enable sampling at the positive edge of touch probe 1 & 2 by setting sub-index 0 of object 0x60B8 to 0x1111.
4. Read the capture status from sub-index 0 of 0x60B9 object.
 - Bit 0 - Touch probe 1 enabled.
 - Bit 1 - Touch probe 1 positive edge position stored.
 - Bit 8 - Touch probe 1 enabled.
 - Bit 9 - Touch probe 1 positive edge position stored.
5. Read sub-index 0 of object 0x60BA for the position value of the touch probe 1 at positive edge.
6. Read sub-index 0 of object 0x60BC for the position value of the touch probe 2 at positive edge.
7. Reset bits 1 and 9 of 0x60B9 for more samples at positive edge of touch probes 1 & 2.

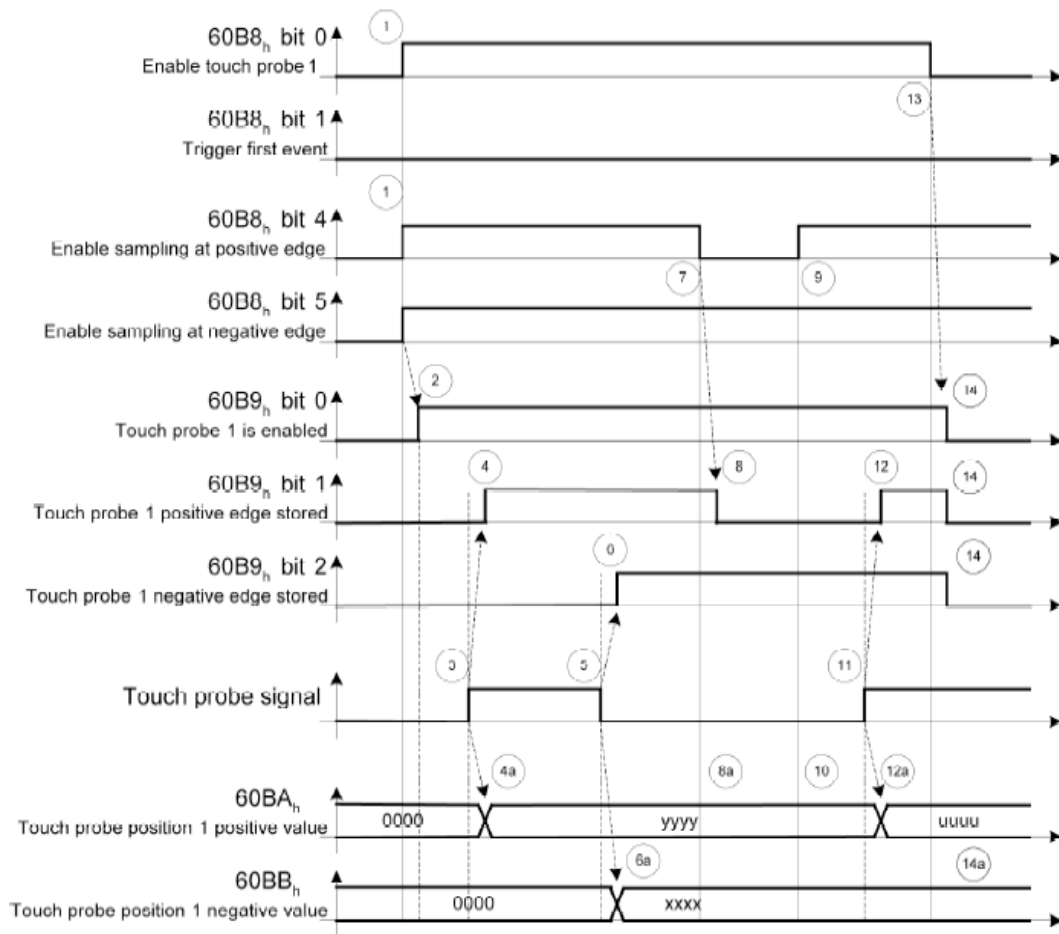


Figure 5: Timing Diagram for the Touch Probe Example

Number	Object	Touch Probe Behavior
(1)	0x60B8, bit 0 = 1	Enable touch probe 1.
	0x60B8, bits 1, 4, 5	Configure and enable touch probe 1, positive and negative edges.
(2)	→ 0x60B9, bit = 1	The status "Touch probe 1 enabled" is set.
(3)		The external touch probe signal has a positive edge.
(4)	→ 0x60B9, bit 1 = 1	The status "Touch probe 1 positive edge stored" is set
(4a)	→ 0x60BA	The touch probe position 1 positive value is stored.
(5)		The external touch probe sign has a negative edge.
(6)	→ 0x60B9, bit 1 = 1	The status "Touch probe 1 negative edge stored" is set.
(6a)	→ 0x60BB	The touch probe position 1 negative value is stored.
(7)	0x60B8, bit 4 = 0	The sample positive edge is disabled.
(8)	→ 0x60B9, bit 0 = 0	The status "Touch probe 1 positive edge stored" is reset.
(8a)	→ 0x60BA	The touch probe position 1 positive value is not changed.
(9)	0x60B8, bit 4 = 1	The sample positive edge is enabled.
(10)	→ 0x60BA	The touch probe position 1 positive value is not changed.
(11)		The external touch probe signal has a positive edge.
(12)	→ 0x60B9, bit 1 = 1	The status "Touch probe 1 positive edge stored" is set.
(12a)	→ 0x60BA	The touch probe position 1 positive value is stored.
(13)	0x60B8, bit 0 = 0	Touch probe 1 is disabled.
(14)	→ 0x60B9, bits 0, 1, 2 = 0	The status bits are reset.
(14a)	→ 0x60BA, 0x60BB	The touch probe position 1 positive/negative values are not changed.

Table 3: Explanation of the Timing Diagram

12.5. DS 402 Homing

DS 402 homing is a motion mode that performs a predefined sequence to find and set the home position of an axis.

For accurate homing identification, the DS 402 homing implementation uses the drive's position capture module. The capture module enables an accurate reading of position during input change, and identification of special cases like home-input plus index scenario's position. These scenarios are used by DS 402 homing in several methods. The drive includes configurations for the relevant input functionalities and logic levels (using the **IL[N]** command). For fast capturing, the relevant inputs must be routed (using the **GI[N]** command) into position feedback sensor strobe/index signals. Note that the strobe and index must be on the same sensor.

By default, DS 402 homing always operates on the main position feedback sensor. This, however, can be changed using object 0x20B0 with sub-index 8 or, alternatively, the **OV[54]** command.

Implementing this motion mode includes two stages:

1. **Configuration stage:** In this stage the user configures the drive's hardware using the following Elmo protocol commands to configure the hardware:
 - The **GI[N]** command – Routes the relevant input into the sensor's index/strobe signals (see Table 20: Supported DS 402 Homing Methods below). According to the homing scenario, the motor index signal should be routed into the sensor's index signal, and the home switch input should be routed into the sensor's strobe signal.
 - The **IL[N]** command – Configures the input functionality needed by the homing routine. For example, home-input or limit switch can be defined. According to the homing scenario, the home switch should be defined as home.

The above commands are non-volatile. They should be configured once according to the homing method which the axis uses.

2. **Operation stage:** In this stage the user uses CANopen objects to implement the homing motion mode:
 - Change homing motion mode related parameters (see relevant objects in Table 21: DS 402 Homing Interface).
 - Start homing motion mode with DS 402 motion state machine, using the control word (object 0x6040).

Once the homing mode is started, the axis will move according to the selected scenario (see object 0x6098). During the homing procedure, the status is reported in the status word (object 0x6041).



Homing Method (Object 0x6098)	IL - Needed input function configuration	GI - Input routing for position feedback sensor
1	Reverse limit switch	Motor Index into sensor's index signal
2	Forward limit switch	Motor Index into sensor's index signal
3	Home	Motor Index into sensor's index signal Home input into sensor's strobe signal
4	Home	Motor Index into sensor's index signal Home input into sensor's strobe signal
5	Home	Motor Index into sensor's index signal Home input into sensor's strobe signal
6	Home	Motor Index into sensor's index signal Home input into sensor's strobe signal
7	Home Forward limit switch	Motor Index into sensor's index signal Home input into sensor's strobe signal
8	Home Forward limit switch	Motor Index into sensor's index signal Home input into sensor's strobe signal
9	Home Forward limit switch	Motor Index into sensor's index signal Home input into sensor's strobe signal
10	Home Forward limit switch	Motor Index into sensor's index signal Home input into sensor's strobe signal
11	Home Reverse limit switch	Motor Index into sensor's index signal Home input into sensor's strobe signal
12	Home Reverse limit switch	Motor Index into sensor's index signal Home input into sensor's strobe signal
13	Home Reverse limit switch	Motor Index into sensor's index signal Home input into sensor's strobe signal
14	Home Reverse limit switch	Motor Index into sensor's index signal Home input into sensor's strobe signal
15-16	Reserved	Reserved
17	Reverse limit switch	No special routing needed
18	Forward limit switch	No special routing needed

Homing Method (Object 0x6098)	IL - Needed input function configuration	GI - Input routing for position feedback sensor
19	Home	Home input into sensor's strobe signal
20	Home	Home input into sensor's strobe signal
21	Home	Home input into sensor's strobe signal
22	Home	Home input into sensor's strobe signal
23	Home Forward limit switch	Home input into sensor's strobe signal
24	Home Forward limit switch	Home input into sensor's strobe signal
25	Home Forward limit switch	Home input into sensor's strobe signal
26	Home Forward limit switch	Home input into sensor's strobe signal
27	Home Reverse limit switch	Home input into sensor's strobe signal
28	Home Reverse limit switch	Home input into sensor's strobe signal
29	Home Reverse limit switch	Home input into sensor's strobe signal
30	Home Reverse limit switch	Home input into sensor's strobe signal
31-32	Reserved	Reserved
33	No special configuration needed	Motor Index into sensor's index signal
34	No special configuration needed	Motor Index into sensor's index signal
35	No special configuration needed	No special configuration needed

Table 4: Supported DS 402 Homing Methods

DS 402 Object	Description
0x607C	Homing offset (user units)
0x6098	Homing method (see Table 20: Supported DS 402 Homing Methods)
0x6099	Index 1 – homing fast speed (user units)
	Index 2 – homing slow speed (user units)
0x609A	Homing acceleration/deceleration (user units)

Table 5: DS 402 Homing Interface

Notes:

DS 402 homing cannot be activated when **HM[]/HF[]** or touch probe are operational.

DS 402 homing and Output Compare not allowed on the same socket when homing uses the strobe input. DS 402 homing can use index while strobe is used for compare.

12.6. PLCopen Homing on Block

Elmo uses DS 402 to implement homing on block with some additions, such as object 0x2020. The homing on block methods are -1 and -2, which are reserved by DS 402 for manufacturer-specific implementations.

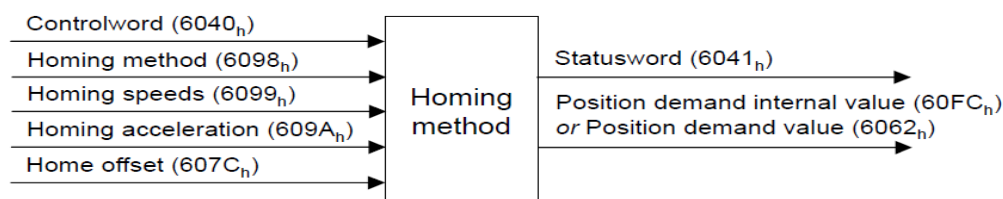


Figure 6: DS 402 Homing Method Definition

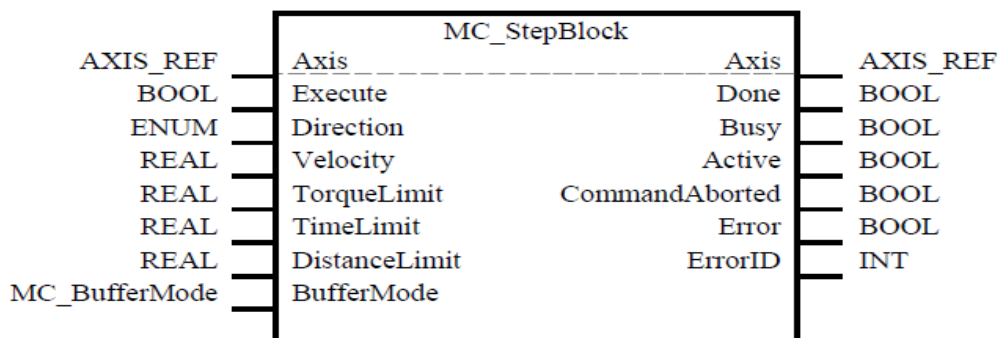


Figure 7: Homing Method by PLCopen

Input

- Homing method (object 0x6098); Homing block values:
 - -1 – reverse
 - -2 – forward
- Homing velocity (object 0x6099)
- Homing acceleration (object 0x609A)
- Control word (object 0x6040)

Output

- Status word (object 0x6041)

Not used in homing on block

- Homing offset (object 0x607C)
- Position demand (object 0x60FC or 0x6062)

Note: In order to have an offset, homing on current position can be used.

For PLCopen users HomeDirect can be used for offsetting.

PLCopen Function	CAN open Object	
	Object	Sub-index
Input		
Execute (bool)	0x6040	0
Direction (enum)	0x6098	0
Velocity (real)	0x6099	0
TorqueLimit (real)	0x2020	1
TimeLimit (real)	0x2020	2
DistanceLimit (real)	0x2020	3
BufferMode (MC_BufferMode)	Always set to buffered mode	
Output		
Done (bool)	0x6041	0(bit 12 = 1)
Busy (bool)	NA	NA
Active (bool)	0x6041	0 (all mode bits 0)
CommandAborted (bool)	NA	NA
Error (bool)	0x6041	0 (bit 13 = 1)
ErrorID (int)	NA	NA

Table 6: PLCopen Function in DS 402

The object 0x2020 Homing limits:

- The object has 3 sub-indexes:
 - a. **Torque limit** - Maximum torque or force. The units of the PLCopen value must be converted by the upper host to "Per thousand of rated torque", which corresponds the units of the CANopen torque and current. Eventually the FW will convert the value from the user torque to current limit which is used to limit the force.

It has to be set to a value that does not overcome the mechanical blocking.

0 = No torque limit.
 - b. **Time limit** - If the Step Block condition is not met within the Time Limit, an error is issued.

0 = No time limit.

The time limit is in milliseconds with 250 µsec resolution. The following calculation determines the maximum time limit allowed for avoiding counter overflow:

$$(2^{32})/4 = 2^{30} \text{ msec}$$
 - c. **Distance limit** - If the Step Block condition is not met within a Distance Limit travel, an error will be issued.

0 = No distance limit.

Distance limit is in counts.

(The accuracy of the distance limit relative to the counts that the axis moves within 250 µsec).

Example:

1. Set the homing method: 0x6098[1] = -1 (reverse)
2. Set the homing velocity: 0x6099[0] = 10000
3. Set the homing acceleration: 0x609A[0] = 1000
4. Set the torque limit: 0x2020[1] = 100
5. Set no time limit: 0x2020[2] = 0
6. Set the distance limit: 0x2020[3] = 1000000
7. 0x6060[0] = 6
8. Begin homing by using the control word: Set 0x6040[0] bit 4.
9. Monitor the status word (object 0x6041) to the end of the homing (home attain) or any possible homing failure.
10. After homing is completed (error or success), homing mode is not exited automatically.

Exit homing mode: use object 0x6060[0] = 0.

12.7. Position Capture Notes and Limitations

1. Elmo's homing, DS 402 homing and touch probe cannot run at the same time.
2. Position capture and Output Compare cannot be enabled on the same socket when capture is set on strobe. The user can capture index and use strobe for comparison.
3. When the **HM[]** and **HF[]** capture modules are enabled on the same sensor, the result has unpredictable behavior.
4. In old hardware using the SCORE module (e.g. G-GUI), only input 5 and input 6 can be configured as home inputs.
5. All inputs (index 0, index 1, and inputs 1-6) can be routed to the sensor's index or strobe signal and can be configured as homing inputs.
6. The frequency of a sensed input pulse varies according to the definition of the input (see Chapter 1 in this manual). If the input pulses are faster, the drive will miss captured events.
7. In touch probe, the first event will always be a rising edge of the input. Any falling edge before the first rising edge is ignored.

12.8. Position Capture-Related Error Codes (EC)

The following table lists relevant error codes for position capture.

EC Value	Description
3	GI[N] $N < 0$ OR $N > 4$. HM[N]/HF[N] Trying to write to indexes 7,8,9, read-only parameters. Trying to read/write to an unsupported index, i.e., $N > 13$ or $N < 0$.
21	Some parameter is out of the allowed range (for example, GI , IL etc.).
71	HM[]/HF[] Commands: Touch probe enabled. DS 402 homing enabled.
102	Touch Probe: Output Compare already defined on the sensor (main position feedback). HM[]/HF[] Commands: Output Compare already defined on the sensor. DS 402 Homing: Output Compare already defined on the sensor (main position feedback).
184	Touch Probe: Position capture module already used by Elmo homing HM/HF .



	Position capture module already used by DS 402 homing. Object 0x6060: Modes of operation: Position capture module already used by Elmo homing HM/HF. Position capture module already used by touch -probe.
--	---

Table 7: Position Capture-Related Error Codes (EC)

Chapter 13: Output Compare

13.1. Introduction

This chapter describes the Output Compare feature supported by the Gold drives.

Output Compare provides the ability to trigger digital output based on a given position.

The output pulse length can be based on time or position.

- When **time-based pulses** are used, the duration of the pulse is X [μsec]. X is specified by the user during the initialization of the Output Compare feature.
- When **position-based pulses** are used, the pulse duration stretches between two positions without any time limitations.

The Output Compare function supports table modes as well, in which the compare positions can be listed in a table prior to activation of the feature. In this case an Output Compare table is configured using the **OC[N]** command, and indexed pairs of start and end positions are added to the selected table array using the **GV[N]** or **GW[N]** command.

There is an option for the user to program the Output Compare function on-the-fly, i.e., while a previous Output Compare function is working, to prepare another logical position section of the compare.

Two sensors in the drive support the Output Compare feature:

Port A (which is	Function	Activation Command
1	Output Compare	OC[1 to 12]
2	Special function	OL[2]
3	General purpose	OL[3]
4	Output Compare	OC[21 to 32]
14, 15, 16	General purpose	OL[14], OL[15], OL[16]

Table 8: Output Compare Function Example

Note: Outputs 14, 15 and 16 can be used for other functions, such as feedback emulation. Refer to the **GO[]** command in the Command Reference for Gold Line Drives.

13.2. Commands Related to Output Compare

This section provides general descriptions of the commands that are used to configure and operate Output Compare.

For detailed specifications of the commands, refer to the Command Reference for Gold Line Drives.

The following table lists the commands included in the Output Compare configuration.

Command	Description in the Context of Output Compare
OL[N]	Output Logic: - Used by Output Compare to define the output logic level as “active high” or “active low”. - Defines the output functionality before Output Compare is enabled and after Output Compare is disabled.
GO[N]	Output Compare source. Used to configure the output port (sensor) source.
GV[N]/GW[N]	Output Compare table editing. Enables editing of compare table positions.
OC[N]	Output Compare configuration/operation and status.

Table 9: Commands Related to Output Compare

The following subsections describe the commands that contribute to the Output Compare mode.

13.2.1. OL[N] Command

The **OL[N]** command is used to configure the output pin logic levels as “active high” or “active low”.

If an output is configured as Output Compare by the **GO[N]** command, the output functionality as described in **OL[N]** is ignored, and the output is routed to the Output Compare function block.

For outputs 1 to 4, changes in the output logic level (to “active high” or “active low”) will take effect immediately even if Output Compare is enabled. For outputs 14 to 16, the logic will take effect on the next activation of the Output Compare feature.

13.2.2. GO[N] Command

This command enables selection of the output source.

The sources can be one of the following:

- 0 – The output is controlled by the **OP**, **OB[i]** and **OL[i]** commands.
- 1 – The output is controlled by the Port B strobe output, i.e., Output Compare on the Port B sensor.

- 2 – The output is controlled by the Port A strobe output, i.e., Output Compare on the Port A sensor.
- 3 – Daisy chain Port B, i.e., the output is connected to Port B.
- 4 – Daisy chain Port A, i.e., the output is connected to Port A.
- 5 – Encoder emulation, i.e., the output is connected to the encoder emulation output.

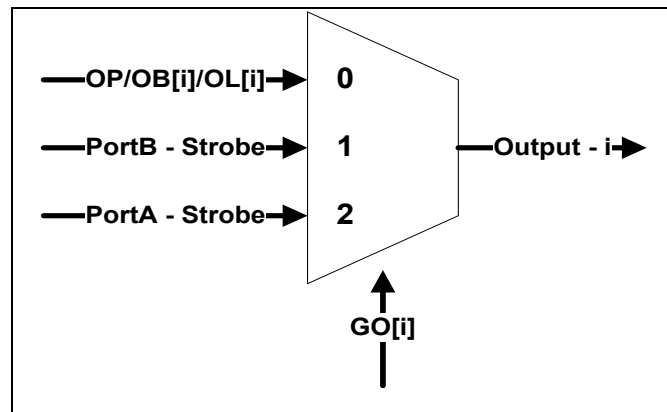


Figure 8: GO[N] Command Block Diagram

13.2.3. GV[N]/GW[N] Commands

The **GV[N]/GW[N]** commands enable editing of the position compare table for modes based on compare tables (**OC[1]** or **OC[21]** = 3, or **OC[1]** or **OC[21]** = 4). These commands provide the user a transparent way to store compare position values in the compare table.

GV[N] is used for editing tables for Output Compare module 1.

GW[N] is used for editing tables for Output Compare module 2.

Note: Before using this command to edit the table, the table array must be selected via the **OC[7]** or **OC[27]** command.

13.2.4. OC[N] Command

The **OC[N]** command is used to:

1. Configure the Output Compare blocks and their parameters.
2. Enable/disable the blocks.
3. Monitor the status and progress of the Output Compare blocks.

For a more detailed explanation regarding the specific command, see the Command Reference for Gold Line Drives.

13.3. Supported Types of Output Compare

The drive supports the Output Compare modes described in the following subsections.

13.3.1. Absolute Position Table Compare Mode

In this mode the drive generates pulses according to the pairs of the positions listed in a table.

The table includes pairs of absolute positions. Each pair of positions includes:

- **Start position** – the absolute position where the pulse starts.
- **End position** – the absolute position where the pulse ends.

The duration of a pulse is the time that elapses from the start position until the end position.

The order of the positions in the table can be configured by setting the start and end table indices (**OC[8]** and **OC[9]**). See the table below.

The following table describes the possible direction settings for the absolute position compare mode.

Compare Mode	Direction	OC[10] Value	Table Order	Table Indices	Pulse Start Position at Table Index
Absolute Position Table	Positive	1	Ascending ¹	OC[8] <= OC[9]	OC[8]
	Positive	1	Descending ²	OC[8] >= OC[9]	OC[9]
	Negative	2	Ascending	OC[8] >= OC[9]	OC[9]
	Negative	2	Descending	OC[8] <= OC[9]	OC[8]
	Both	0	Ascending	OC[8] <= OC[9]	OC[8]
	Both	0	Descending	OC[8] >= OC[9]	OC[9]

Table 10: Motion Directions Supported for Absolute Position Table Compare Mode

¹ Ascending – The positions in the table are monotonically ascending. The smallest position is at the lowest table index.

² Descending – The positions in the table are monotonically descending. The smallest position is at the highest table index.

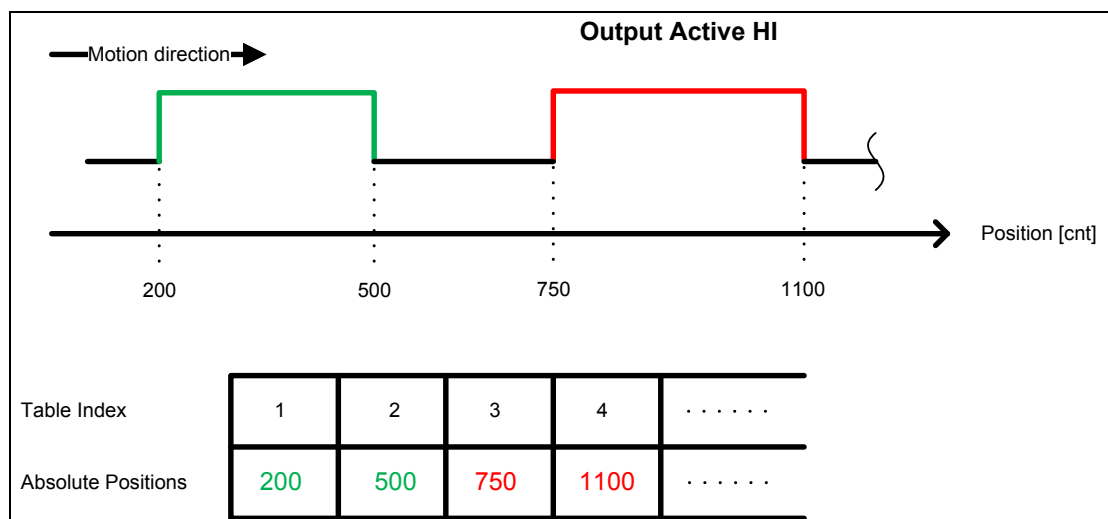


Figure 9: Absolute Position Table Compare Mode – Positive Direction

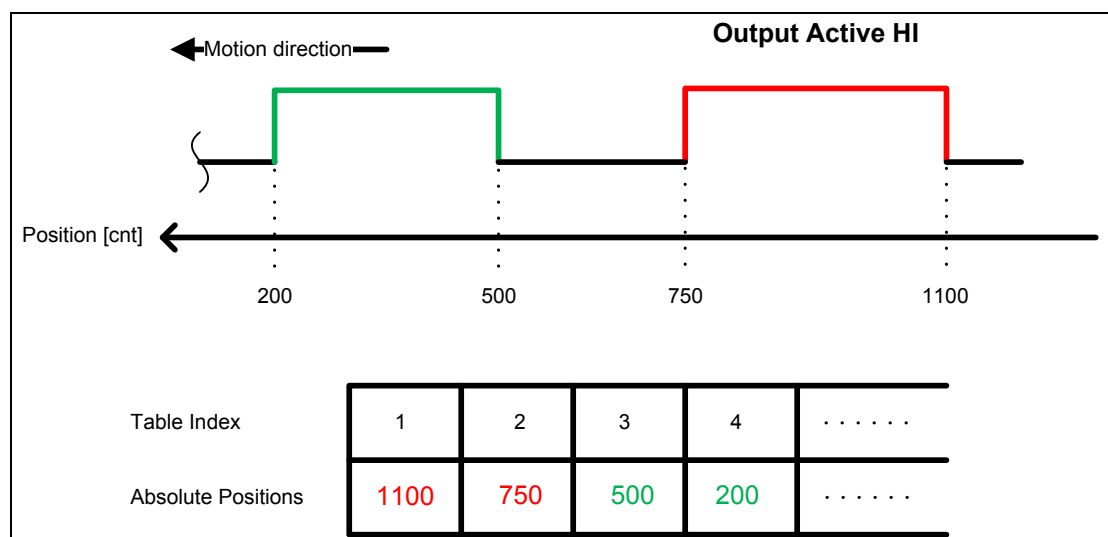


Figure 10: Absolute Position Table Compare Mode – Negative Direction

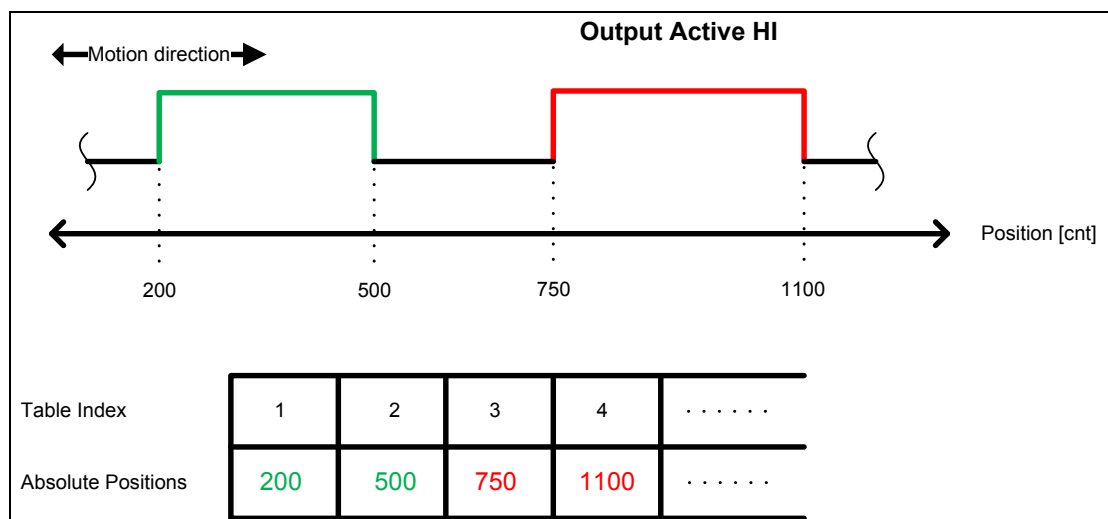


Figure 11: Absolute Position Table Compare Mode – Both Directions

Notes:

In cases in which both directions are enabled (**OC[10]** = 0 or **OC[30]** = 0) and the direction changes during a generated pulse, the pulse will end at the pulse start position. If the direction changes, the pulse will end at the pulse end position.

In table modes with both directions, the generated pulses are within the table boundaries. If the position passes the last index in the table, the next position at which a pulse will be generated is the position specified at the table's last index. If the position passes the first index in the table, the next position at which a pulse will be generated is the position at the table's first index.

13.3.2. Duration Time Table Compare Mode

In this mode the drive generates pulses according to the positions listed in a table. Each position indicates the absolute position where a pulse should be started, and the duration of the pulse is based on the time indicated in the duration parameter (**OC[4]**).

The direction of the positions in the table can be configured by setting the start and end table indices (**OC[8]** and **OC[9]**). See the table below.

This mode is operable in the following directions.

Compare Mode	Direction	OC[10] Value	Table Order	Table Indices	Pulse Start Position at Table Index
Time-Based Table	Positive	1	Ascending ³	OC[8] <= OC[9]	OC[8]
	Positive	1	Descending ⁴	OC[8] >= OC[9]	OC[9]
	Negative	2	Ascending	OC[8] >= OC[9]	OC[9]
	Negative	2	Descending	OC[8] <= OC[9]	OC[8]
	Both	0	Ascending	OC[8] <= OC[9]	OC[8]
	Both	0	Descending	OC[8] >= OC[9]	OC[9]

Table 11: Supported Motion Directions for Duration Time Table Compare Mode

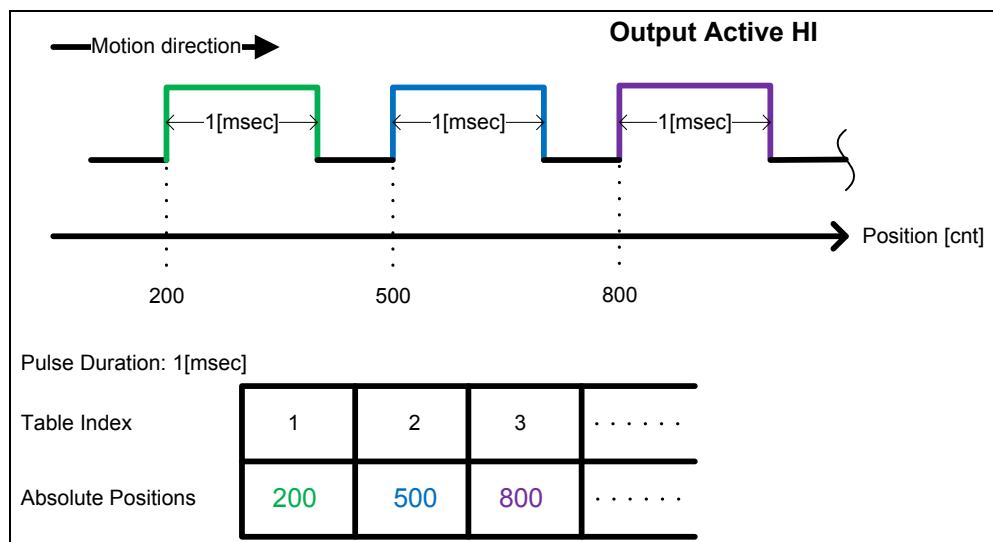


Figure 12: Duration Time Table Compare – Positive Direction

³ Ascending – The positions in the table are monotonically ascending. The smallest position is at the lowest table index.

⁴ Descending – The positions in the table are monotonically descending. The smallest position is at the highest table index.

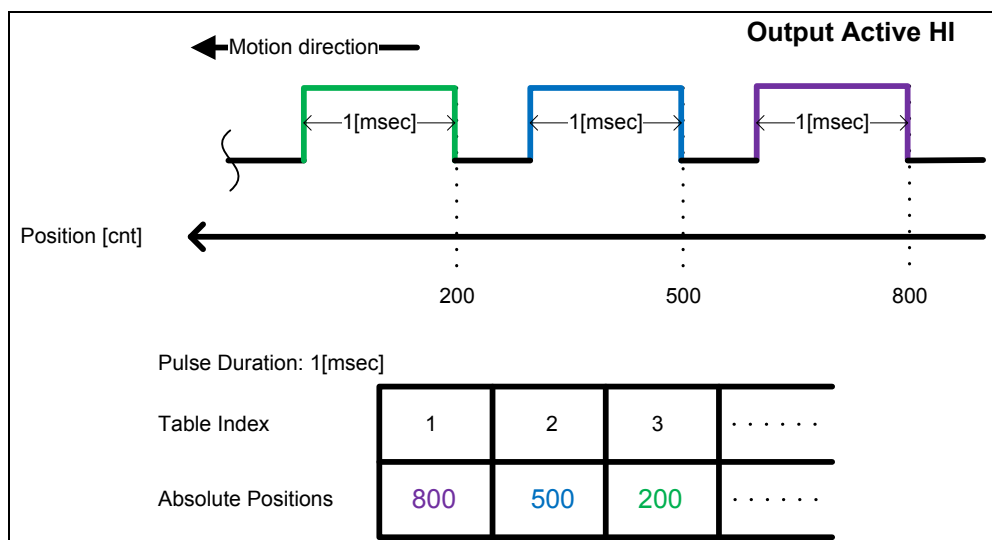


Figure 13: Duration Time Table Compare – Negative Direction

Note: In table modes with both directions, the generated pulses are within the table boundaries. If the position passes the last index in the table, the next position at which a pulse will be generated is the position specified at the table's last index. If the position passes the first index in the table, the next position at which a pulse will be generated is the position at the table's first index.

13.3.3. Absolute Start Position + Delta Position Compare Mode

In this mode the first pulse is generated at the specified absolute position, and subsequent pulses are generated at relative position intervals. The pulse length is time-based and is indicated in the duration parameter (**OC[4]** or **OC[24]**).

The positive/negative value of the position interval should be set according to the desired direction. When the direction is positive (increasing **PX** values), the position intervals should be positive; otherwise, they should be negative.

The number of generated pulses can be controlled by the value of the **OC[5]** or **OC[25]** command.

This mode is operable in the following directions.

Compare Mode	Direction	Configured By
Absolute Start + Delta	Positive	Setting a positive delta position value
	Negative	Setting a negative delta position value

Table 12: Motion Directions Supported for Absolute Start + Delta Compare Mode

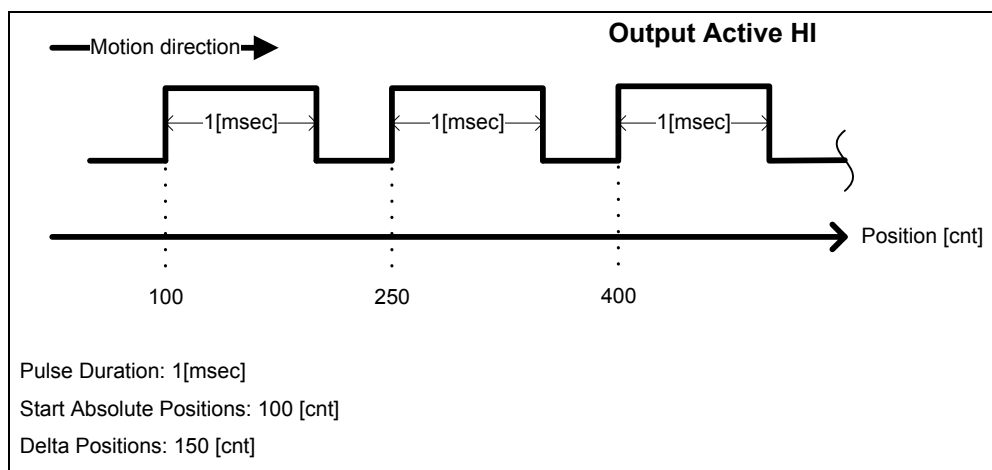


Figure 14: Compare Absolute Start Position + Delta Position – Positive Direction

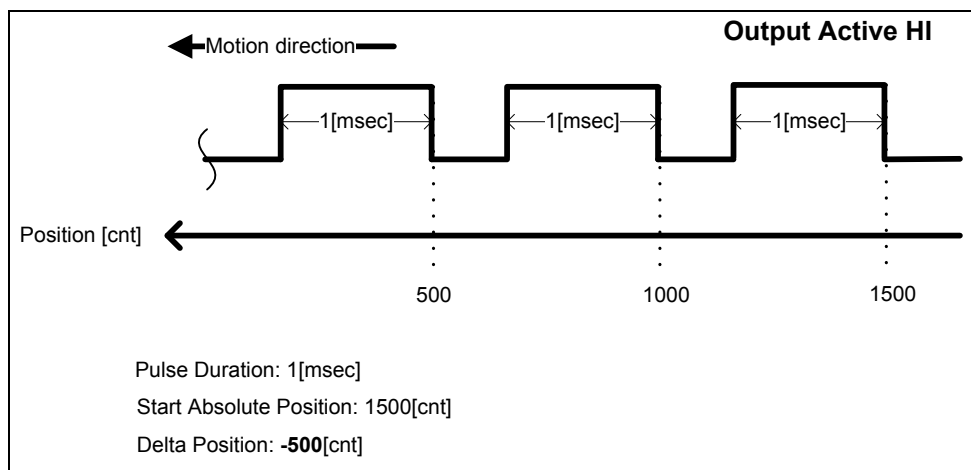


Figure 15: Compare Absolute Start Position + Delta Position – Negative Direction

13.4. Output Compare Configuration

To configure Output Compare, follow the steps:

1. Configure the output pin logic (**OL[N]**) as “active high” or “active low”.
2. Configure the output port source (**GO[N]**).
3. If a table mode is requested, initialize the table array with the **OC[7]** or **OC[27]** command, and then fill the table with the **GV[N]** or **GW[N]** command.
4. Configure the Output Compare mode and the parameters related to it.
5. Start the Output Compare mode using the **OC[1]** (or **OC[21]**) command.
6. After all the requested pulses are generated, the user can stop Output Compare using the **OC[1] = 0** (or **OC[21] = 0**) command.

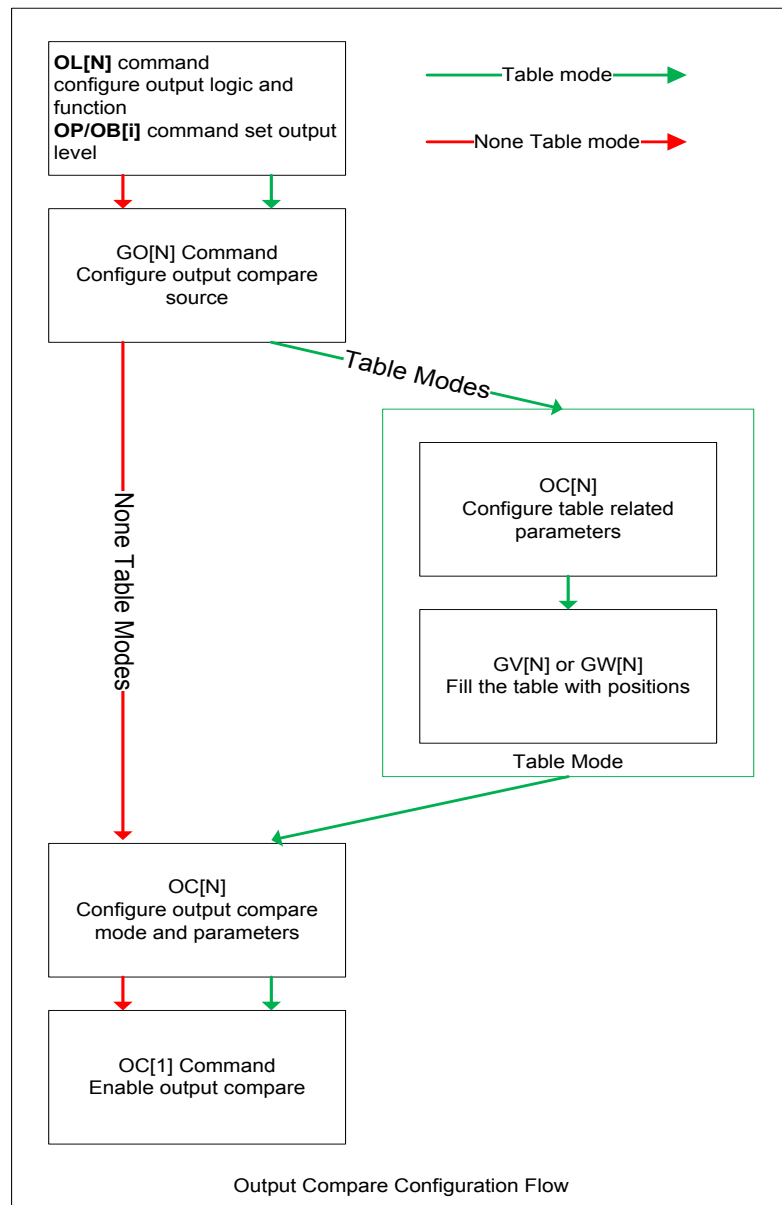


Figure 16: Output Compare Configuration Flow

After configuring and enabling the compare mode, the Output Compare status can be monitored using the following commands.

Command	Description in the Context of Output Compare	
OC[1]	Reports the status of the Output Compare feature.	
OC[21]	Value	Description
	-1	No more pulses are being generated because the number of pulses or table entries has been reached.
	0	The mode is disabled, and output is not controlled by the Output Compare function. The output definition is determined by the OL[M] command value.
	1	In absolute position Output Compare mode, the Output Compare function has started, but the absolute position has not yet been reached; therefore, the train of pulses has not begun. In a position table-based mode, the Output Compare function has started, but the first position in the table has not yet been reached. In time table-based mode, the Output Compare function has started, but the first position in the table has not yet been reached.
	2	The train of pulses is being generated.
OC[12] OC[32]	Reports the number of pulses generated, that is, the counter value.	

Table 13: Output Compare Status Report

13.5. Output Compare Comments and Limitations

- A Gold drive with SCORE hardware (i.e., the Gold Guitar) does not support Output Compare.
- For information about the limitations on the frequency of the generated pulses, see Section 3.5.
- It is the user's responsibility to verify that the table's array is not used by another Output Compare block.
- In order to prevent the generation of an additional pulse during activation or deactivation of the feature, it is recommended to configure the relevant output for the "active high" logic level by calling the **OL[N]** command before activating the Output Compare feature.
- In the absolute start position + delta mode, when movement is in the direction opposite to the direction specified by the **OC[3]** (or **OC[23]**) value, the compare will occur after every $(0xFFFFFFFF - OC[3])$ or $(0xFFFFFFFF - OC[23])$ counts.
- In the absolute start position + delta time mode, pulses with a long duration time might cause overlapping and a loss of output pulses.
- In table modes, the positions in the table are not checked for validity or order. An incorrect table can produce undefined pulse behavior.
- The following features cannot operate when Output Compare is configured on the same sensor source (and vice versa):
 1. Main/general home capture (**HM/HF**).
 2. DS 402 touch probe.
 3. DS 402 homing mode.
 4. Data recording when Output Compare is configured in table mode and the table's array is configured as BH.
 5. The ZX array is used internally during the Wizard. If the ZX array is used when Output Compare is configured in table mode, the position values must be entered again after Motor Tuning.
- In all modes, the first position must be separated by a time equal to at least **TS** from the activation position (the position of the drive at the time of activation by the **OC[1]** (or **OC[21]**) command). Note that in table-based modes the activation time might differ, depending on whether conversion is requested by the user or not. See **OC[11]/OC[31]** for the conversion options.
- In table modes, the data in the table are converted from user units to position units (i.e., socket units) during the activation of Output Compare (**OC[1] > 0/OC[31] > 0**). The converted values are stored in the same table, which modifies the original setting. In order to prevent multiple conversion during an additional activation with the same table, one of the following operations must be performed:
 1. Refill the table with positions before activation.
 2. Disable conversion of the positions in the table. See the **OC[11]/OC[31]** command.

- After conversion, all positions are in feedback units, i.e., socket position. If error mapping is enabled (**PC[1] != 0**) on the socket, the positions are in the corrected socket position values (including the error mapping).
- When error mapping is used, the compare positions are converted. Each conversion result can have up to 1 [count] error.
- In table modes, the table conversion, if specified, can take more than 70 [msec] in the case in which the BH array is selected as the compare table and error mapping is enabled. During the conversion time, the main loop is occupied by the conversion, and the start of the compare mode is delayed by the conversion time.
- In all modes, once Output Compare is enabled, the positions referred by Output Compare are relative to the socket position at the time when Output Compare is activated. If the socket position is changed while Output Compare is active, the compare positions are not automatically updated in order to correlate with the new position.
- The position compare table can be edited on-the-fly while Output Compare is enabled. The table can be edited while Output Compare table mode is enabled on the same table. This scenario is not protected.
- A logic compare table can be used by setting a new index for the same table (**OC[8]/OC[9]**) and filling the entries. After the next activation (toggling is required, for example, by calling **OC[1] = 0** and then **OC[1] = 1**), Output Compare will use the new table settings.
- Output 14 to 16 are fast outputs, allowing accurate pulse length and response time. Outputs 1 to 4 are (relatively) slow outputs, which might distort the pulse timing by adding additional time to the pulse. In many cases this does not matter, but the user should be aware of the HW abilities.

13.6. Minimum Time between Output Compare Pulses

The following table lists the maximum frequency between pulses according to the Output Compare mode enabled.

TS is the drive sampling time and is configured by calling the **TS** command.

Mode (OC[1] or OC[21])	Direction	Minimum Time between Pulses
1	All	1.5*TS
3	Positive	1.5*TS
	Negative	1.5*TS
	Both	2*TS
4	Positive	1.5*TS
	Negative	1.5*TS
	Both	2*TS

Table 14: Maximum Output Compare Frequency

Example:

For TS = 50 [μsec], the following table is applicable.

Mode (OC[1] or OC[21])	Direction	Minimum Time between Pulses for TS = 50 [μsec]
1	All	75 [μsec]
3	Positive	75 [μsec]
	Negative	75 [μsec]
	Both	100 [μsec]
4	Positive	75 [μsec]
	Negative	75 [μsec]
	Both	100 [μsec]

Table 15: Maximum Output Compare Frequency Example

Note: If the frequency of the pulses is above the supported frequency range, Output Compare may stop generating pulses.

13.7. Error Codes (EC) Related to Output Compare

The following table lists relevant error codes for Output Compare.

EC Value	Description
3	OC[N] command: Read/write to an unsupported index. GO[N] command: Assignment to an unsupported index. GV[N]/GW[N] command: Assignment/report when the table's array is undefined (OC[7] or OC[27]).
21	OC[N] command: Assignment to OC[N] with an incorrect value. GO[N] command: Assignment of an unsupported value.
67	Start Output Compare in table mode when the table array is BH and the recorder is active.
102	OC[N] command: Start Output Compare while Output Compare is in progress. RR command: Start recording while Output Compare is enabled in table mode and the BH table array is selected. HM/HF commands: Start capture while Output Compare is enabled on the same sensor. Other commands: Start touch probe while Output Compare is enabled on the same sensor.
103	Start Output Compare on a socket that is not defined as EQEP (AqB sensor, see OC[6]/OC[26]).
104	Start Output Compare in table mode (OC[1]==3 4), and the table parameters are incorrect: Invalid start/end index for the selected table array. In position table mode, the number of table entries is odd.
184	Trying to start Output Compare on a socket that is already activated in capture mode (HM/HF/touch probe/DS 402 homing). Trying to start Output Compare on a socket that is already using Output Compare.

Table 16: Error Codes (EC) Related to Output Compare

13.8. Output Compare Examples

This section includes some examples of operating the Output Compare feature.

The examples are in user program language. For more information about this language, see the *User Program Manual* (MAN-G-USRPGM).

13.8.1. Absolute + Delta Output Compare Examples

The following example generates:

- 100 pulses from 25000 [cnt] with a delta of 25 [cnt] on output 1.
- The duration of each pulse is 300 [μsec].
- The main position socket is configured as Port A (EQEP 2).

```
/* Initialize Output Compare */
GO[1]=2          /* route output-1 to compare on Port A (EQEP-2) */
OC[2]=25000      /* start ABS position 25000 [cnt] */
OC[3]=25         /* delta position 25 [cnt] */
OC[4]=129        /* pulse duration 300 [μsec] */
OC[5]=100        /* number of pulses 100 */
OC[6]=0          /* take main position feedback as the socket */
OC[1]=1          /* enable Output Compare - ABS + Delta */

/* Wait until all the pulses are generated */
while(OC[12]!=100)
end

/* Stop Output Compare and return output to OL[N] configuration */
OC[1]=0
```

The following example generates:

- 5 pulses from 25000 [cnt] with a delta of 250 [cnt] on output 1.
- Then it generates 5 pulses from 26000 [cnt] with a delta of -250 [cnt] on output 1 (this is in the direction that is opposite to the direction of the generated pulses in the previous stage).
- The duration of each pulse is 200 [μsec].
- The main position socket is configured as Port A (EQEP 2).

```
/* Initialize Output Compare */
GO[1]=2      /* route output-1 to compare on Port A (EQEP-2) */
OC[2]=25000  /* start ABS position 25000 [cnt] */
OC[3]=250    /* delta position 250 [cnt] */
OC[4]=128    /* pulse duration 200 [μsec] */
OC[5]=5      /* number of pulses 5 */
OC[6]=0      /* take main position feedback as the socket */
OC[1]=1      /* enable Output Compare - ABS + Delta */

/* prepare the next pulse generating */
OC[2]=26000  /* start from 26000 [cnt] */
OC[3]=-250   /* delta position is -250 [cnt] negative direction */

/* wait until all the pulses are generated */
while(OC[1]!=-1)
end

/* start compare on the negative direction, the data already configured
above */
OC[1]=1      /* enable Output Compare - ABS + Delta */

/* wait until all the pulses are generated */
while(OC[12]!=5)
end

/* stop Output Compare and return output to OL[N] configuration */
OC[1]=0
```


13.8.2. Position Table-Based Output Compare Examples

The following example generates:

- 50 pulses from 25000 [cnt] on output 2.
- The duration of each pulse is 100 [cnt].
- The mode is enabled at position 0, and the direction is positive.
- The main position socket is configured as Port B (EQEP 1).

```
int i
GO[2]=1          /* route output-2 to Port B (EQEP-1) */
OC[7]=1          /* select ZX array as a table */
OC[8]=1          /* position table start index */
OC[9]=100        /* position table end index */

/* fill the table with 50 pairs of positions, starting from 25000 */
i=3
GV[1]=25000
GV[2]=25100
while(i<100)
    GV[i]=GV[i-1]+50
    GV[i+1]=GV[i]+100
    i=i+2
end

OC[6]=0          /* take main position feedback as the socket */
OC[10]=1         /* positive direction */
OC[1]=3          /* enable Output Compare - position table-based */

/* wait for compare mode to be done */
while(OC[1]!=-1)
end

/* stop Output Compare and return output to OL[N] configuration */
OC[1]=0
```

The following example uses the same table as the previous example, but the motion is from 45000 [cnt] to 0 [cnt] position.

- The main position socket is configured as Port B (EQEP 1).

```
int i
GO[2]=1      /* route output-2 to Port B (EQEP-1) */
OC[7]=1      /* select ZX array as a table */
OC[8]=100    /* position table start index */
OC[9]=1      /* position table end index */

/* fill the table with 50 pairs of positions, starting from 25000 */
i=3
GV[1]=25000
GV[2]=25100
while(i<100)
    GV[i]=GV[i-1]+50
    GV[i+1]=GV[i]+100
    i=i+2
end

OC[6]=0      /* take main position feedback as the socket */
OC[10]=2     /* positive direction */
OC[1]=3      /* enable Output Compare - position table-based*/

/* wait for compare mode to be done */
while(OC[1]!=-1)
end

/* stop Output Compare and return output to OL[N] configuration */
OC[1]=0
```

13.8.3. Time Table-Based Output Compare Examples

The following example generates:

- 6 pulses from 10000 [cnt] with a delta of 250 [cnt] on output 14.
- The duration of each pulse is 50 [μsec].
- The main position socket is configured as Port A (EQEP 2).

```
int i
GO[14]=2      /* route output-14 to Port A (EQEP-2) */
OC[27]=2      /* select NT array as a table */
OC[28]=10     /* position table start index */
OC[29]=15     /* position table end index */

/* fill the table with 6 positions */
i=10
while(i<16)
    GW[i]=10000 +((i-OC[28])*250)
    i=i+1
end

OC[24]=50     /* pulse duration 50 [μsec] */
OC[26]=0      /* take main position feedback as the socket */
OC[30]=1      /* positive direction */
OC[21]=4      /* enable Output Compare - time table-based*/

/* wait for compare mode to be done */
while(OC[21]!=-1)
end

/* stop Output Compare and return output to OL[N] configuration */
OC[21]=0
```

The following example uses the same table as the previous example, but the motion is from 15000 [cnt] to 0 [cnt] position.

- The main position socket is configured as Port A (EQEP 2).

```
int i
GO[14]=2      /* route output-14 to Port A (EQEP-2) */
OC[27]=2      /* select NT array as a table */
OC[28]=15     /* position table start index */
OC[29]=10     /* position table end index */

/* fill the table with 6 positions */
i=10
while(i<16)
    GW[i]=10000 +((i-OC[28]*250)
    i=i+1
end

OC[24]=50     /* pulse duration 50 [µsec] */
OC[26]=0      /* take main position feedback as the socket */
OC[30]=2      /* negative direction */
OC[21]=4      /* enable Output Compare - time table-based*/

/* wait for compare mode to be done */
while(OC[21]!=-1)
end

/* stop Output Compare and return output to OL[N] configuration */
OC[21]=0
```

Chapter 14: Feedback Emulation

14.1. Introduction

This chapter describes the Feedback Emulation feature supported by Gold line drives.

The Feedback Emulation feature emulates any feedback (socket) in one of the following formats:

7. A/B quadrature format
8. Pulse/direction format
9. Up/down format
10. Hall signal format

For more details about the signals, refer to Supported Types of Feedback Emulation.

A typical drive setup includes:

- The emulated feedback socket – the feedback sensor socket to be emulated. It is configured by setting the **CA[41 to 44]** commands.
- Port C must be configured as the feedback emulation output. This is done by setting the **GO[14]** and **GO[15]** commands.
- The **EA[]** parameter must be configured by the user according to the required functionality.
- Start the emulation by calling the **EA[1]** command.

Note: Daisy Chain. If the emulated socket is quadrature, there is an option to connect the quadrature input port directly to Port C without using emulation. For more information, see the Daisy Chain option in the GO[] command.

The following figure shows a typical drive emulation flow.

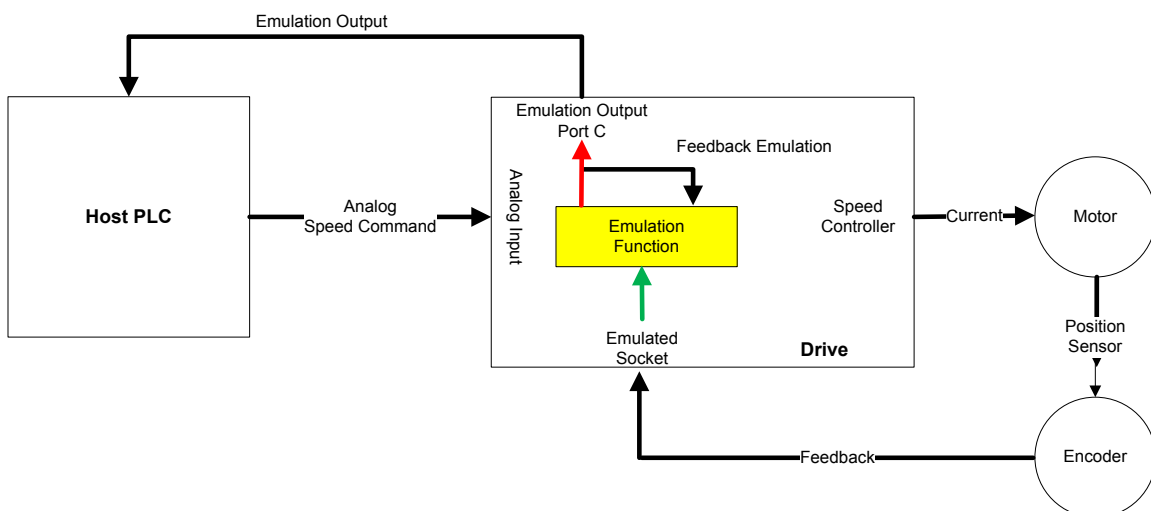


Figure 17: Encoder Emulation Setup with an Emulation Output Socket on Port A/B

14.2. Commands Related to Feedback Emulation

This section describes the relevant commands in general terms.

For detailed specifications of the commands, refer to the Command Reference for Gold Line Drives.

The following table lists the commands that specify the feedback emulation configuration.

Command	Feedback Emulation-Related Description
OL[N]	Defines the output logic of the drive and is used to configure the output logic level of Port C as “active high” or “active low”.
GO[N]	Routes the emulation functions to Port C output channels.
CA[N]	Enables configuration of the emulated socket. Defines the emulation output socket.
EA[N]	Enables the configuration and activation of the Feedback Emulation function.

Table 17: Commands Related to Feedback Emulation

The following subsections describe the commands that contribute to the feedback emulation mode.

14.2.1. OL[N] Command

The **OL[14]** and **OL[15]** commands are used to configure the Port C output logic levels as “active high” or “active low”.

When the Port C output channels are configured in the **GO[]** command to be connected to the emulation output, the functionality configured in the **OL[]** command is ignored, and only the logic level is used. When the emulation is completed, the previous functionality of the output is not restored, and the user must set **GO[]** for that purpose.

14.2.2. GO[N] Command

With the **GO[14]** and **GO[15]** commands, the feedback emulation output is connected to Port C pins. Setting **GO[14]** and/or **GO[15]** to **5** connects the A and/or B pins of Port C to the feedback emulation output.

Note: **GO[14] and GO[15] should be set before enabling the emulation (EA[1]), if these commands are set after the emulation is enabled, the emulation will not produce pulses on Port C.**

14.2.3. CA[N] Command

The **CA[41 to 44]** commands are used to do the following:

1. Configure the emulated sensor's socket.
2. Configure the emulation feedback quadrature socket (where **EA[8]** is not **0**)

14.2.4. EA[N] Command

The **EA[1]** command is used to start feedback emulation with one of the supported wave formats or to stop an ongoing emulation.

The **EA[2 to 8]** commands are used to configure the emulation mode, such as the pulse width, the multiplier and scale factor between original pulses and emulated pulses, the emulated direction and more.

Note: The **EA[]** command is non-volatile, and if **EA[1]** is saved while emulation is operating, the next time that the drive is powered up the emulation will start automatically.

14.3. Supported Types of Feedback Emulation

A Gold line drive supports 4 types of output wave formats, which are specified by the value set in the **EA[1]** command.

EA[1] Value	Feedback Emulation Wave Format
0	Disable emulation
1	Quadrature A/B wave format
2	Pulse/direction A/B wave format
3	Up/down A/B wave format
4	Hall signal wave format

Table 18: Feedback Emulation Output Wave Formats (EA[1] Values)

Using **GO[14]**, **GO[15]** and **GO[16]**, the Feedback Emulation function signals can be outputted to Port C channels A, B and I, respectively.

The following paragraphs describe the four emulation output formats:

1. **Quadrature emulation output.** In the Quad A/B format, there are two output waves, which are called A and B. The two output waves are 90 degrees out-of-phase from each other. These signals are decoded to produce a count up or count down. The direction of motion is defined by the following tables:

Phase	A Signal	B Signal
1	1	0
2	1	1
3	0	1
4	0	0

Table 19: Clockwise Quad A/B State

Phase	A Signal	B Signal
1	0	0
2	0	1
3	1	1
4	1	0

Table 20: Counterclockwise Quad A/B State

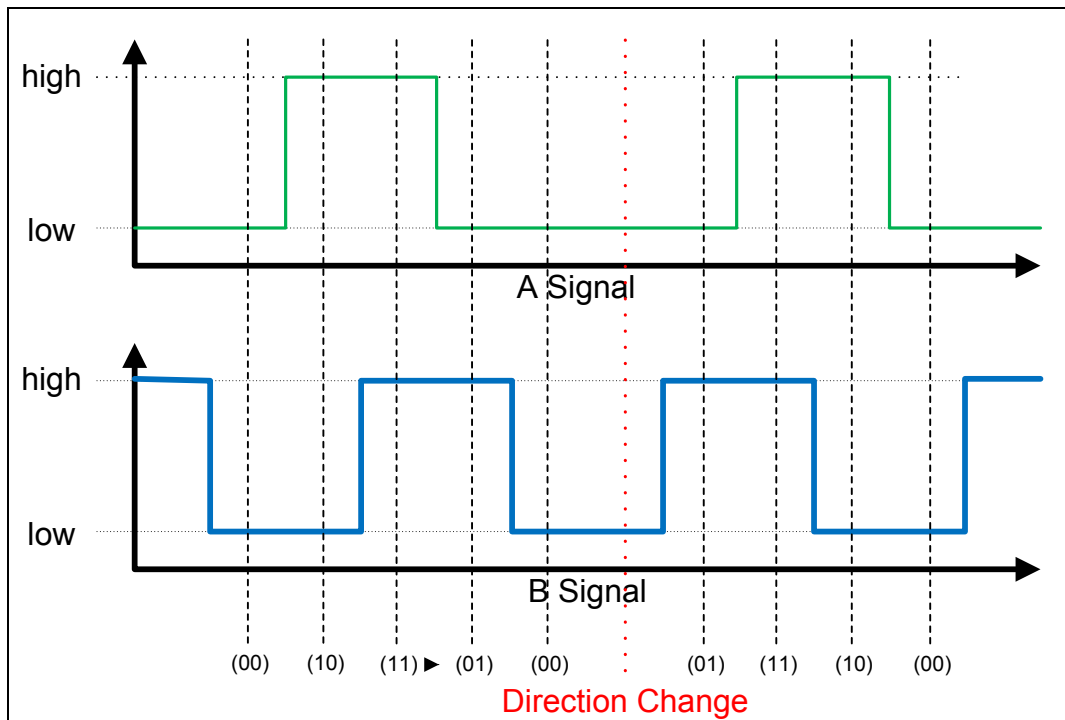


Figure 18: Quadrature Signal Definition

2. **Pulse/direction emulation output.** The pulse/direction waves include two signals:
 - The first wave signal (which is routed to channel A of Port C) includes pulses. Each pulse indicates a single count.
 - The second wave signal (B) indicates the direction of the pulses in signal A. Its amplitude is low for the negative direction and high for the positive direction.

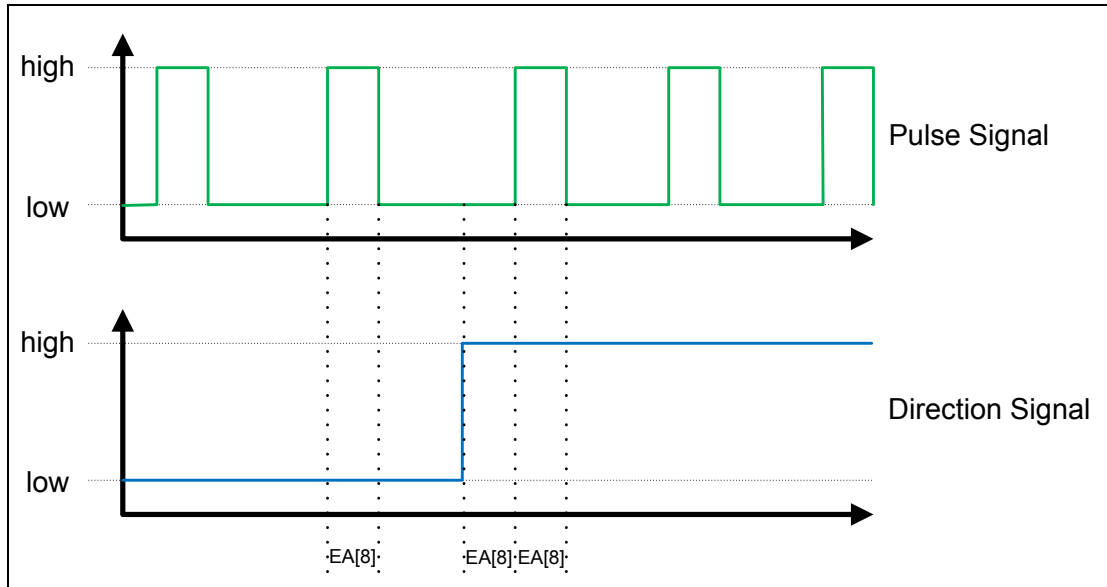


Figure 19: Pulse/Direction Signal Definition

3. **Up/down emulation output.** The up/down output includes two waves:
 - The format of the first wave (on channel A of Port C) consists of pulses in the clockwise direction. Each pulse represents a count “up”.
 - The format of the second wave (on channel B of Port C) consists of pulses in the counterclockwise direction. Each pulse represents a count “down”.

Note: There cannot be pulses on channels A and B at the same time.

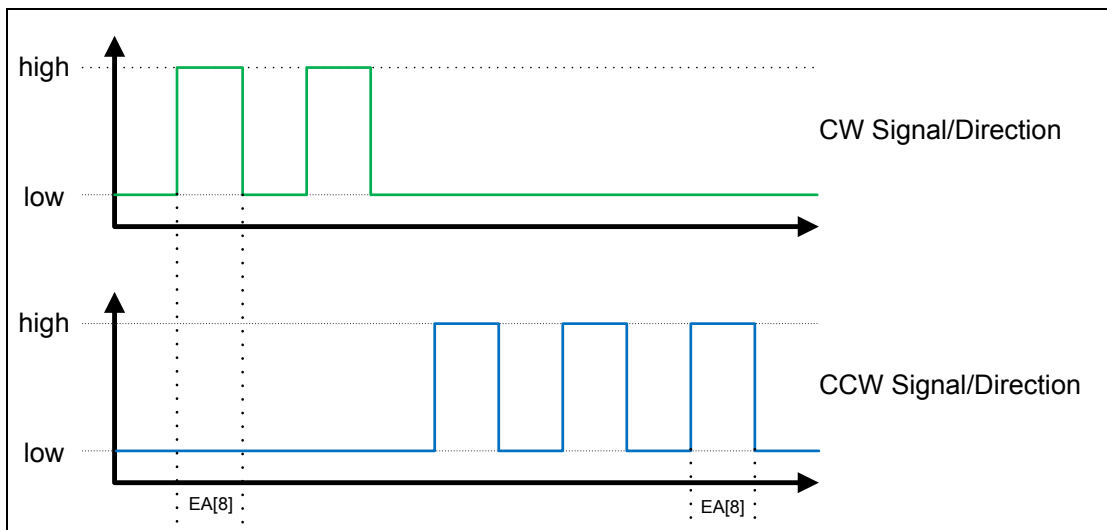


Figure 20: Up/Down Signal Definition

4. **Hall emulation output.** In this mode the emulation output is in the Hall signal format. The Hall output includes three waves:
 - The signal in channel A of Port C is the Hall A signal.
 - The signal in channel B of Port C is the Hall B signal.
 - The signal in channel I of Port C is the Hall C signal.

The following table describes the Hall A, B and C signal states:

Hall State	Hall A/Port C-A	Hall B/Port C-B	Hall C/Port C-I
1	0	0	1
3	0	1	1
2	0	1	0
6	1	1	0
4	1	0	0
5	1	0	1

Table 21: Hall A, B and C Signal States

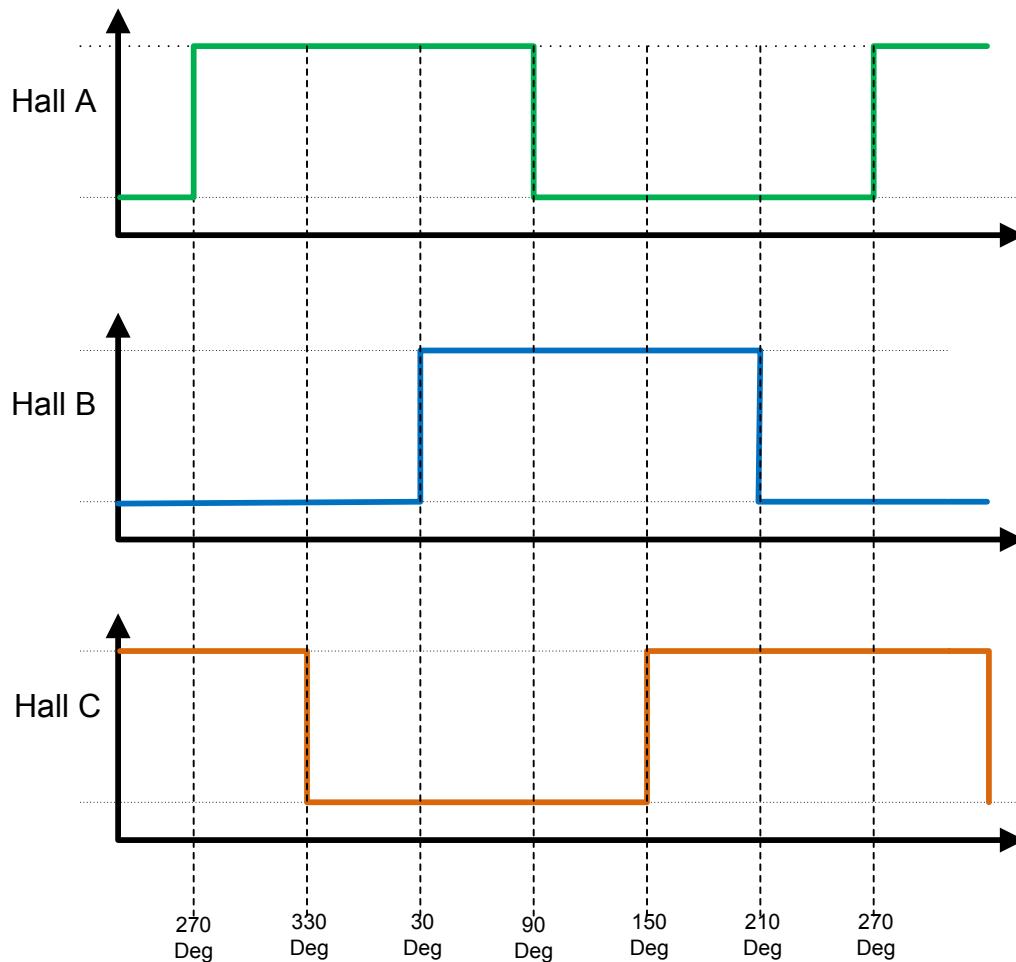


Figure 21: Hall Emulation Format

Notes:

After emulation is enabled or re-enabled, the first Hall state is always state 5.

The Hall state changes according to the direction of the emulated socket on every emulated socket count.

In most cases the emulated Hall output should reflect the motor angle. This can be achieved using the emulation multiplier and scale factor. The use of these parameters will produce the Hall state according to the rotary angle.

14.4. Feedback Emulation Configuration

To configure feedback emulation, follow the steps:

1. Define the sensor socket to be emulated by setting the **CA[]** command.
2. If emulation feedback is selected as **AqB (EA[8] != 0)**, define the emulation feedback quadrature socket by the **CA[]** command.
3. Configure the emulation parameters by setting the **EA[]** command.
4. Configure the Port C output pin logic level as “active high” or “active low” by setting **OL[M]**.
5. Connect the Port C output pins to the emulation output by setting the **GO[]** command.
6. Start the emulation by setting **EA[1] > 0**. The **EA[1]** value selects the type of wave that will be produced by the emulation.

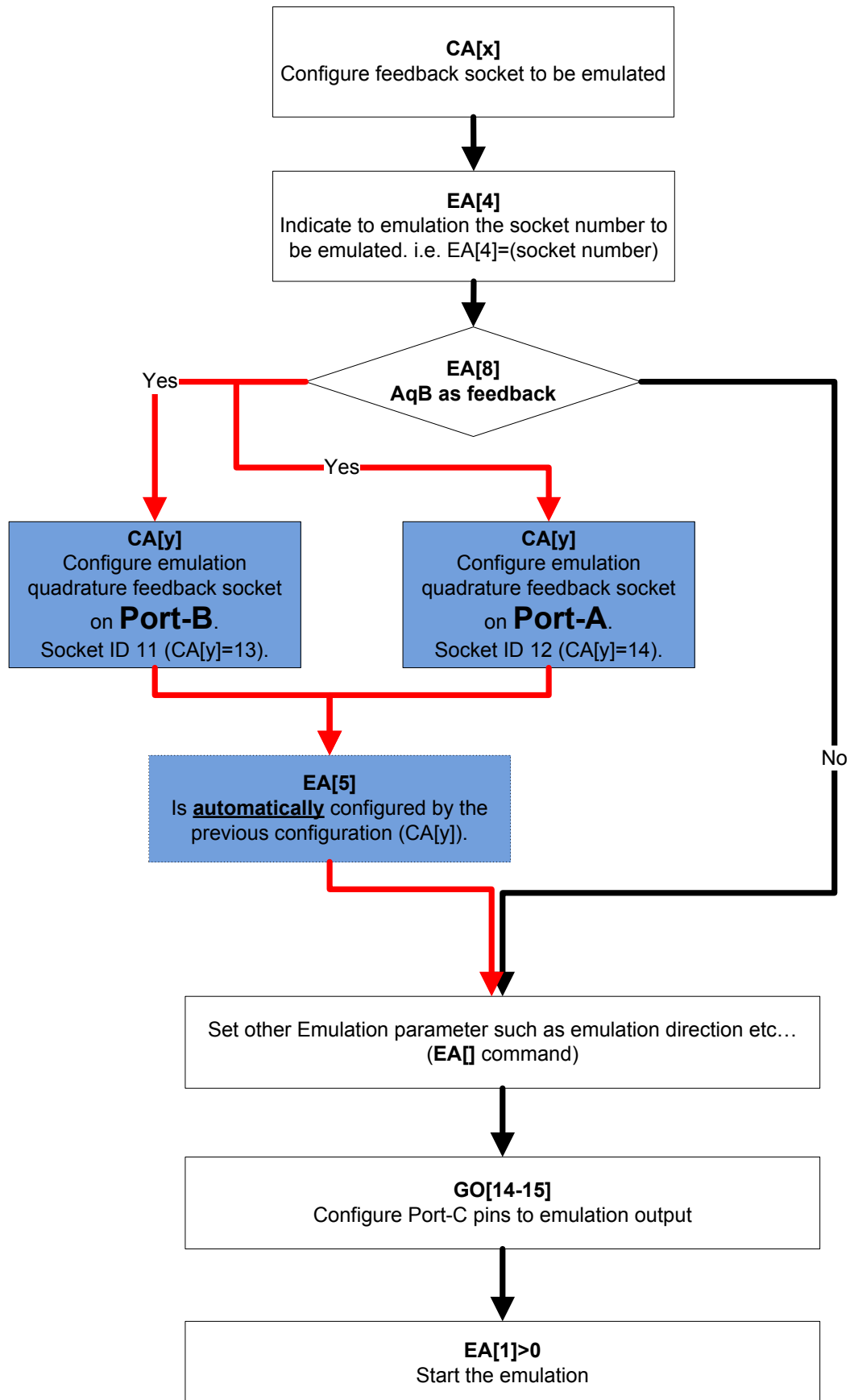


Figure 22: Example of Feedback Emulation Configuration Flow

14.5. Feedback Emulation Comments and Limitations

This section lists the limitations and some comments regarding the feedback emulation feature.

- Feedback emulation can generate up to 8,000,000 pulses per second.
- If the follower error between the emulated socket and the emulation position is higher than $\pm 1,000,000,000$, the emulation will stop, and **EA[1]** will report an error (see the error table below). In case of an error, the **user must disable the emulation** before re-enabling it.
- In cases in which the wave format is pulse and direction or up/down counter, the pulse width should not be greater than the pulse frequency. Otherwise, the pulses might overlap and produce a single endless pulse.
- Feedback emulation is supported only in drives with supported hardware (GCON core).
- While the emulation is active, the user must select one of the Quad A/B sockets to be used as the internal emulation output socket. For this reason, one of the Quad A/B sensors will be occupied by the emulation feedback and will not be available to the user.
- The Port C outputs must be configured for emulation, i.e., **GO[14–16] = 5**, before enabling feedback emulation.

14.6. Feedback Emulation Runtime Errors

During emulation, the Feedback Emulation feature checks for possible errors. If Feedback Emulation detects an error, the following logic is applied:

- The emulation is automatically stopped.
- The **EA[1]** command will report the error value (error values are < 0).
- After an error, the user must disable the emulation, i.e., set **EA[1]=0**, before re-enabling emulation.

Error Value in EA[1]	Feedback Emulation Error Description
-1	Follower position error is greater than $\pm 1,000,000,000$.

Table 22: Feedback Emulation Error Values (EA[1] Values)

14.7. Feedback Emulation Error Codes (EC)

The following table lists relevant error codes for feedback emulation.

EC Value	Description
2	CA[41 to 44] commands: The selected socket hardware is already occupied. For example, this can occur when the following are set: CA[41]=1 and CA[42]=13 or CA[41]=14 and CA[42]=2.
21	EA[1] command: The EA[1] value is not supported. EA[2] command: The emulation pulse width must be in the range from 2 to 202. EA[3] command: The emulation direction can be 0 or 1. EA[4] command: The emulated socket number is out of range. EA[6] command: The emulated multiplier value must be from 1 to 32767. EA[7] command: The emulated divider value can be between 0 and 31. EA[8] command: The emulation feedback value can be between 0 and 1.
3	EA[5] command This is a read-only parameter. Any attempt to write to this value will return this error.
107	EA[1] > 0 command (Enable emulation): The emulated socket number is not entered into EA[4]. The emulation output socket is not configured (EA[5] == 0). Note that this value is automatically updated by the CA[41 to 44] commands when the sensor ID is set to 13 or 14. This is relevant only if EA[8] is not 0. The emulation multiplier set in EA[6] is 0. The emulation pulse width set in EA[2] is out of range.
108	EA[1] > 0 command (Enable emulation) An attempt was made to run emulation while emulation is in progress. An attempt was made to run emulation after an emulation error without first disabling emulation.

Table 23: Feedback Emulation Error Codes (EC)

14.8. Feedback Emulation Examples

This section includes some examples of operating the Feedback Emulation feature.

The examples are in user program language. For more information about this language, see the *User Program Manual* (MAN-G-USRPGM).

14.8.1. Port A Quadrature Feedback Emulation to Pulse/Direction Example

The following example initializes the Feedback Emulation feature to produce pulse/direction wave output on the A/B pins of Port C. The emulated socket is Port A configured as socket #1, and the emulation output socket is configured on Port B as socket #2.

```
/* Configure sockets related parameters */
CA[41]=2;      /* Quadrature on Port A */
EA[4]=1;      /* run emulation on socket number-1 (Port A) */
CA[42]=13;    /* emulation feedback quadrature on Port B
               Note - this will also update EA[5] to 2 */

/* Connect Port C pins to feedback emulation, and set the logic */
GO[14]=5;     /* connect Port C pin A to feedback emulation */
GO[15]=5;     /* connect Port C pin B to feedback emulation */
OL[14]=1;     /* Port C pin A is "Active-High" */
OL[15]=1;     /* Port C pin B is "Active-High" */

/* Configure other feedback emulation parameters */
EA[2]=3;      /* pulse width 3*13.3=40[nSec] */
EA[3]=0;      /* do not inverse the direction */
EA[6]=1;      /* multiplier 1 */
EA[7]=0;      /* emulation divider 0 - i.e. no shift right */
EA[8]=1;      /* use AqB socket as emulation feedback */

/* Start emulation with pulse/direction output */
EA[1]=2;
```

14.8.2. Port B Quadrature Feedback Emulation to Up/Down Example

The following example initializes the Feedback Emulation feature to produce up/down wave output to the A/B pins of Port C. The emulated socket is Port B configured as socket #2, and the emulation output socket is configured on Port A as socket #1.

```
/* Configure socket-related parameters */
CA[42]=1;     /* Quadrature on Port B socket #2 */
EA[4]=2;     /* run emulation on socket number-2 (Port B) */
CA[41]=14;    /* emulation feedback quadrature on Port A
               Note - this will also update EA[5] to 1 */

/* Connect Port C pins to feedback emulation, and set the logic */
GO[14]=5;     /* connect Port C pin A to feedback emulation */
GO[15]=5;     /* connect Port C pin B to feedback emulation */

```



```
OL[14]=1;      /* Port C pin A is "Active-High" */
OL[15]=1;      /* Port C pin B is "Active-High" */

/* Configure other feedback emulation parameters */
EA[2]=76;      /* pulse width (76-75+1)*1[μsec]= 2[μsec] */
EA[3]=0;      /* do not inverse the direction */
EA[6]=1;      /* multiplier 1 */
EA[7]=0;      /* emulation divider 0 - i.e. no shift right */
EA[8]=1;      /* use AqB socket as emulation feedback */

/* Start emulation with Up/Down output */
EA[1]=3;
```

14.8.3. Port A BiSS Feedback Emulation to Quadrature Example

The following example initializes the Feedback Emulation feature to produce quadrature wave output to the A/B pins of Port C. The emulated sensor is BiSS configured on Port A as socket #2, and the emulation quadrature feedback is configured on Port A as socket #1. The emulation output signal generates ½ of the emulated feedback.

```
CA[41]=5;      /* BiSS encoder on Port A */
EA[4]=1;      /* run emulation on socket number-1 (BiSS) */
CA[42]=13;     /* emulation feedback quadrature on Port B
                Note - this will also update EA[5] to 2 */

/* Connect Port C pins to feedback emulation, and set the logic */
GO[14]=5;      /* connect Port C pin A to feedback emulation */
GO[15]=5;      /* connect Port C pin B to feedback emulation */
OL[14]=1;      /* Port C pin A is "Active-High" */
OL[15]=1;      /* Port C pin B is "Active-High" */

/* Configure other feedback emulation parameters */
EA[2]=2;      /* pulse width 26.6[nsec] - (not used in A/B wave) */
EA[3]=0;      /* do not reverse the direction */
EA[6]=1;      /* multiplier 1 */
EA[7]=0;      /* emulation divider 0 - i.e. no shift right */
EA[8]=0;      /* do not use AqB socket as emulation feedback */

/* Start emulation with quadrature output */
EA[1]=1;
```

Chapter 15: Buffering



Chapter 16: Operating Gold Drives with the Gold Maestro (G-MAS)

16.1. Scope and Purpose

This chapter introduces the commands for Elmo's Gold servo drives. It covers somewhat more than the commands, since it aims to give a brief introduction about the abilities and features of these drives.

16.2. General

Along with the Gold servo drives, the Gold family includes Elmo Application Studio (EAS) and the Gold Maestro (G-MAS).

The G-MAS is a multi-axis controller, which controls a network over CANopen or EtherCAT communication.

EAS is a PC interface that enables straightforward communication to a servo drive for application setup, diagnostics, programming and more. It can communicate with Gold drives and the G-MAS.

16.3. Concept and Features

The main concept of the Gold drives is to provide an advanced motion control environment for any field in the motion industry for controlling a single axis within CAN or EtherCAT networks.

16.3.1. Compatibility

We strive to maintain compatibility with Elmo's previous generations (SimplIQ and Metronome). This concept is natural for allowing easy migration between generations.

However, in some cases, due to the far more advanced abilities of the Gold servo drives, this compatibility could not be maintained. These points are described in the Compatibility Appendix and, in some cases, in the relevant commands.

16.3.2. Motion Concept

The motion is handled according to the CANopen DS 402 v3.0 standard, which is used by both types of networks: CAN and EtherCAT. For the sake of compatibility, a simple motion can be set by using Elmo's motion commands.

CANopen commands are given by *objects*. Each object refers to a callback function, which performs the requested command. **Elmo commands** are typically denoted by 2 characters. In most cases CANopen objects and Elmo commands can be addressed from one another. The related commands are marked in the command table.

Note that conflicts may occur when both methods (Elmo commands and DS 402) are used simultaneously or alternately. As a rule of thumb, we can stipulate that "the last command

**Maestro (G-MAS)**

MAN-G-ADMINING (Ver. 1.100)

wins”; however, setting motion commands from the Elmo interface (**AC, DC, JV, PA** etc) should be considered only after the Elmo **BG** (motion begin) command.

Another point to be clarified is that the different profilers in the drive, as well as the control loops, use a single instance of each parameter for real-time calculations. This instance is typically influenced by the last command given.

16.3.3. User Program

A Gold drive can be programmed as a standalone entity with the User Program interface as in the case of a SimplIQ drive. The programming and debugging environment is one of the tools provided by EAS. For more details about programming Gold line drives, refer to the Gold Language & User Program Manual.

16.3.4. Communication Concept

Two types of communication are distinguished:

1. Fast communication for synchronization and control purposes
2. Background communication for setup, downloads, upload and diagnostics

The types of background communication that are enabled in the Gold drives are:

- USB
- UDP
- Telnet
- EtherCAT EoE
- EtherCAT CoE mailbox
- CANopen SDO
- CANopen binary interpreter (PDO2 default)
- RS232 (in some models)

The types of fast communication are:

- CANopen PDO
- EtherCAT CoE with data processing using Sync Manager 2 or 3

Each of the background communication channels can be addressed at any time. The drive will distinguish between the commands and will address the response to the correct channel.

Note: Recording and any of the uploading or downloading procedures can be performed with only one type of communication at one time. (For example, data cannot be uploaded from USB and TCP channels at the same time). The drive does not check for this, and it is strongly recommended that no attempt should be made to do it. EtherCAT FoE is used as part of the firmware downloading sequence.

**Maestro (G-MAS)**

MAN-G-ADMINING (Ver. 1.100)

16.3.5. Interpreters

With respect to the communication concept (as mentioned above), different interpreters are used in Gold drives.

- **Elmo's interpreter.** This 2-character interpreter is used for ASCII interpretation and supports line termination by either a semicolon (;) or a carriage return (CR). USB, UDP, Telnet and EoE are addressed to this interpreter.
- **The binary interpreter.** When CANopen is used, the binary interpreter can be used to address Elmo commands in a compatible method. (For more information, refer to the CANopen Drives and Motion control device profile document)
- **The OS interpreter.** The OS interpreter allows the user to address Elmo's interpreter using SDO messages in a regular ASCII format.
- **The CANopen stack.** The CANopen stack is used to interpret all the CANopen objects from both CAN and EtherCAT channels.

16.3.6. CPUs and Memory

Basically there are three main processes in the drive:

1. Profiler processes and communication
2. Control-loop processes
3. Background sequences

To allow the network to be synchronized in a motion level, the fast communication task and the profile task are synchronized. In CANopen this happens when a SYNC command is issued, and in EtherCAT this is achieved when Distributed Clock (DC) is used.

The drive has internal flash memory, which contains, among other things, user parameters, the user program, the CANopen EDS file and the EtherCAT DDF (XML) file in a zipped format.

The user parameters comprise a list of parameters, which are loaded each time that boot up is performed (power up, soft reset, watchdog...) and when object 0x10## or the **LD** command is used. During the loading, all the relevant internal parameters are initiated using a post-process sequence. If one of the parameters fails to perform, the user parameter database is reset to the default. Use the **CD** (CPU dump) command to detect which of the parameters failed.

The list of parameters and the default values are included in the command table.

16.3.7. Position Reference and Modulo Concept

The drive manages two sets of position coordinates: one for the user and one for control. The user position (which is denoted, for example, by PosUU) is a user unit value, which is handled in modulo. The modulo high and low limits are set by object 0x607B. Setting **XM[1]/XM[2]** will override these values.

When the motor is enabled (by any means), all the relevant position derivatives are set to PosUU₀.

**Maestro (G-MAS)**

MAN-G-ADMINING (Ver. 1.100)

16.3.8. Indications

Indications refer to status and fault signals. The most obvious one is the LED. The LED displays the following status.

16.3.8.1. LED Indications

Red for a few milliseconds indicates BIOS setup, blank for few seconds indicates boot-up, and **green** or **red** indicate that the firmware is running. Here green indicates all OK, and red indicates an amplifier fault. See the table of amplifier fault status indications below for details.

The following faults are also indicated during power-up:

- **Red** blinks forever. Boot is corrupt.
- **Red** and **green** blink alternately. No boot in the drive. The firmware cannot be activated.
- **Red** blinks 3 times. The firmware is corrupt. The firmware must be downloaded again.
- **Red** blinks 5 times. The firmware is not present in the drive. Download the firmware.

16.3.8.2. Motor Fault Indications

A motor fault occurs when the servo was shut due to a problem.

The **MF** command latches the reason for a fault and presents it in a bit-field format. The following table details the possible faults and troubleshooting options.

MF Fault	Details
1	Main feedback error
2	Auxiliary feedback error (reserved)
4	Hall feedback mismatch
8	Current exceeded the peak limit.
16	External inhibit input is enabled.
32	Reserved
64	Hall sensor speed is too high.
128	Speed tracking error
256	Position following error
512	Parameter database failure or conflict between parameters
1024	ECAM table problem (reserved)
2048	Heartbeat event
4096:32768	Amplifier problem: undervoltage, overvoltage, short protection, overtemperature, disabled safety input. See the table of amplifier fault status indications.

**Maestro (G-MAS)**

MAN-G-ADMING (Ver. 1.100)

MF Fault	Details
65536	Encoder only phasing problem
131072	Speed over HL[2] or LL[2] values
262144	Stack overflow (reserved)
1048576	Internal timing problem
2097152	Motor is stuck according to CL[2] , CL[3] (reserved).
4194304	Feedback is outside the position limits (HL[3] , LL[3]).
8388608	Mathematical overflow. Ambiguity in results.

The following table lists the amplifier fault status indications. (These will cause a **red** LED.)

MF Indication	Fault Reason
12288	Undervoltage
20480	Overvoltage
28672	Safety input is not active.
45056	Short protection
53248	Overtemperature

When EtherCAT or CANopen are used and an emergency message is produced when a motor fault occurs, the following table lists the possible reasons.

Error Register	EMCY Code (Hex)	
81	7300	Main feedback error
21	7306	Auxiliary feedback fault (reserved)
81	7380	Hall and main feedback mismatch
21	8311	Current exceeded the peak limit for a long time.
21	5441	External inhibit input is enabled.
81	52FF	Reserved
81	7381	Hall sensor speed is too high.
81	8480	Speed tracking error
21	8611	Position following error
21	6320	Parameter database failure or conflict between parameters

**Maestro (G-MAS)**

MAN-G-ADMING (Ver. 1.100)

Error Register	EMCY Code (Hex)	
81	5280	ECAM table problem (reserved)
11	8130	Heartbeat event
81	8380	Encoder only phasing problem
81	8481	Speed is outside the range of HL[2] or LL[2] value.
81	6180	Stack overflow (Reserved)
81	6181	Reserved
81	5281	Internal timing problem
21	7121	Motor is stuck according to CL[2] , CL[3] (reserved)
81	868	Feedback is outside the position limits (HL[3] , LL[3]).
81	8381	(Reserved)
81	FF34	Numeric overflow limit.
81	FF10	Cannot start motor - check parameters
81	FF30	Mathematical overflow. Ambiguity in results.
5	3120	Undervoltage
5	3310	Overvoltage
5	FF20	Safety input is not active
3	2311	Overcurrent
3	2340	Short between phases or ground to phase
9	4310	Overtemperature
81	5282	Reserved

16.3.8.3. Status Register Indications

The status register is reflected by the **SR** command or CANopen object 0x1002. It is updated constantly and upon request. The bits are not latched.

The following is indicated.

Bit in SR (Object 0x1002)	Details
0:3	Amplifier status: 0: All OK, 3: Undervoltage

**Maestro (G-MAS)**

MAN-G-ADMINING (Ver. 1.100)

Bit in SR (Object 0x1002)	Details
	5: Overvoltage 7: Safety input is disabled 11: Short protection 13: Overtemperature protection
4	Motor is enabled (SO is 1)
6	Fault occurred. The reason is in the MF command.
8:9	Units mode: Reflects UM (See mode of operation) 1: Current mode 2: Velocity mode 3: Stepper mode 5: Position mode correct this input
10	Gain schedule is active (GS[2] is 64).
13	Current command is saturated to the CL[1] value.
14:15	Safety input 1/safety input 2 is enabled (1: enabled, 0: disabled).
16:17	Recorder status: 0: Recorder is at rest. 1: Waiting for trigger. 2: Recorder has finished. Data are ready for uploading. 3: Trigger has been reached. Recording is in progress.

* All unused bits are reserved.

16.3.9. Feedbacks

The Gold drives have three physical feedback ports, which are used for position and velocity control, field angle control (commutation), feedback emulation and reference input for follower applications. The following sensors are supported:

- Digital encoder
- Resolver
- Analog encoder
- Analog Halls
- Digital Halls
- SSI and NRZ absolute digital feedback (such as BiSS, Tamgawa, Panasonic, EnDAT 2.2##)

**Maestro (G-MAS)**

MAN-G-ADMING (Ver. 1.100)

The feedback inputs are handled via 2 physical ports, which are designated as Port A and Port B.

There are 4 virtual inputs (sockets), which are used as software representatives of the physical feedbacks. Each of these sockets can be mapped to any of the available feedbacks. Each of these sockets can be addressed for a specific function. Thus, for example, socket #1 can be mapped to a digital encoder, socket #2 to the analog Halls and socket #3 to the digital Halls. Then the position feedback can be taken from socket #1, the velocity feedback from socket #2 and the commutation from socket #3. In our example socket #2 can be addressed to Port 3 as emulation of the analog Halls for a follower application.

The drive can run a virtual profiler; the resulted position command of the profiler can also be addressed to a socket, which the drive can then follow. This concept can be used to synchronize several drives to the same profiling path.

Note: The availability of feedbacks is determined by the drive HW configuration.

16.3.10. I/Os

Digital input and digital outputs are available in the drive. Analog inputs are available as well.

Each of the I/Os can be mapped to different available functions.

16.3.10.1. Digital Inputs

- **General purpose.** For a user program or input sensing for the host.
- **Abort.** When this input is logically set, the motor is disabled. Restoring the input to inactive does not enable the servo.
- **Inhibit.** When this input is logically set, the motor is disabled. When the input is logically inactive, the motor is enabled.
- **Homing fast input.** For datum point and fast position capturing input.
- **Reverse limit (RLS).** Prevents movement in the negative direction.
- **Forward limit (FLS).** Prevents movement in the positive direction.
- **Begin on input.** Allows the beginning of a motion when the switch is logically set.
- **General stop (or HW stop).** Prevents any motion in any direction.
- **Soft stop (or SW stop).** Prevents movement from the profiling reference.

Input 1 is dedicated for safety and cannot be modified.

All inputs can be set logically to a positive or negative response.

All inputs can be programmed with HW and SW filters.

All input bits can be read separately (by the **IB[]** command) or simultaneously (by the **IP** command).

**Maestro (G-MAS)**

MAN-G-ADMING (Ver. 1.100)

16.3.10.2. Digital Outputs

- **General purpose.** For setting an output by external host or user program.
- **Amplifier OK.** Indicates that there is no HW fault, such as safety, overtemperature, short protection, overvoltage or undervoltage, which prevents motor enable.
- **Motor on indication.** Indicates that the motor is enabled.
- **Output Compare function.** Fast output on position.
- **Physical brake.** Engage or release a brake with respect to the servo with up to 250 mA.

Outputs are “sources”.

All outputs can be logically set according to the user HW configuration.

Output bits can be set and cleared separately (by the **OB[]** command) or simultaneously (by the **OP** command).

16.3.10.3. Analog input

- **General purpose.** For general usage by the user program.
- **Current mode reference.** The input is used as a reference to the current control loop after conversion by a factor (volts to amperes).
- **Velocity mode reference.** The input is used as a reference to the velocity control loop after conversion by a factor (volts to counts/sec).
- **Position mode reference.** The input is used as a reference to the position loop after conversion by a factor (volts to counts).

Analog inputs are filtered with a programmable low-pass filter.

16.3.10.4. Mode of Operation, Unit Mode and Control Loops

We define the following terms:

- **The unit mode** is the control loop that can be used. It is specified by the value of **UM**.
- **The operation mode** determines the control loop that is required by the user. It is specified by object 0x6060 or the **OF[7]** command.
- **The operation mode display** points out the actual control loop that is being used. It is specified by object 0x6061 or the **OV[2]** command.

The Gold servo drives allow the user to switch between control loops. The switch can be made while the motor is on as long as the unit mode allows it. The following table lists the different available modes:

Maestro (G-MAS)

MAN-G-ADMINING (Ver. 1.100)

Value of UM	Unit Mode	Operating Mode	Related Reference Commands
1	Current	Profile Torque	TC
2	Velocity	Profile Torque Profile Velocity Homing Mode(for future use)	TC JV
5	Position	Profile Torque Profile Velocity Profile Position Interpolated Position Homing Mode	TC JV PA PT (TBD) PV (TBD)

Switching between operation modes can be done at any time. Switching between unit modes the motor must be off.

Note: Unit mode 3 is stepper mode. Stepper mode behaves very much like position mode. However, it introduces some limitation because the mode is an open-loop mode that moves the electric field in internal stepping counts. The rotor follows the electric field, allowing a motion with a constant torque, which is specified by the TC command. The stepping resolution is 512 counts for a motor's pole pair (360 degrees).

Stepper mode is mainly used for internal purposes during wizard and phasing sequences.

16.3.10.5. Units

Several units are available in the Gold line drives.

- **Voltage and Current:** The **BV** and **MC** commands refer to the nominal bus voltage and maximum current, respectively. Their values are HW-dependent and cannot be modified manually. It is mainly used to convert the A2D counts and the physical values (volts and amperes). The drive reports the actual bus voltage and current with respect to these values and presents the physical units to the user (when using, for example, the recorder and the relevant commands such as **AN[6]** for the actual bus voltage and **IQ** for the active current).
- **User Units** are used for velocity and position where the command and feedback can be monitored by user with respect to DS 402 standard factors. These factors and the relation between the units are illustrated in the following table:

**Maestro (G-MAS)**

MAN-G-ADMING (Ver. 1.100)

Item	Description
Ratio between UU and position feedback counts	$\frac{\text{counts UU}}{\text{counts Encoder}} = \frac{[\text{Feed ratio}]}{[\text{Gear ratio}] \times [\text{Position encoder resolution}]}$ $[\text{Feed ratio}] = \frac{[0x6092].1}{[0x6092].2} = \frac{\text{Feed}}{\text{Driving shaft revolutions}}$ $[\text{Gear ratio}] = \frac{[0x6091].1}{[0x6091].2} = \frac{\text{Motor shaft revolutions}}{\text{Driving shaft revolutions}}$ $[\text{Position encoder resolution}] = \frac{[0x608F].1}{[0x608F].2} = \frac{\text{Encoder increments}}{\text{Motor shaft revolutions}}$
Ratio between UU and velocity feedback counts	$\frac{\text{counts UU}}{\text{counts/sec Velocity Encoder}} = \begin{cases} 0x606A=0 & \text{same as } \frac{\text{counts UU}}{\text{counts Position Encoder}} \\ 0x606A=1 & \frac{1}{[\text{Velocity encoder resolution}]} \end{cases}$ $[\text{Velocity encoder resolution}] = \frac{[0x6090].1}{[0x6090].2} = \frac{\text{Encoder increments/sec}}{\text{Motor shaft revolutions/sec}}$
Correct relation between velocity and position encoders	If the sensor selection code [0x606A]=0 (position derivative), the gear ratio may be set 1.0, and the distance user units for velocity/position are similar. If the sensor selection code [0x606A]=1 (separate sensor), the gear ratio may be set so that when the position encoders moves N counts, the speed encoder moves $N \times [\text{Gear Ratio}] / [\text{Feed Ratio}]$ counts

Note: The feed ratio and the gear ratio are separate objects, although only their product matters. As a good design practice, we set the gear ratio so that when the position encoders moves N counts, the velocity encoder moves $N \times [\text{Gear Ratio}]$ counts.

16.3.10.6. Current Versus Torque**16.3.10.7. Standardization**



16.4. Terminology

16.4.1. Command Table Entries

Mnemonic: The reference Id of the command as typed by the host when Elmo's interpreter is used. When square brackets are used ([]) (for example, **IL[]**) the command is an indexed command, and the brackets enclose the index number (such as **IL[1]**). If each index in the command has the same meaning, as in the case of **IF[]**, the size of the index is indicated in the **Mnemonic** column. If each index in the command specifies a different meaning, as in the case of **CA[]**, the description of each index is given in the command appendix.

Description: Information about the command abbreviation (when possible) and some more details about its behavior are given. In this column the set/get/cmd behavior is also mentioned.

Get: The command can retrieve the value of the relevant parameter.

Set: The command allows setting/modification of the relevant parameter.

Cmd: The command is an "execution command," which only performs an action. The user cannot set or get any value from it.

Restriction: Further information about the command and the conditions to activate it.

Note: A get function, if available, has no restriction.

Object Similarity: For each of the parameters/command there is a corresponding CANopen object that represents it. The CANopen object can be used for EtherCAT channel as well (with some restrictions). This column details the objects that have similar behavior. The reference here is only as a concept, and further details are required for using the object (refer to the *Elmo Gold Line CANopen Manual*).

Default: The value of a parameter after power-up reset, software reset (NMT reset) or watchdog reset.

Note: The parameters will be set to their default values if the integrity test of the database fails. The reason for the failure can be retrieved by the CD command. Integrity tests are performed in the following situations:

- Motor enable
- Power on / boot up
- The **SV** command (before the save is performed)
- The **LD** command (after the loading from the non-volatile memory)

16.4.2. Type/Size

Parameter: The parameter that is related to the command is saved in the non-volatile memory. In case of an array, the whole array is saved.

Value: The parameter that is related to the command is a value which is not saved and will be set to the default value after power-up reset or boot up.

**Maestro (G-MAS)**

MAN-G-ADMINING (Ver. 1.100)

Cmd: The command is related to a function, which is executed when the command is called. The command cannot be retrieved or set, since there is no associated parameter.

Int: Integer value. The size of the integer depends on the range.

Float: IEEE translation of float a value. The size depends on the range.

Mixed: The parameter is from an array type, not all entries are of the same type, and they may include integer and float.

Array[x]: The parameter is an array which has x entries. The type of the array can be any of the above types. In some cases different types may be set to one of the indices. In this case the type of the array is "mixed", and the information about the specific entry type is given in the command appendix.

Note: Sometimes not all indices are relevant. A "Bad Index" error will be transmitted on any attempt to address an index which has no valid parameter.

Bit field: The parameter can refer to several pieces of information, which are in the bit level of the parameter. The size of the parameter is typically 32 bits. Bit field parameters have no range.

Static: The value is static or hardcoded. It cannot be modified or changed by any means.

String: The command returns a string (null-terminated). Strings are typically also static.

16.4.3. Range/Unit

Colon (:): A colon (:) indicates a range in the form *from value : to value*.

Units: The units that are presented by the drive. **UU** means user units and will typically have a factor which converts it to internal or physical units. The factor is added to the notation.

Cnt (count): The meaning of a count is internal and depends on the mode and physical units referenced by the drive. The following cases are distinguished:

- In the case of a digital A quad B encoder, a count is the lowest tick that can be read from the encoder. One count is 1/4 of the physical encoder pulse. An encoder that generates 1000 pulses per revolution has 4000 counts.
- In the case of an analog encoder, a feedback count is the outcome of the encoder's physical resolution and the interpolation chosen by the user.
- In case of an absolute digital encoder, a count represents a physical unit, which cannot be interpolated to any other value.
- In case of digital Hall sensor, a count represents 1/6 of a single electrical cycle (an electrical cycle is completed when the magnetic field in the motor turns 360 degrees between a pair of magnetic poles). For example, a motor that has 4 pole pairs will produce 24 counts per revolution.
- In the case of a stepper (**UM=3**), a count is 1/512 of an electrical cycle. For example, if the drive has 4 pole pairs and the user sets the counts to 1024, the motor will move two electrical fields, which in the case of a rotary motor is half a turn.



Maestro (G-MAS)

MAN-G-ADMIING (Ver. 1.100)

Note: If UM=3, when the actual position is read (using the PX command or by directly reading socket read), the values are not the electrical angle and are not in electrical angle units. UM=3 switches the drive to open loop, where the physical feedback must be correlated manually to the electrical angle (and calculated).

Amps: Amperes

16.4.4. General

Object: CANopen and EtherCAT refer to the commands.

Chapter 17: Error Correction

17.1. Introduction

This chapter describes the error mapping feature supported by the Gold drives.

Error mapping is required for correcting nonlinear mechanical position errors.

For a selected socket, error mapping is applied according to an error correction table. A correction table is defined in the following manner:

1. The horizontal series of values in the correction table defines an equally spaced grid of points. These points represent an actual encoder value as read by the sensor. The first position point in the table is defined by **PC[7]**, and it gives the lowest value of the sensor position reading to be corrected. Below this point, the correction value is 0.

The horizontal coordinate of the last point defines the last socket position to be corrected. Above this value, the correction value is 0.

All position points between the first and last points in the table are equally spaced.

2. For each table entry, a value is defined as the number of sensor counts to be added to the sensor position reading.

The corrected position reading is given by the following formula:

$$\text{Corrected Position} = \text{Actual Position} + \text{Correction Value}$$

3. When the actual sensor reading falls between the values of two horizontal positions in the correction table, linear interpolation is used to evaluate the desired correction value.

Notes:

When error mapping is active, the actual sensor encoder reading is transparent to the user. Only the corrected sensor reading is visible through the **PX** command or object 0x6064.

When error mapping is active, the sensor position is dynamically updated by the error-mapping table after time intervals equal to **TS**.

The user can record the sensor's uncorrected reading using the "Socket x HwPosition" variables in the data recording area in the EAS software.

The maximum available size for the correction table is 2048 (using the **ET** array).

The overall corrected position (horizontal coordinate + error) must increase monotonically. This limitation directly implies that the correction table uniquely defines all positions, i.e., for each corrected position there is **one and only one** actual sensor reading that satisfies the following relation:

$$\text{Corrected Position} = \text{Actual Position} + \text{Correction Value}$$

This limitation is not checked by the drive before enabling error mapping. It is up to the user to verify this.

17.2. Correction Table Structure

The correction table selected can be one of the following arrays in the controller:

- The **NT** array with a correction table index range from 1 to 254
- The **ET** array with a correction table index range from 1 to 2048
- The **UI** array with a correction table index range from 1 to 24

All of these arrays are non-volatile.

The array to be used is specified by the **PC[3]** command, and editing of the table is performed using the **GP[N]** command. The table size is derived from the selected array specified in the **PC[3]** command (see the **PC[]** command).

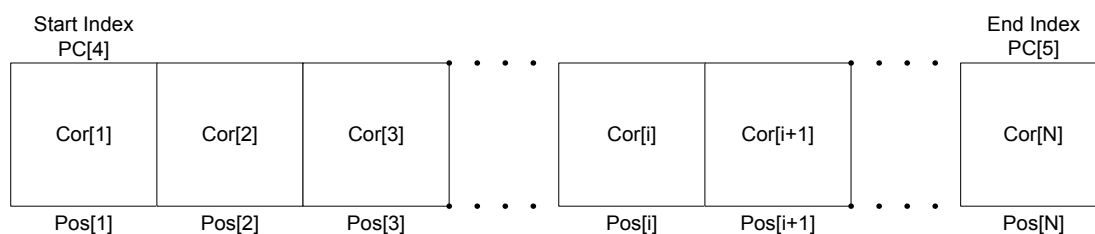
The table indices indicate the sensor positions, starting from the position indicated by **PC[7]**. These positions are defined at equally spaced intervals equal to 2^N , where N , which is specified by **PC[6]**, can be any integer from 3 to 19, between minimum and maximum encoder readings. The values in the table are the position correction values to be added to the sensor positions.

The correction values are signed values between $-2,147,483,647$ and $+2,147,483,647$ counts.

Notes:

If the correction table values are in user units, that is, UU, the values should be converted to sensor values. See the **PC[1]** enable command.

After the table values are converted to sensor values, the drive **does not** make corrections back from sensor units to user units (UU) when the table values are read by the **GP[]** command.



$$\text{Pos}[1] = \text{PC}[7]$$

$$\text{Pos}[i] = \text{PC}[7] + (2^{\text{PC}[6]} \times (i - 1))$$

$\text{Cor}[i]$ – correction value at table index i

$$-2,147,483,647 \leq \text{Cor}[i] \leq 2,147,483,647$$

Figure 23: Correction Table Format

The following table is an example of a correction table and the sensor positions after error correction.

The error-mapping example is defined on socket #1 as follows:

- PC[2]=1 – Use error mapping on socket #1.
- PC[3]=3 – The **UI** array is used for the correction table.
- PC[4]=1 – The start index in the correction table.
- PC[5]=7 – The end index in the correction table.
- PC[6]=8 – The table position spacing is $2^8 = 256$ counts.
- PC[7]=-1000 – The table start position is -1000 counts.

Point No.	Hardware Encoder Reading (Counts)	Error Correction (Encoder Counts)	Actual Position for Servo
	< -1000	Not defined	Actual encoder value
1	-1000	0	-1000
2	-744	10	-734
3	-488	20	-468
4	-232	30	-202
5	24	25	49
6	280	5	285
7	536	-5	531
	> 536	Not defined	Actual encoder value

Table 24: Correction Table Example

17.3. Commands Related to Error Mapping

This section provides general descriptions of the commands that are used to configure and perform error mapping.

Note: For detailed specifications of the commands, refer to the Command Reference for Gold Line Drives.

The following table lists the commands that are used to configure error mapping.

Command	Description in the Context of Error Mapping
PC[N]	Error mapping configuration and operation.
GP[N]	Edit correction table values.

Table 25: Error Mapping Related Commands

The following subsections describe the contributions of these commands to the error mapping mode.

17.3.1. The PC[N] Command

The **PC[N]** command is used to configure and perform error mapping. The error mapping parameters are configured using the **PC[2 to 8]** commands.

PC[1] is responsible for stopping ongoing error mapping or to start error mapping on the configured sensor.

Note: The **PC[]** command is non-volatile, and if **PC[1]** is saved while error mapping is enabled, error mapping will be enabled automatically at power-up.

17.3.2. The GP[N] Command

After the array to be used as the correction table is configured by the **PC[3]** command, the **GP[N]** command enable editing of the correction table without the need to know how the drive stores the correction values internally.

17.4. Error Mapping Configuration

To configure error mapping, follow the steps:

1. Define the sensor socket on which error mapping will be enabled using the **CA[41 to 44]** command.
2. Define the correction table array to be used using the **PC[3]** command.
3. Fill the correction table using the **GP[]** command.
4. Configure all other error mapping parameters.
5. Enable error mapping using the **PC[1]** command.

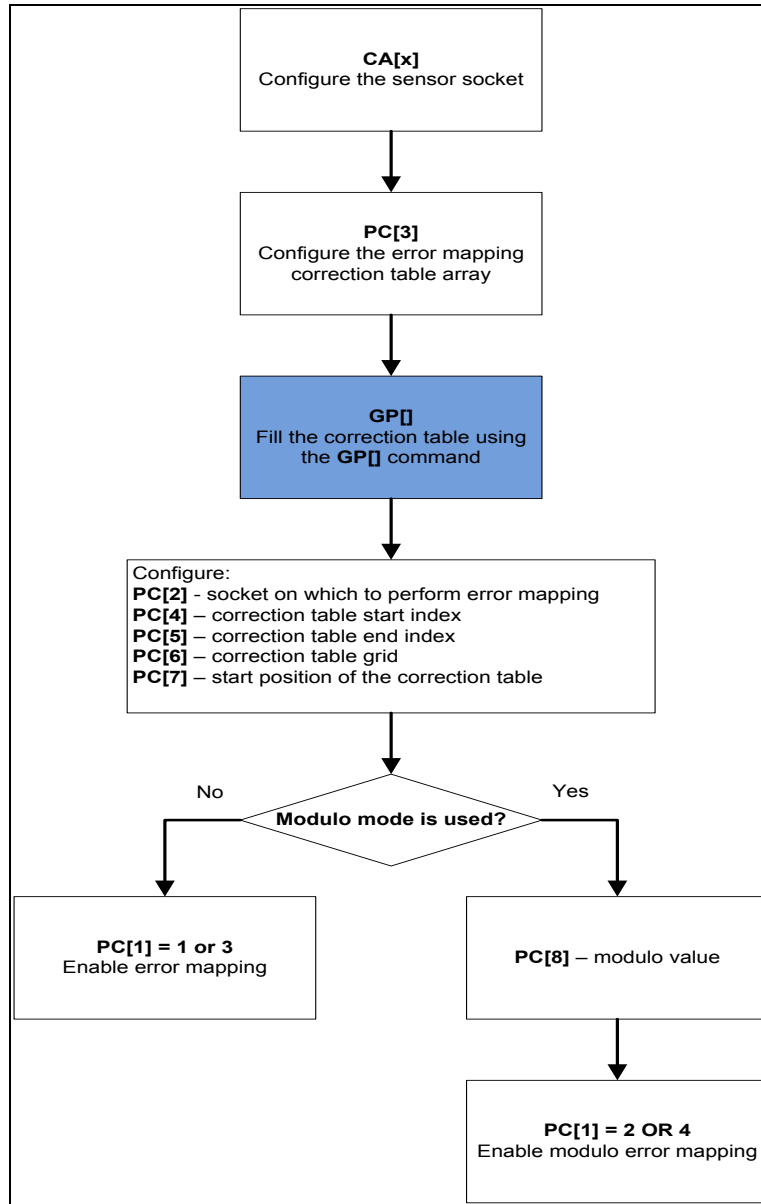


Figure 24: Error Mapping Configuration Flow

17.5. Error Mapping with Modulo

A sensor that needs to repeat the correction table every time that a predefined sensor count is reached should use error mapping with modulo.

In this mode the sensor actual position is converted into modulo between 0 and the value of PC[8]. On the 'Actual modulo position' the correction value is calculated. This correction is added to the 'Actual Position'.

In this mode the corrected position is calculated from the following equation:

$$\text{Corrected Position} = \text{Actual Position} + \text{Corrected Value at Modulo Position}$$

Notes:

The sensor must not exceed the modulo counts in the time configured in the **TS** command.

The sensor can be at any position during enabling of this mode. The true modulo is calculated during enabling of the mode.

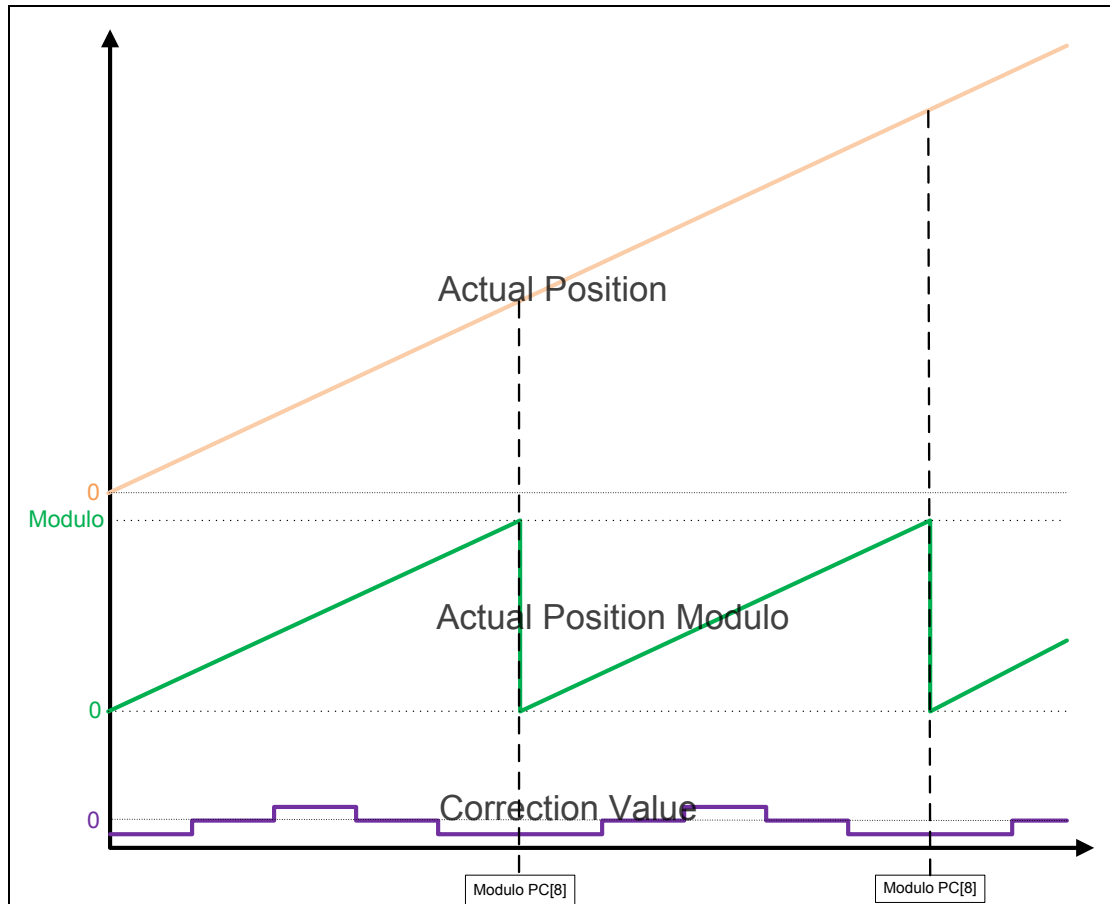


Figure 25: Error Mapping with Modulo

17.6. Error Mapping Comments and Limitations

This section lists the limitations and some comments for error mapping.

Error mapping is supported on one selected socket at a time.

Error mapping cannot be enabled during the following scenarios:

1. While the DS 402 homing motion mode is being configured.
2. **HM** or **HF** is operational with homing mode, i.e., $HM[5] \neq 2$ or $HF[5] \neq 2$.

The following operations cannot be performed while error mapping is enabled:

1. Switching to the DS 402 homing motion mode.
2. Position assignment on the main feedback sensor cannot be performed (**PX** command assignment).
3. **HM** or **HF** with homing mode, i.e., $HM[2] \neq 2$ or $HF[2] \neq 2$.

All captured positions, captured while error mapping is enabled, are reported with the corrected position (the position after the correction is added).

To edit the correction table, use the **GP[]** command. Editing a correction table directly with the selected array interface is not recommended and can lead to unpredictable results.

When error mapping is enabled, starting Output Compare will refer to positions as they were before correction. If Output Compare with table conversion is enabled, the position in the table will be converted using the inverse calculation of the error mapping with a deviation of up to 1 count.

The overall corrected position (abscissa + error) must increase monotonically. This limitation directly implies that the correction table uniquely defines all positions, i.e. for each corrected position there is one and only one actual sensor reading that satisfies the following relation:

$$\text{Corrected Position} = \text{Actual Position} + \text{Correction Value}$$

This limitation is not checked by the drive before enabling error mapping. It is up to the user to verify this.

When using the error mapping with modulo option, the sensor must run less than the modulo counts during every time interval equal to **TS**.

Note: This limitation is not check during the operation of error mapping. It is up to the user to configure the axis speed according to this limitation.

17.7. Error Codes (EC) Related to Error Mapping

The following table lists relevant error codes for error mapping.

EC Value	Description
21	PC[1] command: Value should be from 0 to 4. PC[1] command != 0: Correction table is not selected (PC[3]=0). PC[4] is out of selected correction table low index range. PC[5] is out of selected correction table high index range. PC[5] <= PC[4]. PC[2] is out of range. PC[2] command: Selected socket is out of range. PC[3] command: Selected table number is not supported. PC[6] command: Table grid can be between 3 and 19.

	PC[8] command: Modulo value is < 1
71	PC[1] command != 0: Error mapping cannot be enabled while DS 402 homing is in progress
76	PC[1] command != 0: HM or HF is enabled with a position change at the last event, i.e. HM/HF[5]!=2.
94	PC[1] command !=0: Error mapping is already enabled, disable first. PC[3] command: Cannot change correction table selection while error mapping is enabled. PX command: Assignment to PX while error mapping is enabled. Object 6060 Trying to enter DS 402 homing motion mode while error mapping is enabled.

Table 26: Error Mapping related error codes (EC)

17.8. Error Mapping Examples

This section includes examples of the operation of error mapping.

The examples are in user program language, see the *User Program Manual* (MAN-G-USRPGM).

Example 1:

The example performs the following:

1. Configure socket #1 as AqB on Port A.
2. Initialize the error correction table in the **ET** array.
3. Fill the correction table using the **GP[]** command.
4. Configure other error mapping parameters.
5. Start error mapping.

```
/* Configure socket #1 to AqB on Port A */
CA[41]=2; /* Quadrature on Port A */

/* Select the correction table as the ET array, and fill it with
correction values */
PC[3]=2; /* select correction table on ET array */
for(i=10;i<=100;i++)
{
    GP[i]= i;
}
```



```
/* configure error mapping parameters */
PC[2]=1          /* error mapping on socket #1 */
PC[4]=10         /* correction table array low index is 10 */
PC[5]=100        /* correction table array hi index is 100 */
PC[6]=8          /* correction position grid is 256 */
PC[7]=10,000     /* start correction position is 10,000 */

/* configuration is done, enable error mapping, without converting
correction table values */
PC[1]=1
```

Example 2:

The example performs the following:

1. Configure socket #1 as AqB on Port B.
2. Initialize the error correction table in the **ET** array.
3. Fill the correction table using the **GP[]** command.
4. Configure other error mapping parameters.
5. Configure modulo to 10,000 counts.
6. Start error mapping with modulo.

```
/* Configure socket #1 to AqB Port B */
CA[41]=1; /* Quadrature on Port B */

/* Select the correction table as ET array, and fill it with correction
values */
PC[3]=2; /* select correction table on ET array */
for(i=10;i<=100;i++)
{
    GP[i]= i;
}

/* configure error mapping parameters */
PC[2]=1          /* error mapping on socket #1 */
PC[4]=10         /* correction table array low index is 10 */
PC[5]=100        /* correction table array hi index is 100 */
PC[6]=7          /* correction position grid is 128 */
PC[7]=0          /* start correction position is 0 */
PC[8]=10000      /* error mapping modulo of 10,000 */

/* configuration is done, enable error mapping with modulo and no
converting correction table values */
PC[1]=2
```

Chapter 18: The External Reference Generator

18.1. Introduction

This chapter describes how the drive generates the external motion command for the position controller.

The external position reference may be used in the following applications:

- Positioning a manipulator on a moving object**
 The desired position of the manipulator with respect to the object on the conveyor can be programmed by a ratio factor (follower) or table (ECAM). The position of the conveyor is not known in advance and must be measured online, e.g. using the auxiliary encoder input.
- Synchronizing several drives**
 These may be driven by an external encoder signal (physical or communicated). Each drive uses its follower ratio or/and ECAM table to derive its own motion path from the auxiliary signal.

The external position reference is generated according to the following scheme:

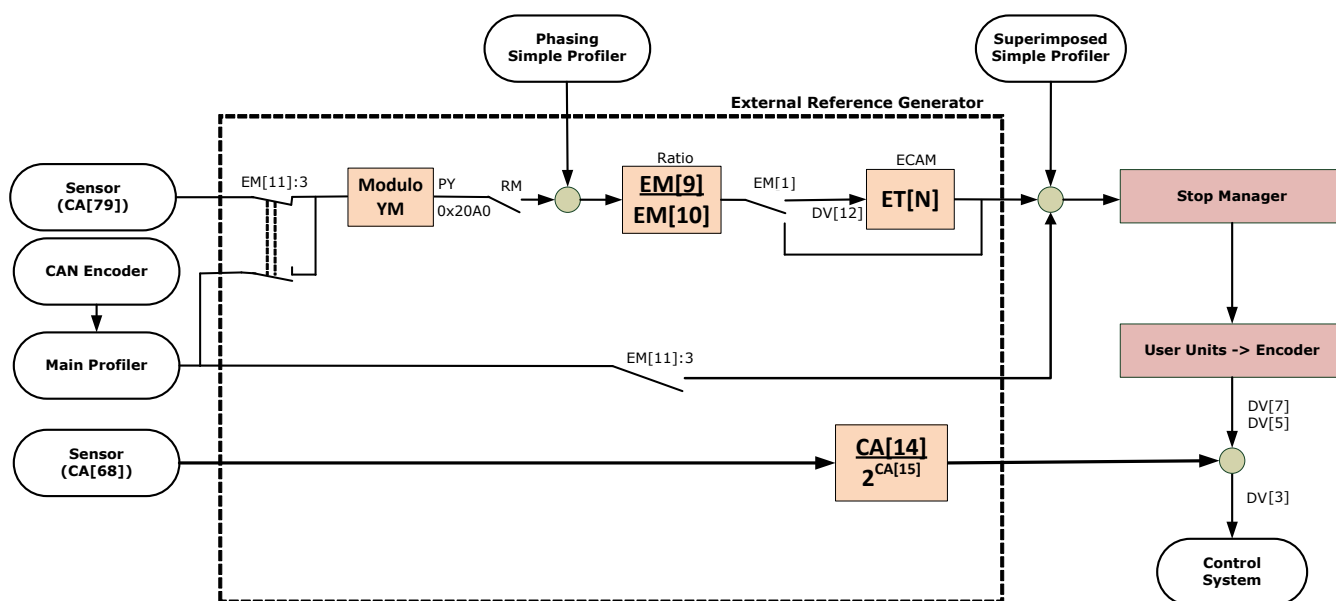


Figure 26: External Position Reference Scheme

The following parameters determine the composition of the position reference:

Parameter	Action
RM	Define whether an external reference is used: RM=0: Do not use external reference RM=1: Use external reference



Parameter	Action
EM[1]	Define whether the ECAM table transforms the external reference: EM[1]=0 : Do not use ECAM table (Follower mode). EM[1]=1 : Use non-periodical ECAM table to transform external command. EM[1]=2 : Use periodical ECAM table to transform external command.
EM[11]:bit3	Define whether the Main Profiler is used as an external reference source: EM[11]:bit3=0 : External encoder is connected to the external position generator EM[11]:bit3=1 : Main Profiler output is connected to the external position generator and disconnected from the control system
EM[9],EM[10]	Scale the external reference input.
CA[14],CA[15]	Scale the socket external reference input
PY	Sets/reports the present position of the external reference in counts

18.2. Commands Related to External Reference Generator

This section provides general descriptions of the commands used to configure the external reference generator.

Note: For detailed specifications of the commands, refer to the Command Reference for Gold Line Drives.

The following table lists the commands used to configure external reference generator.

Command	Description in in the Context of External Reference Generator
EM[N]	External Reference Generator configuration and operation
ET[N]	ECAM Table values
EI	Initialize External Reference Generator
RM	Enable/disable of External Reference Generator
CA[N]	CA[14],CA[15] defines the socket reference ratio
YM[N]	External position reference modulo
KV[N]	KV[61] to KV[70] define the ECAM output low-pass filters (LPF)
EE[6]	ECAM configuration error

Table 27: External Reference Generator Related Commands

The following briefly describe the contributions of these commands. Detailed descriptions can be found in the Command Reference Guide.

- **EM[N] command.** The EM[N] command determines the behavior of external reference generator in Follower and ECAM (Electronic CAM) motion modes.
- **ET[N] command.** ET[N] (N=1...2048) specifies the integer array which is used to store ECAM tables values.
 - The ECAM table array ET[N] can store several distinct movements, with a portion of the table used for each movement. This enables a future movement to be programmed into the drive while the present movement is executing.
 - When the ECAM table is not used, ET[N] can be used by different drive algorithms: error mapping, output compare.
- **EI command.** Parameters defined by EM[N] are synchronously activated when EI command is applied.
- **RM command.** RM command defines whether external reference is used. It does not affect the external sensor reference defined by CA[68].
Setting RM=1 resets ECAM LPF and FIR filters.
- **CA[N] command.**
 - CA[79] defines the external reference sensor used for ECAM/Follower path.
 - CA[68] defines the external sensor used for the fast external referencing which does not pass ECAM and stop manager.

CA[14],CA[15] defines the fast external reference ratio:

$$Ratio = \frac{CA[14]}{2^{CA[15]}}$$
- **YM[N] command.** YM[N] command specifies the counting range for the auxiliary feedback, which is [YM[1]...YM[2]-1].
- **KV[N] command.** KV[61]-KV[65] specify ECAM velocity output LPF. KV[66]-KV[70] specify ECAM acceleration output LPF.
- **EE[6] command.** Returns a value that indicates the cause of the error in case of EC=27.

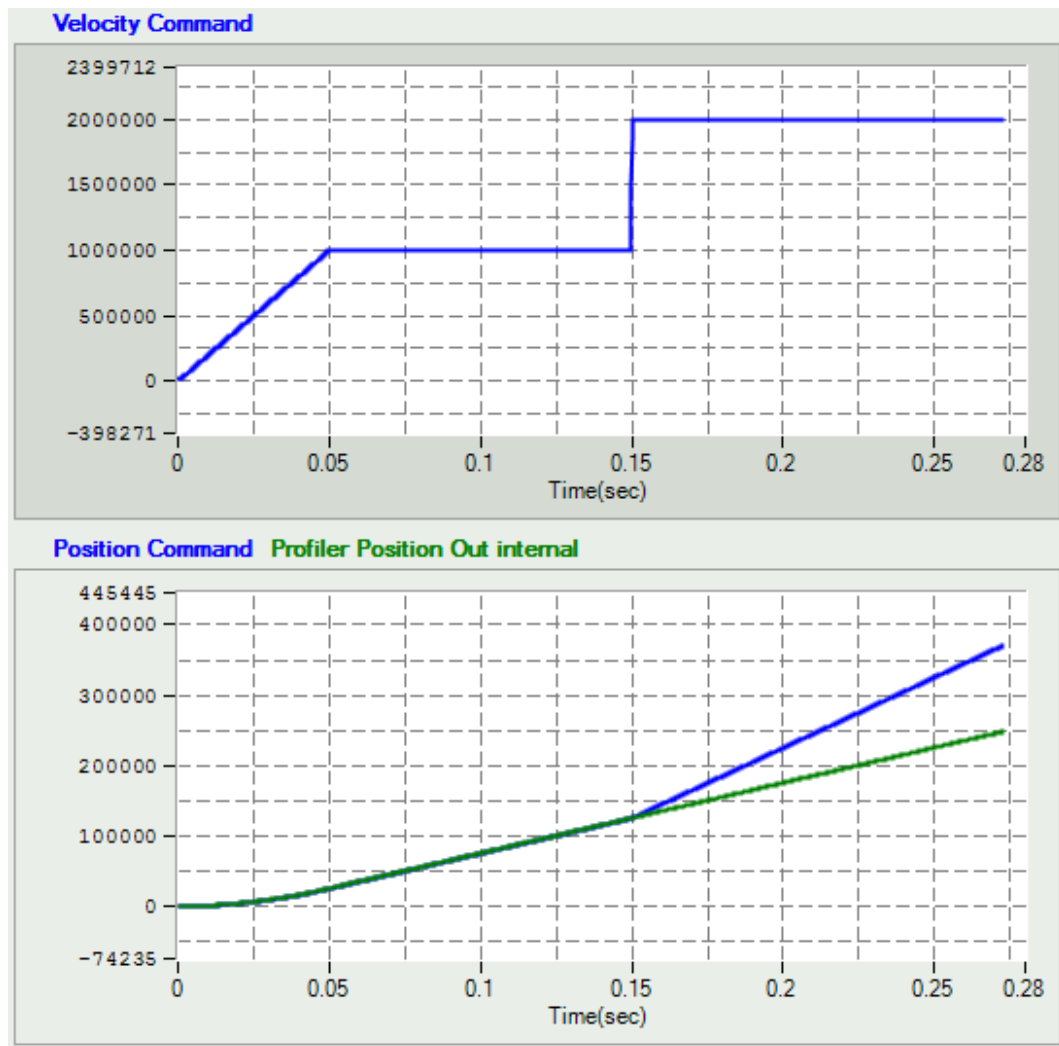
18.3. Follower

In Follower mode (**EM[1]=0**) the master position reference tracks the slave position command at a ratio of **EM[9]/EM[10]**, as depicted in the example below.

Example

The drive program is:

```
// set initial Follower Ratio
em[9]=1;em[10]=1;           // ratio=em[9]/em[10]=1
em[11]=8;                   // use Main Profiler as Virtual Master (set bit3)
em[12]=1;                   // velocity FIR length
em[13]=1;                   // acceleration FIR length
em[1]=0;                    // follower
ei;                          // init external reference generator
rm=1;                       // enable external reference
//configure master reference (using main profiler)
ac=20000000;dc=ac;sd=ac*2;sp=1000000;sf=0;
mo=1;                       // motor ON
jv=1000000;bg;              // run Master
wait(150);
// change Follower Ratio
em[9]=2;                    // ratio=em[9]/em[10]=2
```



In this example, when follower ratio **EM[9]/EM[10]** changes from 1 to 2, the position command starts advancing at twice speed.

18.4. ECAM

ECAM is an acronym for electronic CAM, in which the position reference to the drive is not directly proportional to the external reference, but is rather a function of it.

18.4.1. ECAM table types

Two ECAM table types are supported (see figure below):

- Constant master gap**
 ECAM table array ET describes slave positions. Master positions are defined by gap parameters **EM[4]** and **EM[7]**.
- Variable (non-constant) master gap**
 ECAM table array is divided to two tables, describing master and slave positions separately. The master table should start from 0 (**ET[EM[5]]=0**) and be strictly monotonic rising or falling.

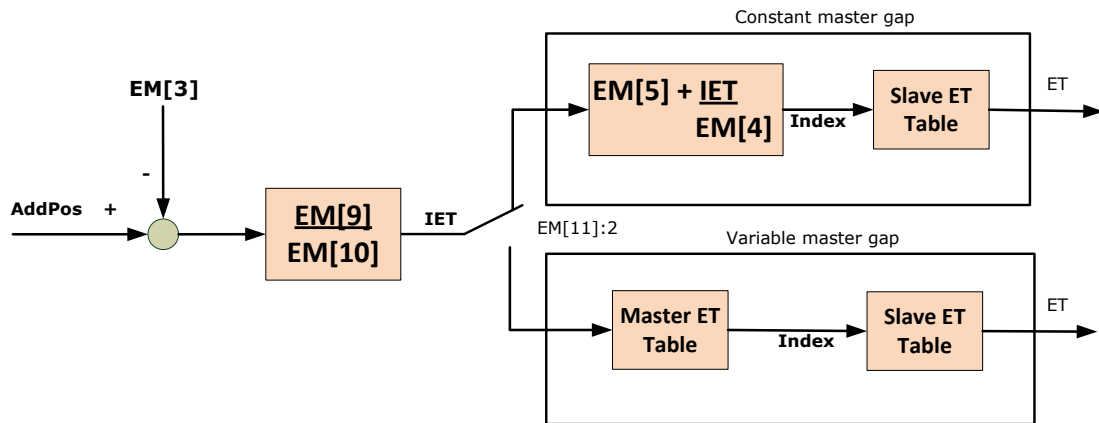
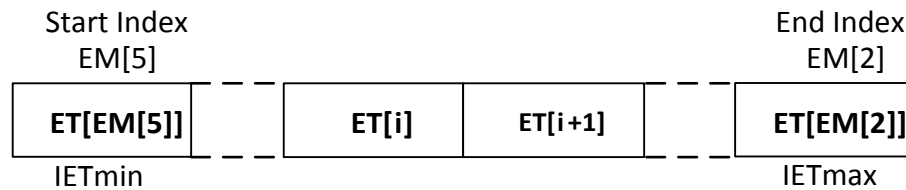


Figure 27: ECAM table types

The input to the ECAM table (IET) is $\frac{EM[9]}{EM[10]} * (AddPos - EM[3])$

18.4.1.1. Constant master gap table (EM[11]:bit2=0)

Constant gap ECAM table contains only slave positions ET[i].



It defines the slave position references for IET values between 0 and IET_{max}:

$$IET_{max} = (EM[2] - EM[5]) * EM[4] - EM[7].$$

Example

The following table is a simple example for the constant master gap ECAM table:

EM[5]=4, EM[2]=8, EM[3]=50, EM[4]=100, EM[7]=25, EM[9]=EM[10]=1.

ECAM table for **ET[N]**:

Index	1	2	3	4	5	6	7	8	9
ET	0	0	0	30	40	50	40	30	0

Calculating the slave position reference corresponding master position $AddPos = 350$:

$$IET = \frac{EM[9]}{EM[10]} * (AddPos - EM[3]) = \frac{1}{1} * (350 - 50) = 300;$$

$$I_{slave} = EM[5] + \frac{IET}{EM[4]} = 4 + \frac{300}{100} = 7;$$

$$ET[I_{slave}] = ET[7] = 40$$

18.4.1.2. Non-Constant Master Gap Table (EM[11]:bit2=1)

Consists of two separate tables: master table and slave table.



where $TL = (EM[2] - EM[5] + 1) / 2$ - table length

The slave table defines the slave position references for IET values between $ET[EM[5]] = 0$ and $IET_{max} = ET[EM[2] - TL]$. IET values are defined by the master table.

Example 1

The following table is a simple example for the master and slave ECAM tables:

EM[5]=1, EM[2]=10, EM[3]=50, EM[9]=EM[10]=1.

Index	1	2	3	4	5	6	7	8	9	10
ET	0	10	20	100	110	0	40	500	100	0

Table length:

$$TL = \frac{EM[2] - EM[5] + 1}{2} = \frac{10 - 1 + 1}{2} = 5;$$

The Master table is defined by ET[1]...ET[5], and slave table – by ET[6]...ET[10].

To calculate the slave reference corresponding to the master position $AddPos = 70$.

ECAM table input is:

$$IET = \frac{EM[9]}{EM[10]} * (AddPos - EM[3]) = \frac{1}{1} * (70 - 50) = 20;$$

The master and slave table indexes for IET=20 are:

$$I_{master} = 3;$$

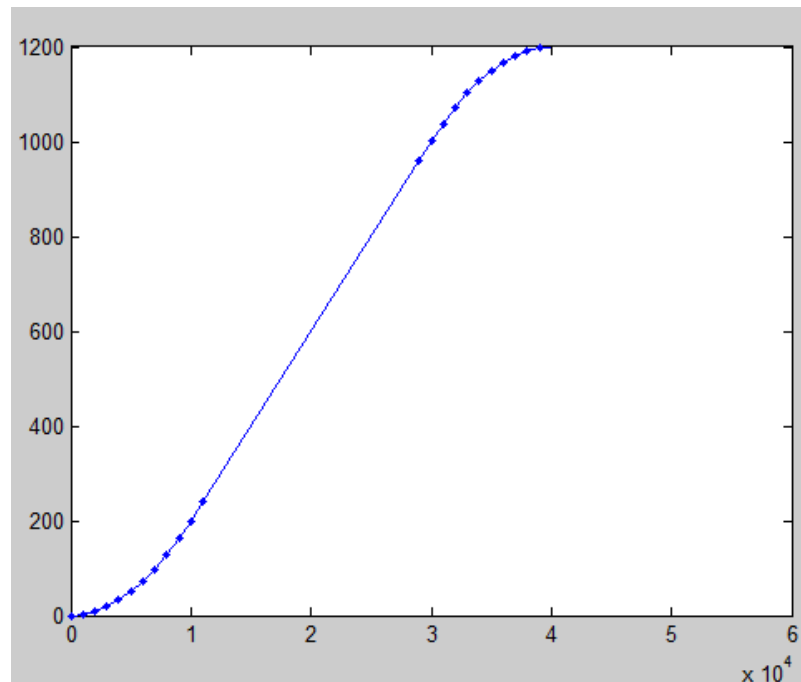
$$I_{slave} = I_{master} + TL = 3 + 5 = 8.$$

Therefore, ECAM position output corresponding $AddPos = 70$ is:

$$ET[I_{slave}] = ET[8] = 500.$$

Example 2

The non-constant master gap helps to exclude multiple segments describing the motion at constant speed from the ECAM table:



18.4.1.3. Table index search (EM[11]:bit5)

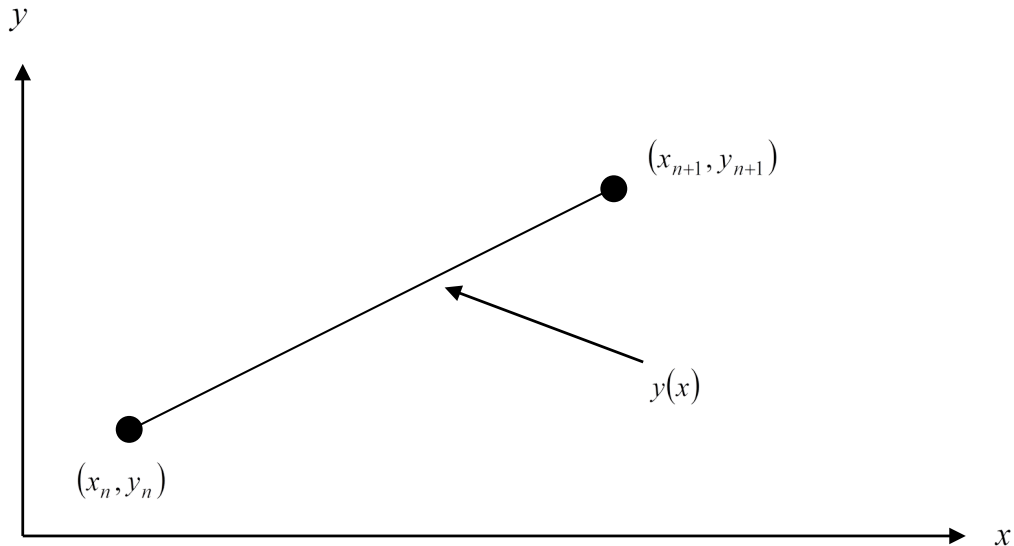
Direct search of the table index I_{master} requires quite significant DSP time, especially when using a master table with a non-constant gap. In order to optimize this time, one of two approaches can be chosen:

- Direct search/calculation of the ECAM table index (**EM[11]:bit5=1**). In this case, the index is calculated every ECAM execution period and consumes extra DSP time.
- Incremental calculation of the ECAM table index (**EM[11]:bit5=0**). Here, the index is directly calculated only one time at the ECAM initialization and, then, it is incremented or decremented when IET crosses the border between two adjacent ECAM table segments. This method assumes that difference between two IET samples does not exceed the segment length.

18.4.2. ECAM table interpolation

ECAM table interpolation is used to find the slave reference command between two adjacent ECAM table points. One of two interpolation methods can be used, linear or quadratic.

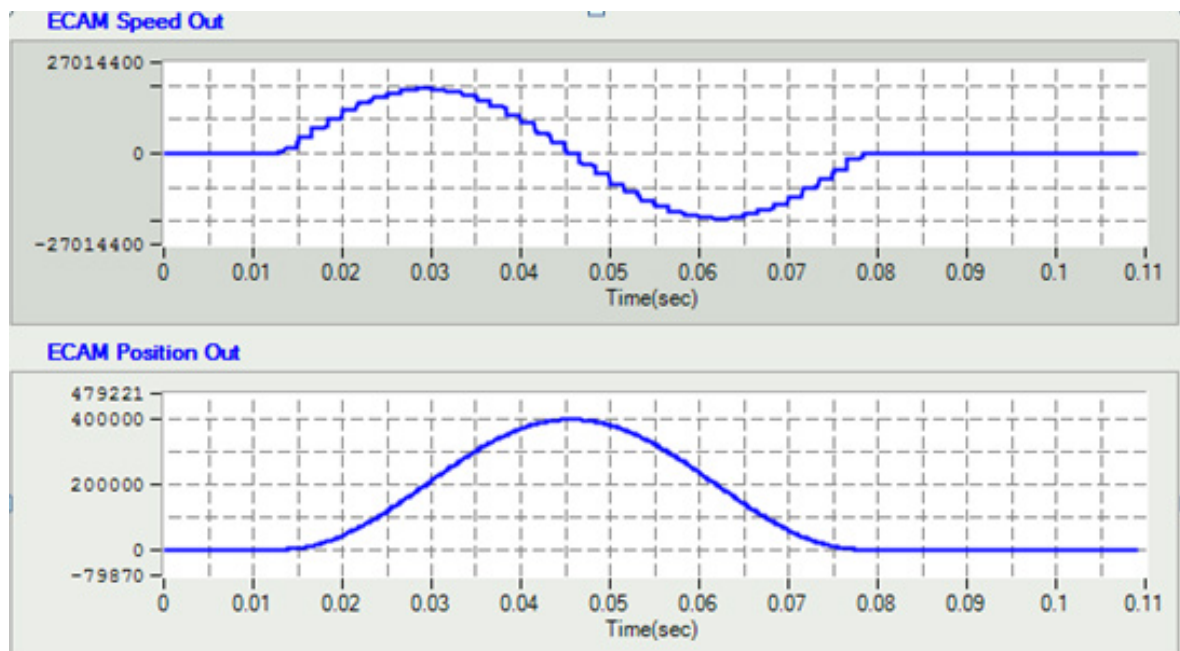
18.4.2.1. Linear interpolation (EM[11]:bit4=1)



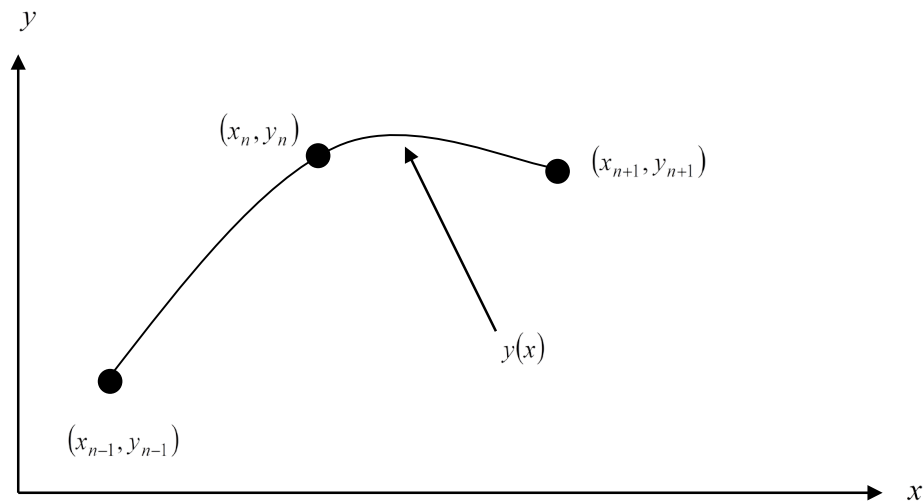
The linear interpolation is employed using two ECAM table points:

$$y(x) = y_n + \frac{y_{n+1} - y_n}{x_{n+1} - x_n} * (x - x_n).$$

Example



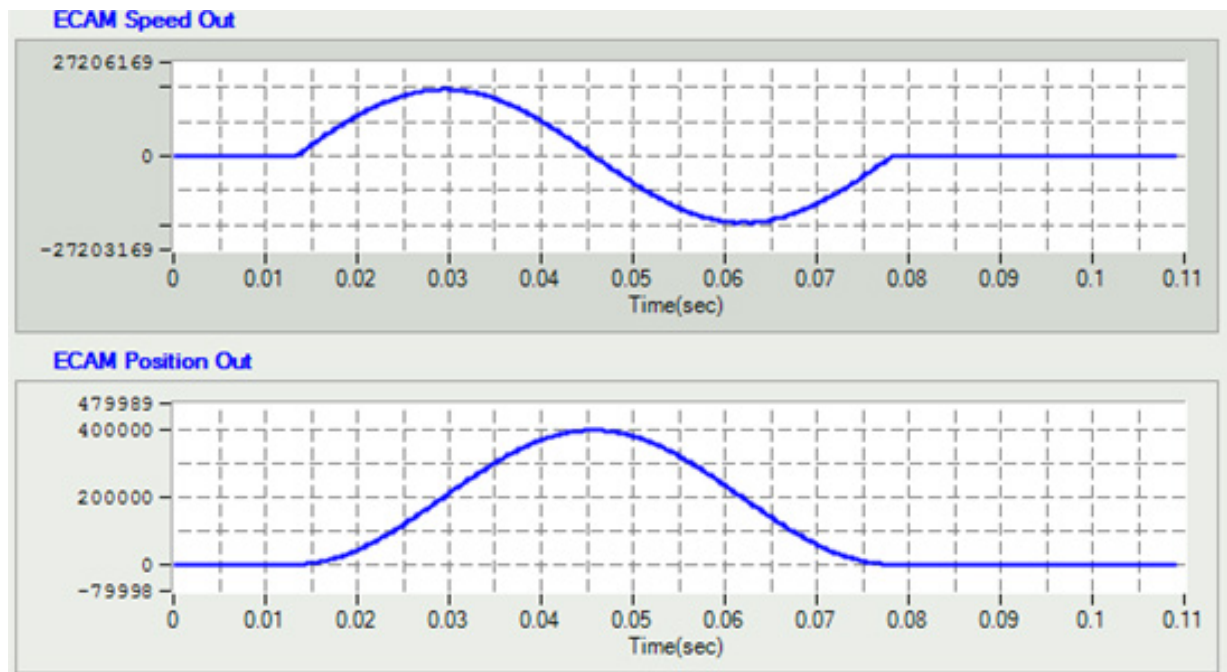
18.4.2.2. Quadratic interpolation (EM[11]:bit4=0)



The quadratic interpolation is employed using three ECAM table points:

$$y(x) = y_n + d_1 * (x - x_n) + d_2 * (x - x_n)^2;$$

Example



18.4.3. Non-periodical ECAM (EM[1]=1)

Non-periodical ECAM is selected by **EM[1]=1**. In this mode the algorithm operates as follows :

- If $IET < 0$, the slave position command (output of the ECAM table) will be **ET[EM[5]]**.
- If $IET > IET_{max}$, the slave position command will be **ET[EM[2]]**.
- If $0 < IET < IET_{max}$, the additional position command will be derived, for example, by linear interpolation of the constant gap ECAM table. The algorithm will then:

1. Search for the low and high table indexes where the IET value is between them:

IET - ECAM table input can be converted

iLo - Index to table where $IET \geq IET[iLo]$;

iHi - Index to table where $IET < IET[iHi]$;

$$iLo = EM[5] + \frac{IET}{EM[4]};$$

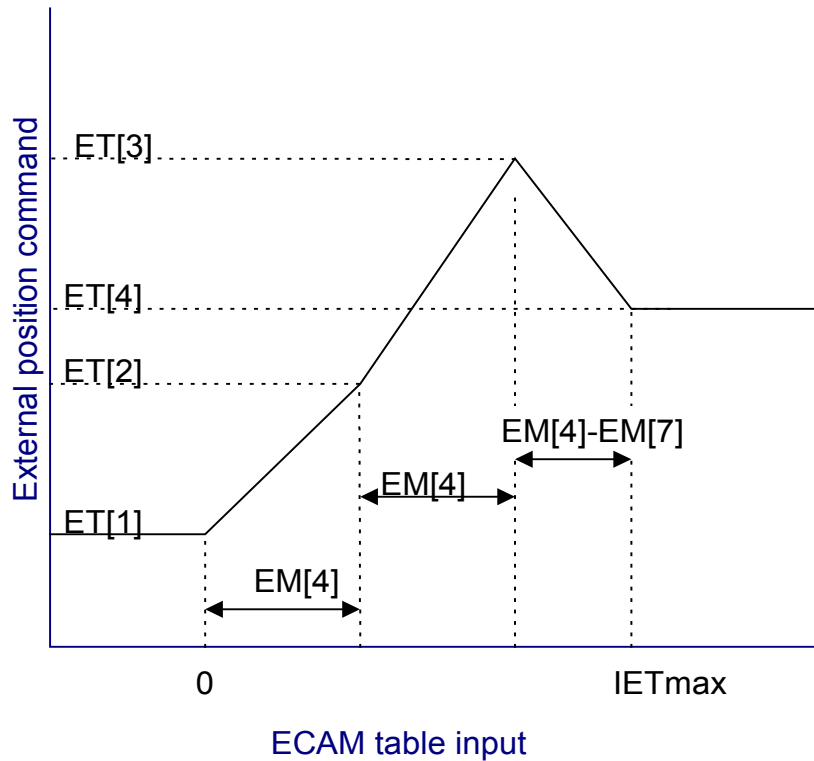
$$iHi = iLo + 1;$$

2. Then calculates the ECAM output command, using interpolation on the two table values with indexes iLo and iHi:

$$ET = ET[iLo] + \frac{ET[iHi] - ET[iLo]}{EM[4]} * (IET - IET[iLo]);$$

If $iLo = (N-1)$ (latest table segment which may be shortened by EM[7]):

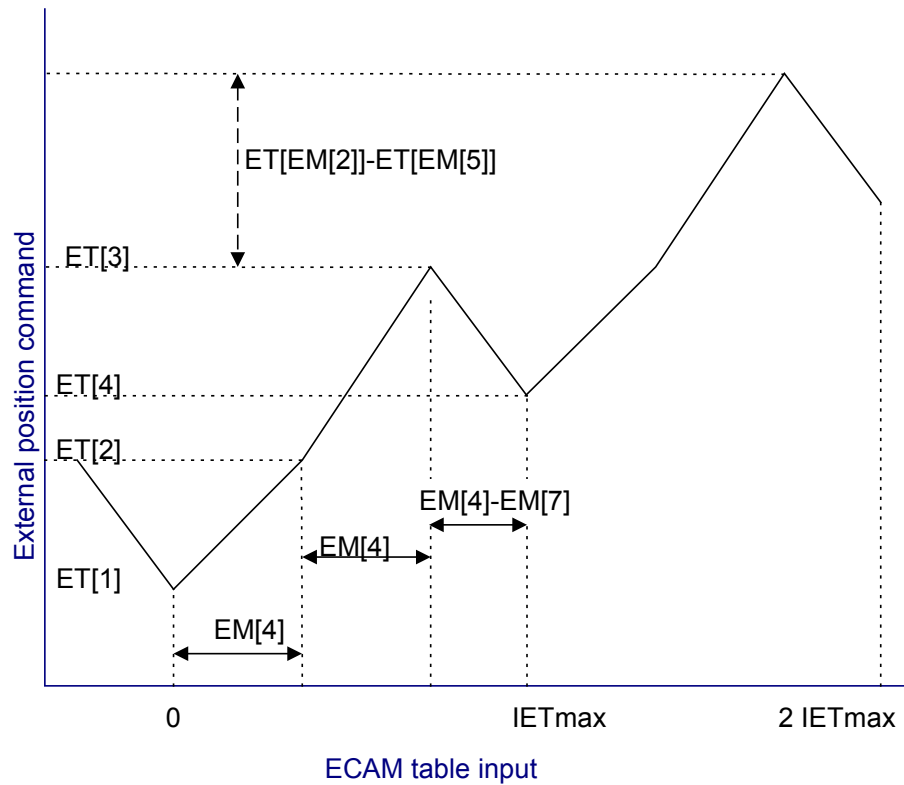
$$ET = ET[iLo] + \frac{ET[N] - ET[iLo]}{EM[4] - EM[7]} * (IET - IET[iLo]).$$



18.4.4. Periodical ECAM (EM[1]=2)

Periodical ECAM mode is selected by EM[1]=2. In this mode, the master can advance indefinitely. The ECAM table defines the slave (motor position) command for one master period, which is the input range to the ECAM table, IET_{max} .

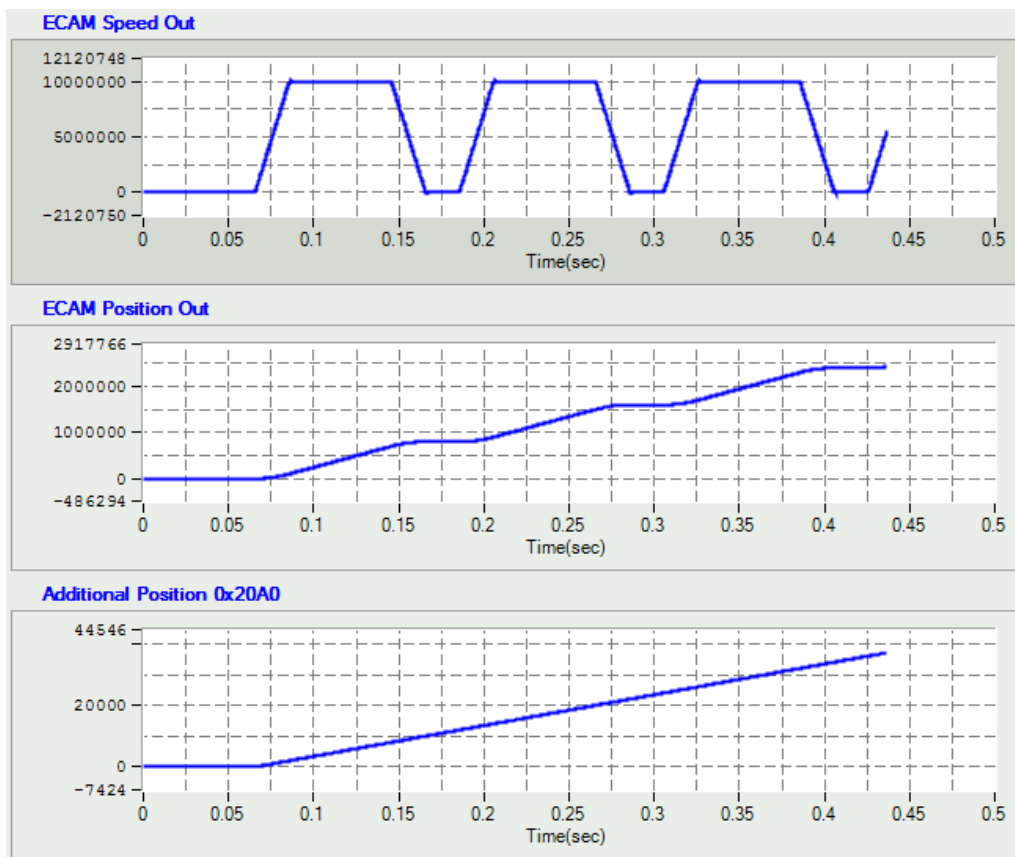
In each master period (in which the master completes a travel of $IET_{max}/ratio$ counts), the slave advances by $ET[EM[2]] - ET[EM[5]]$. The following figure illustrates the behavior of periodical constant gap ECAM for EM[5]=1 and EM[2]=4.



Note that the slave position command is summed from the ECAM table outcome and a cumulative offset, which is $N \times (ET[EM[2]] - ET[EM[5]])$ with N being an integer.

The cumulative offset is lost, when EI command is called (ECAM initialization).

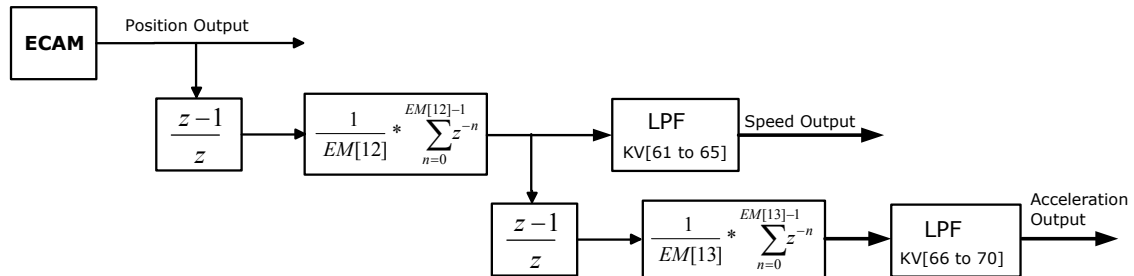
Example



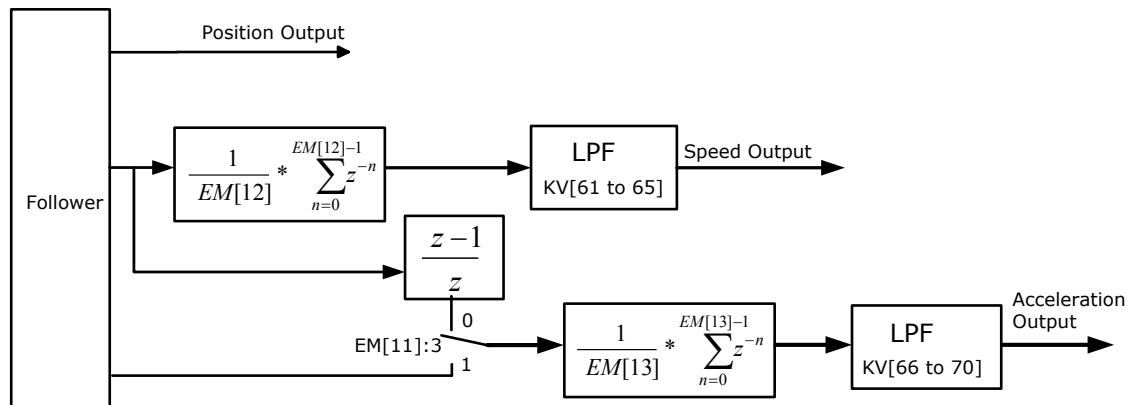
18.5. Speed and Acceleration Output Filtering

ECAM/Follower speed and acceleration outputs should pass through configurable LPF and FIR filters according to the algorithm:

- For ECAM:



- For the Follower:



18.6. External Reference Generator Initialization

ECAM/Follower configuration is initialized every time an EI command is issued.

Note: Changing EM[1] to EM[5], EM[7], EM[10] to EM[11] has no immediate effect. The EI command synchronously activates the entire set of ECAM parameters.

A new ECAM/Follower ratio (EM[9]/EM[10]) can be also activated by setting of EM[9] without issuing the EI command (see example in 1.3).

ECAM speed and acceleration LPFs (KV[61] to KV[70]) are activated at RM=1 command

The ECAM/Follower will be initialized at the drive power-up, if the RM parameter stored in the drive flash memory was set to 1.

18.7. Engage/Disengage

ECAM Engage/Disengage (CamIn/CamOut) can be performed using the RM command, digital inputs, or by setting/resetting ECAM ratio (**EM[9]/EM[10]**).

Examples of user programs depicting engage/disengage sequences are shown below.

18.7.1. Engage/Disengage by RM Command

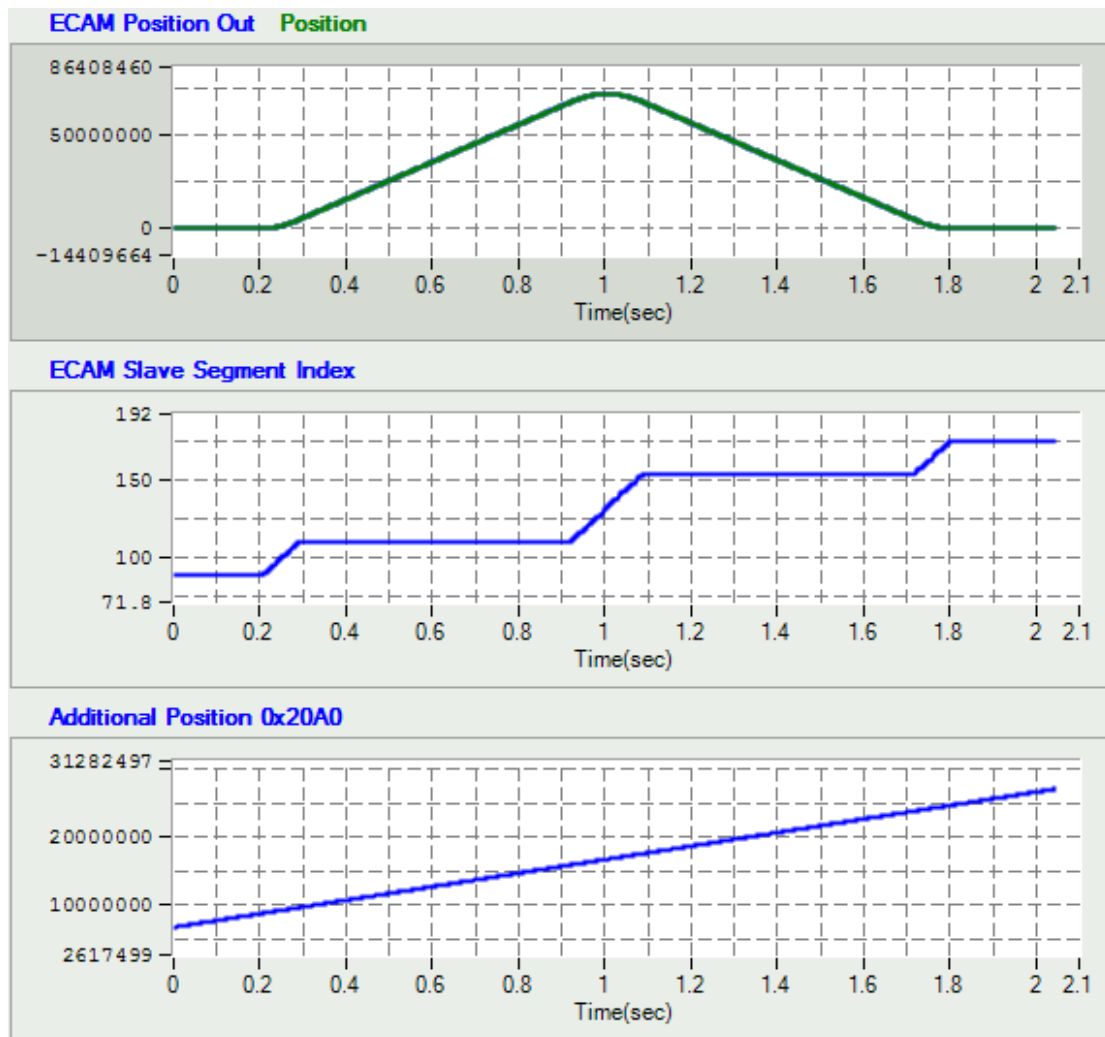
When setting **RM=1**, ECAM/Follower position output jump is prevented. Velocity output can change by step, for example, in Follower mode or when ECAM is enabled in the middle of the table.

After setting **RM=0**, ECAM/Follower position output continues keeping the latest value, velocity and acceleration outputs are set to 0.

Example

The drive program is:

```
rm=0;
em[3]=0;           // ECAM starting position
em[9]=10;em[10]=1; // ratio=em[9]/em[10]=10
em[11]=4+8;        // bit3-master/slave table, bit5-main profiler is master
em[12]=4;           // velocity FIR length
em[13]=1;           // acceleration FIR length
em[1]=1;            // ECAM mode: non-periodical
ei;                 // init external reference generator
mo=1;               // motor ON
pa=200000000;bg;    // run virtual master reference (main profiler)
wait(1000);         // wait some time
rm=1;               // enable external reference - engage ECAM
```



In this example, RM was set to 1 at t=0.2sec.

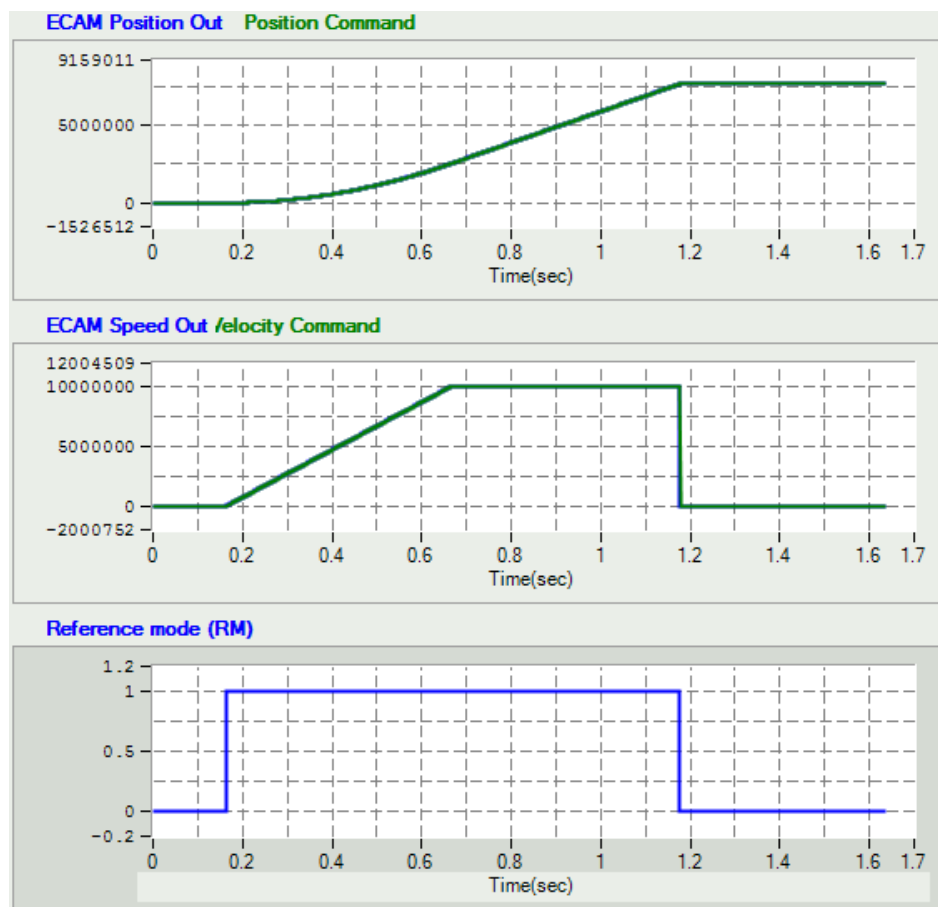
18.7.2. Engage/Disengage by Digital Inputs

In this example, the reference mode (RM) is set by the raising edge of the digital input 1 (ECAM engage) and is cleared by the raising edge of the digital input 2 (ECAM disengage).

Example

The drive program is:

```
il[1]=29;IF[1]=20;          // DI1 - engage
il[2]=31;IF[2]=20;          // DI2 - disengage
em[3]=0;                     // ECAM starting position
em[9]=1;em[10]=1;           // zero ratio=em[9]/em[10]=0
em[11]=4+8;                  // bit3-master/slave table, bit5-main profiler is master
em[12]=1;                    // velocity FIR length
em[13]=1;                    // acceleration FIR length
em[1]=1;                     // ECAM mode: non-periodical
ei;                           // init external reference generator
mo=1;                         // motor ON
jp=1000000;bg;               // run virtual master reference (main profiler)
```

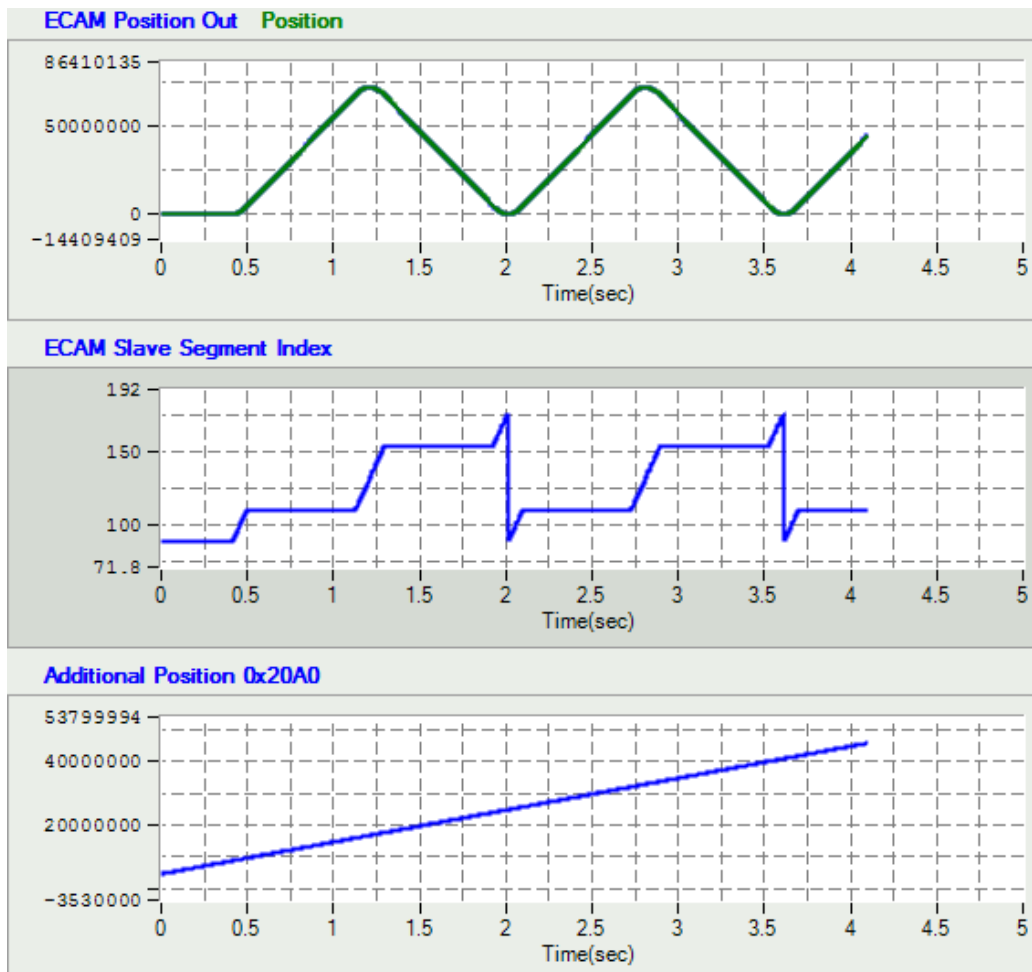


18.7.3. Engage/Disengage by Ratio

Example

The drive program is:

```
em[3]=0;           // ECAM starting position
em[9]=0;em[10]=1;  // zero ratio=em[9]/em[10]=0
em[11]=4+8;        // bit3-master/slave table, bit5-main profiler is master
em[12]=4;          // velocity FIR length
em[13]=1;          // acceleration FIR length
em[1]=2;           // ECAM mode: periodical
ei;               // init external reference generator
mo=1;             // motor ON
jp=10000000;bg;    // run virtual master reference (main profiler)
rm=1;             // enable external reference
wait(1000);       // wait some time
em[9]=10;         // set ratio to operational value - engage ECAM
```



18.8. Jump-Free Motor Starting Rules

Upon starting a motor using the MO=1 command, the motor should never jump. The first and most important reason is safety. The other reason is to avoid an excessive position error fault immediately after the motor is started.

To prevent the motor from jumping, the following actions are done:

- **If EM[11]:bit3=0:**
 - the initial main profiler position command is automatically set to the present position of the motor;
 - the initial superimposed profiler position command is automatically set to the present value of the ECAM output taken with the opposite sign.
- **If EM[11]:bit3=1:**
 - the initial superimposed profiler position command is automatically set to the sum of the present position of the motor and of present value of the ECAM output taken with the opposite sign.

For example:

Suppose that

MO=0,

RM=1,

EM[9]/EM[10]=1,

EM[1]=0, EM[11]=0;

PY=3000

PX=1000 (motor is off, ECAM position command is generated by auxiliary encoder input with a unit follower ratio, the auxiliary position is 3000 and the present position is 1000 counts).

Entering MO=1 will automatically set the main profiler position command to PX=1000 and the superimposed profiler output to:

$$(-(EM[9]/EM[10])) * PY = -1 * 3000 = -3000.$$

Finally, software position command will be DV[7]=PX=1000.

Chapter 19: Troubleshooting